XCP
The standard protocol for the embedded Development

Basics and areas of application

Dipl.-Ing. Andreas Patzer, Dipl.-Ing. Rainer Zaiser
Vector Informatik GmbH

Status 2025 | Reprint only with permission of Vector Informatik GmbH, Holderäckerstr. 36, 70499 Stuttgart, Germany

© 2025 by Vector Informatik GmbH. All rights reserved. This book is intended for personal use only, not for technical or commercial use. It may not be used as a basis for contracts of any kind. All information in this book has been compiled with the greatest possible care. However, Vector Informatik does not warrant or guarantee the accuracy of the information contained herein. Vector Informatik's liability is excluded except for intent and gross negligence unless liability is mandatory by law.

Information in this book may be copyrighted and/or patented. Product names of software, hardware and other product names used in this book may be registered trademarks or otherwise protected by trademark law, whether or not identified as registered trademarks.

Table of contents

Introduction 4
1. Basics of the XCP protocol 8
1.1. The XCP communication model 11
1.2. XCP protocol layer 15
1.2.1. Identification Field 16
1.2.2. The Counter Field 17
1.2.3. Timestamp 17
1.2.4. Data Field 18
1.3. Exchange of CTOs 18
1.3.1. XCP command structure 18
1.3.1.1. CMD 20
1.3.2. RES 23
1.3.3. ERR 23
1.3.4. EV 24
1.3.5. SERV 24
1.3.6. Adjust parameters in the slave 25
1.4. Exchange of DTOs - synchronous data exchange 28
1.4.1. Measurement method: Polling versus DAQ 28
1.4.2. The DAQ-measurement method 29
1.4.3. The STIM-transmission method 37
1.4.4. XCP packet addressing for DAQ and STIM 37
1.4.5. Bypassing = DAQ + STIM 39
1.5. Time correlation and synchronization 40
1.5.1. Introduction 40
1.5.2. XCP's own method for improving time synchronization 40
1.5.3. Support for external time synchronization techniques 40
1.5.3.1. Short introduction to PTP 41
1.5.4. Sharing PTP and XCP 43
1.5.4.1. Scenario 1: Free-running clock in XCP slave 44
1.5.4.2. Scenario 2: XCP slave clock is synchronized with Grandmaster Clock 45
1.5.4.3. Scenario 3: XCP slave clock is syntonized with Grandmaster Clock 45
1.5.4.4. Scenario 4.a: XCP slave with two clocks: Free-running clock combined with a synchronized clock 46
1.5.4.5. Scenario 5: Free-running XCP slave clock and a controller clock 47
1.6. XCP transport layers 47
1.6.1. CAN 48
1.6.2. CAN FD 50
1.6.3. FlexRay 51
1.6.4. Ethernet 55
1.6.4.1. Detecting XCP on Ethernet slaves 56
1.6.5. SxI 57

1.6.6. USB 57
1.6.7. LIN 58
1.7. XCP Services 58
1.7.1. Memory page switching 58
1.7.2. Backup of memory pages - Data Page Freezing 60
1.7.3. Flash programming 60
1.7.4. Automatic detection of the slave 63
1.7.5. Block transfer mode for upload, download and flashing 63
1.7.6. Saving a DAQ configuration and direct data transfer on power-up 64
1.7.7. Protection mechanisms with XCP 64
2. ECU description file A2L 66
2.1. Structure of an A2L file 68
2.2. Manual creation of A2L file 69
2.3. A2L content versus ECU implementation 71
3. Calibration concepts 72
3.1. Parameters in Flash 72
3.2. Parameters in RAM 74
3.3. Flash overlay 75
3.4. Dynamic Flash Overlay Allocation 77
3.5. RAM pointer-based calibration concept according to AUTOSAR 78
3.5.1. Single pointer concept 78
3.5.2. Double pointer concept 80
3.6. Flash pointer-based calibration concept 81
4. Areas of application for XCP 82
4.1. MIL: Model in the Loop 83
4.2. SIL: Software in the Loop 84
4.3. HIL: Hardware-in-the-loop 85
4.4. RCP: Rapid Control Prototyping 85
4.5. Bypassing 86
4.6. Shortening iteration cycles with virtual ECUs 89
4.7. External XCP slaves 90
4.8. Software debugging via XCP 91
5. Exemplary XCP implementation 93
5.1. Description of the functions 94
5.2. Parameterization of the driver 96
6. Overview of the protocol development 98
6.1. XCP Version 1.1 (2008) 98
6.2. XCP Version 1.2 (2013) 98
6.3. XCP Version 1.3 (2015) 98
6.4. XCP Version 1.4 (2017) 99
6.5. XCP Version 1.5 (2017) 100
List of abbreviations 101
Bibliography 103
Web addresses 103
List of Figures 104

Appendix - XCP Solutions at Vector 106
Subject index 107

Introduction

XCP is the standard protocol that enables read and write access to internal memories of embedded systems. It has been established as a universal measurement protocol for many years and allows the parameterization of algorithms in embedded systems at runtime.

XCP was standardized by an ASAM working group (Association for Standardization of Automation and Measuring Systems). Even though its origin is the automotive industry, XCP is useful in all areas with embedded systems. For example, in the aerospace, transportation, or medical industries. In the automotive industry, embedded systems are referred to as ECUs (Electronic Control Unit). Therefore, the two terms will be used as synonyms in the further course.

First of all, some facts about XCP:

- XCP is the abbreviation for "Universal Measurement and Calibration Protocol". The "X" stands for the variable and exchangeable transport layer.
  - Common designation: XCP on CAN, XCP on Ethernet, etc.
- The main applications of XCP are the measurement and calibration of internal ECU variables. The protocol offers the possibility of recording measurement data "event-synchronously" to processes in ECUs. This ensures the consistency of the data among each other.
- XCP is implemented as a single master multi slave concept. The master is a measurement and calibration tool. Usually, the master is a computer application or e.g. a data logger. Slaves are integrated in the embedded system. A master can be connected in parallel with many slaves at the same time, while a slave has only one master at a time.

Accelerate development processes with XCP

Besides code generation, parameterizing the application to a specific embedded system is an iterative task. Parameters are changed, for example, to set the system to a certain state, to simulate a switching state, or to change the behavior of a controller.

In a classic development process, it is necessary to make the changes in the code. For the changes to become active in the embedded system, compilation and flashing must be performed with each change.

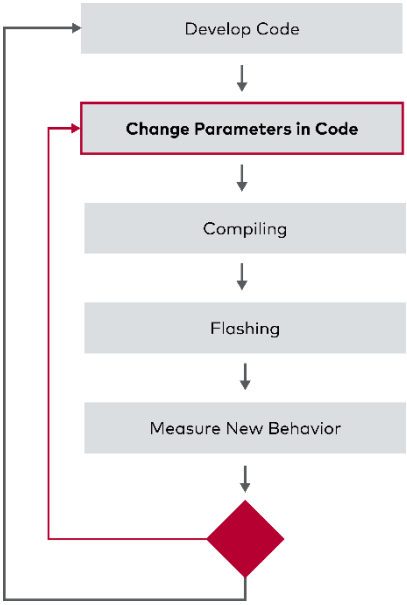


Figure 1 Classic process flow when developing applications for embedded systems

This is different in a development process with XCP:

Any number of parameters can be changed at runtime. Changes are no longer made by making changes in the code with subsequent compiling and flashing, but quickly and easily via the user interface of the XCP master in the memory of the control unit.
To check the system, the measurement data is acquired directly via XCP. The acquisition of the measurement data is triggered by events in the embedded system (e.g. a time sequence). The data are always related to each other in time. The data chosen for measurement is not specified in the code. It is done by the configuration of the measurement and calibration tool.

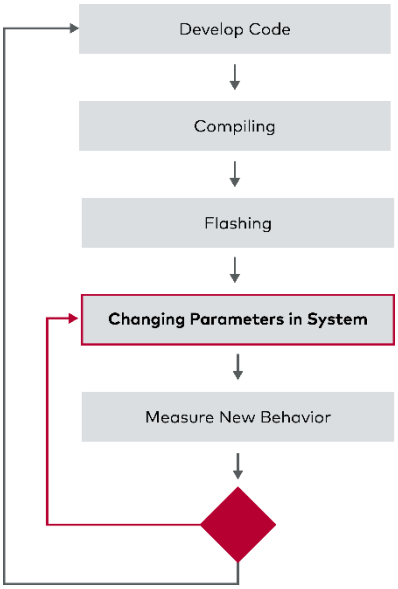


Figure 2 Process flow when developing applications for embedded systems with the XCP standard

The ASAM interface model

XCP is based on the ASAM interface model. The following figure shows the interfaces of a measurement and calibration system as XCP master to the XCP slave in an ECU (Electronic Control Unit), to the description file and the connection to a higher-level automation system.

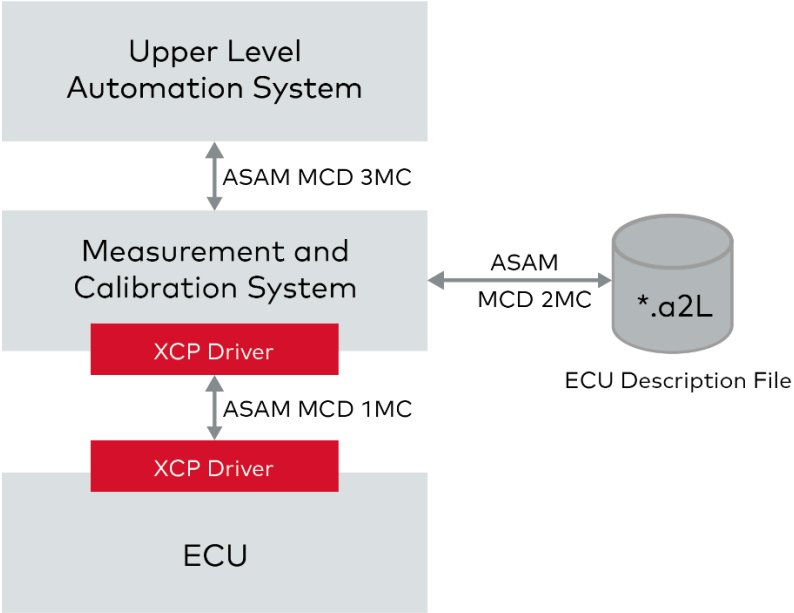


Figure 3 The interface model of ASAM

Interface 1: "ASAM MCD 1MC" between ECU and measuring and calibration system
ECU stands for Electronic Control Unit and is a common abbreviation for an embedded system or control unit. This interface describes the physical and the protocol-specific part. The most common interfaces are XCP on CAN, XCP on CAN FD, XCP on Ethernet and XCP on FlexRay.

Interface 2: "ASAM MCD 2MC" ECU description file A2L
XCP works address oriented. Read or write access to objects in the memory is based on the specification of an address. Ultimately, however, this would mean that the user in the master would have to select his objects in the control unit based on the addresses. This would be highly inconvenient. For the user to work with symbolic object names, among other things, a file is necessary from which the relationship between object name and address can be seen.

Interface 3: "ASAM MCD-3 MC" automation interface
This interface is used to connect another system to the measurement and calibration tool, e.g. for test bench automation. The interface is not explained further in this document, as it is not relevant for understanding XCP.

XCP as an interface throughout the entire development process

The use of XCP is not limited to embedded systems. The protocol can be used in different environments: from a simulated development environment, hardware- and software-in-the-loop environments to embedded systems with AUTOSAR Adaptive or POSIX operating systems. The rest of the book's content is strongly related to ECUs. The behavior of the other systems can be derived very well from it.

This means that XCP is available over a very large part of the development process. Therefore, the same mechanisms, tools, descriptions, etc. are always used. There is less upheaval between work steps. Fewer tools must be purchased and trained. Iteration loops, in which results from later work steps are passed back to previous work steps, can be drastically simplified.

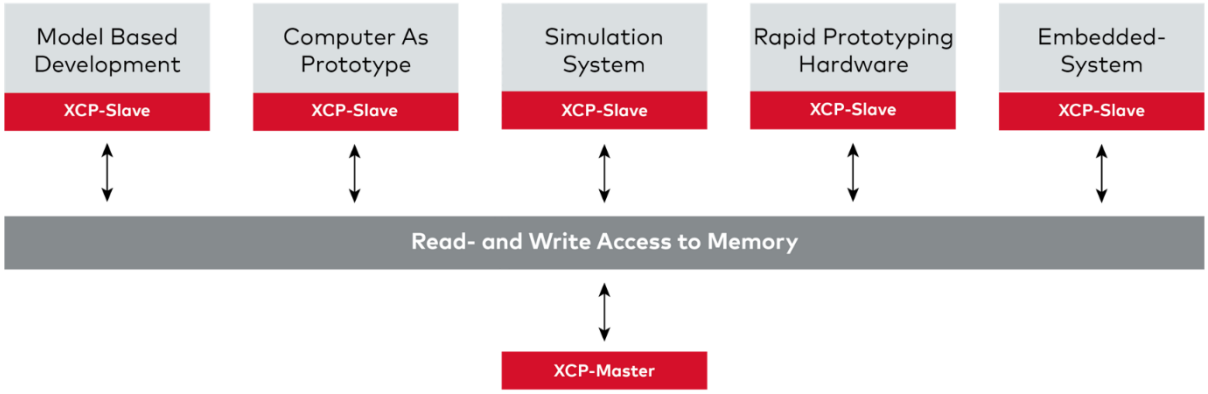


Figure 4 Universal access via the development process

About the book

This book presents the functionality and background of the protocol. It is an explanatory supplement to the XCP standard. This version of the book has been fundamentally revised and refers to the current XCP version 1.5.

Among others, the following topics have been added:

- XCP on CAN FD
- Time correlation and synchronization
- DAQ measurements with Packed Mode

The screenshots used were made with the help of the CANape measurement and calibration tool from Vector Informatik GmbH. CANape is also used to explain other processes, including the creation of an A2L file and much more. You can follow this at any time with a free demo version, which is available in the download center of the Vector website at www.vector.com/canape_demo.

1. Basics of the XCP protocol

Interface 1 of the ASAM interface model describes the sending and receiving of commands and data between slave and master. To realize the independence of a physical transport layer, XCP was divided into a protocol and a transport layer.

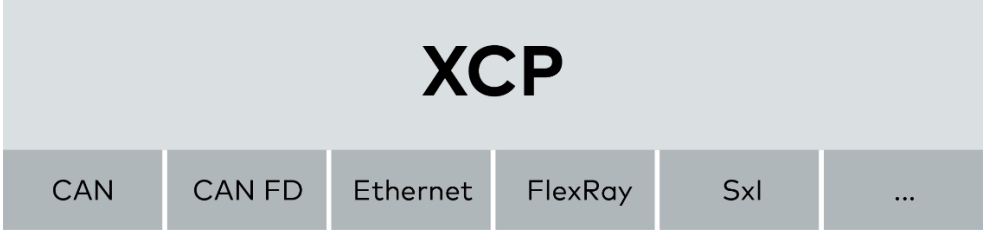


Figure 5 Division of the XCP protocol into protocol and transport layers

Depending on the transport layer, one speaks of XCP on CAN, XCP on Ethernet, etc. The XCP protocol is currently available in version 1.5, which was adopted in 2017.

In designing the protocol, great emphasis was placed on adherence to the following principles:

- Minimal resource consumption in the embedded system
- Efficient communication
- Simple slave implementation
- Plug-and-play configuration with only a small number of parameters
- Scalability

Read and write memory access
The read access offers the possibility to measure the temporal course of a variable internal to the ECU. ECUs are systems with discrete time behavior whose variables change only at certain time intervals: only whenever the processor determines or recalculates the value and updates it in RAM. One of the great strengths of XCP is its ability to capture measurement data from RAM that changes synchronously with processes or events in the control unit. This allows the user to infer the direct correlations between the time sequences in the ECU and the changing values. This is referred to as event-synchronous measurements. The corresponding mechanisms will be explained in detail later.

Write access allows the user to change parameters in the slave. In the case of control algorithms, the controller is thus optimized. However, other parameters in the memory can also be changed, e.g. to achieve a defined status.

The accesses are address-oriented, i.e. the communication between master and slave refers to the addresses in the memory. The measurement of a

quantity therefore represents a request from the master to the slave: "Give me the value of memory location 0x1234". The calibration of a parameter - the write access - to the slave means: "Set the value to 5 at the address 0x9876 ".

Storage Basics

Each type of memory allows data to be read. Writing to a memory is more complex. To be able to change individual variables at runtime in the control unit, there must be a way to access them in write mode. Not every type of memory allows the operation. Only in RAM (Random Access Memory) is read and write access to memory addresses possible without any problems. In the following, a brief outline of memory technologies is intended to clarify the differences between the individual types of memory, knowledge of which is very important for understanding in the further course.

Flash memory are usually integrated into microcontroller chips for control units and are used for permanent storage of code and data even without a power supply. The special feature of flash memory is that while individual bytes can be accessed for reading at any time, writing new content is only ever possible block by block, usually in quite large blocks.

Flash memories have a limited lifetime, which is specified in a maximum number of erase cycles (depending on the technology, this is up to one million cycles). This also corresponds to the maximum number of write cycles, since the memory must be erased block by block before it can be written to again. The reason for this lies in the structure of the memory: electrons are "pumped" via tunnel diodes. The storage of a bit on a memory location works as follows: Electrons must be transported across an electrically insulating layer into the memory location. If the electrons are then behind the insulating layer, they build up an electric field through their charge, which is interpreted as 1 when the memory location is read. If there are no electrons behind the layer, the cell information is interpreted as 0. In this way, a 1 can be set, but not a 0. Setting the 0 (= erasing the 1) is done by a separate erase routine, in which existing electrons behind the insulating layer are diverted. For architectural reasons, however, such an erase routine does not work on byte level, but only on group or block level. Depending on the architecture, blocks of e.g. 128 or 256 bytes are common. If one wants to overwrite a byte within such a block, first the entire block must be deleted. Then the entire content of the block can be rewritten.

If this erasing routine is repeated several times, the insulating layer ("tunnel oxide film") can be damaged. This causes the electrons to flow away slowly and thus the information changes from 1 to 0 over time. For this reason, the number of flash cycles permitted in an ECU is severely limited. In the series ECU, it is often only in the single-digit range. The restriction is imposed by the Flash Boot Loader, which uses a counter to record how many flash operations

have already been performed. If the specified number is exceeded, the Flash Boot Loader rejects a new flash request.

RAM requires a permanent power supply, otherwise it loses its contents. While the flash memory is used to permanently store the application, the RAM is used to temporarily store calculated data and other temporary information. When the power supply is switched off, the contents in the RAM are lost. Unlike flash memory, RAM can be easily accessed for reading and writing.

So let's keep in mind: If parameters are to be changed at runtime, it must be ensured that they are located in RAM. Understanding this fact is important. Therefore, let's look at the flow of an application in the ECU using the following example:

In the application, the quantity y is calculated from the sensor value x.

```
// Pseudocode representation
a = 5;
b = 2;
y = a * x + b;
```

If the application is flashed into the controller, the controller handles this code after booting as follows: The value of x corresponds to a sensor value. So, at some point in time, the application must query the sensor value and the value is then stored in a memory location that is associated with the x. Since this value must be written again and again at runtime, the memory location can only be in RAM.

The quantity y is calculated. The values a and b as factor and offset are stored as information in the flash memory. They are stored there as constants. The value of y must also be stored in the RAM, since write access is only possible there. Exactly where in the RAM the variables x and y or where in the flash a and b are located is determined during the compiler/linker run. Here the unique assignment of the objects to addresses takes place. The relationship between object name, data type and address is documented in the linker MAP file. The linker MAP file is generated by the linker run and can exist in different formats. However, all formats have in common that at least object name and address are contained in it.

If the offset b and the factor a in the example depend on the vehicle, the values of a and b must be individually adapted to the conditions. This means that the algorithm remains as it is, but the parameter values change from vehicle to vehicle.

In the normal operating mode of a control unit, the application runs out of the flash memory. This does not allow write access to individual objects. This means that parameter values located in the flash area cannot be changed at

runtime. If it is to be possible to change parameter values during runtime, the variables to be changed must be in RAM and not in flash. How do the parameters and their initial values get into the RAM? How do you solve the problem if more parameters are to be changed than can be accommodated in RAM at the same time? These questions lead us to the topic of calibration concepts.

Summary of XCP basics

The mechanisms of the XCP protocol provide read and write access to memory contents. The access is address oriented. The read access allows the measurement of variables from the RAM and the write access allows the calibration of the parameters in the RAM. XCP allows the measurement to be carried out synchronously with the events in the control unit. This ensures that the measured values correlate with each other. Each time a measurement is restarted, the signals to be measured can be freely selected. For write access, the parameters that are to be adjusted must be in RAM. This requires a calibration concept.

This raises two important questions:

> How does the user of the XCP protocol know the correct address of the measurement and calibration variables in RAM?
> What does the calibration concept look like?

The questions are answered in the chapters 2. ECU description file A2L and 3. Calibration concepts.

1.1. The XCP communication model

XCP data is exchanged between master and slave on a frame or packet basis. A distinction is made between the exchange of commands (CTO) and the sending of synchronous data (DTO).

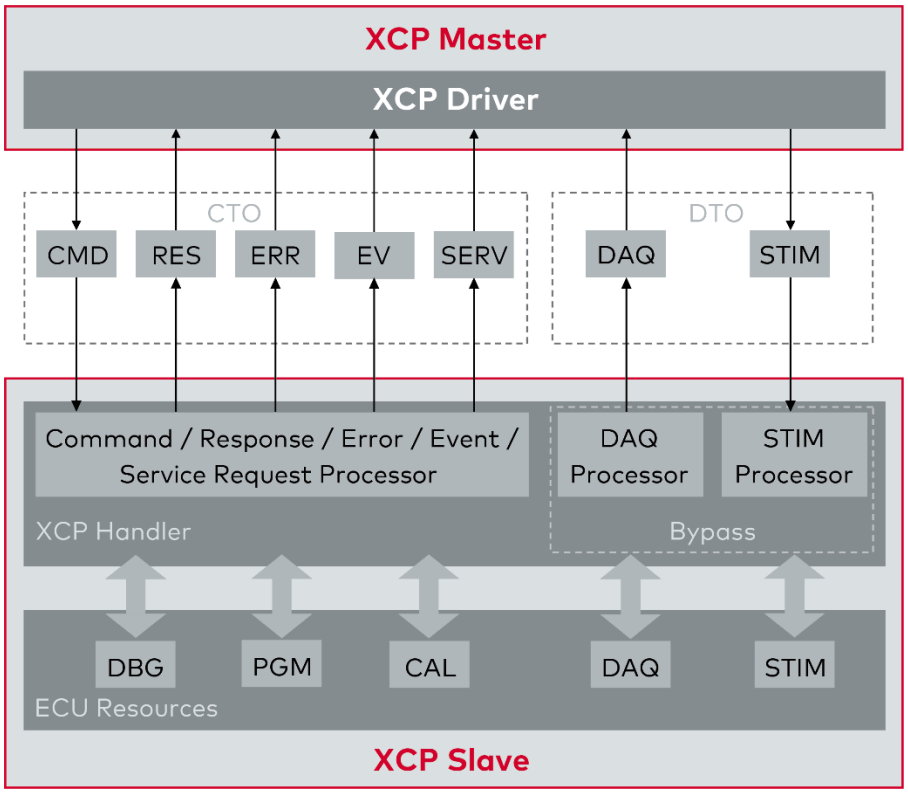


Figure 6 XCP communication model with CTO and DTO

The abbreviations used stand for:

CMD	Command Packet	Send commands
RES	Command Response Packet	positive answer
ERR	Error	negative answer
EV	Event Packet	asynchronous event
SERV	Service Request Packet	Service request
DAQ	Data AcQuisition	Send cyclic measurement data
STIM	Stimulation	cyclic stimulation of the slave
DBG	Debugging	Debugging via XCP slave
PGM	Programming	Flashing new program or data status into the control unit
CAL	Calibration / Paging	Changing parameters or switching the calibration pages

Commands are exchanged via CTOs (Command Transfer Objects). This is how the master establishes contact, for example. A CMD must always be answered by the slave with RES or ERR. The other CTO frames are sent asynchronously. The Data Transfer Objects (DTO) are used for the exchange of synchronous measurement and stimulation data.

Exchange of status information between XCP master and slave

In more complex ECUs, the XCP slave and the ECU application can be different instances. For example, the XCP slave can be active without the ECU application running. Or one of the ECU resources can assume an active or inactive state. The XCP slave can inform the XCP master about this.

The XCP slave must support the GET_STATUS command. The slave responds to the master with information about the current status, which resources are protected via seed-and-key, and other details.

Optional: If the XCP slave supports asynchronous event messages, it sends the EV_ECU_STATE_CHANGE event to the XCP master and can thereby inform it that there has been a status change. The XCP master can then query the details again using the GET_STATUS command.

Standard Communication Mode

The master sends a request and waits for the response of the slave before sending another command.

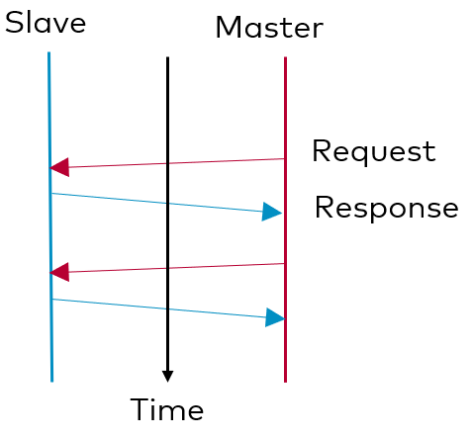


Figure 7 Standard CTO exchange

Master Block Transfer Mode

The master can send several commands in succession to the slave without waiting for an individual response from the slave to each command.
The block transfer mode is optional and speeds up the process for large data transfers (for example, upload or download processes). However, this mode's performance must be weighed against the slave's capabilities. This includes the minimum time interval MIN_ST between two commands and the maximum number of commands without response MAX_BS. Optionally these communication settings can be read out from the slave by the master via GET_COMM_MODE_INFO.

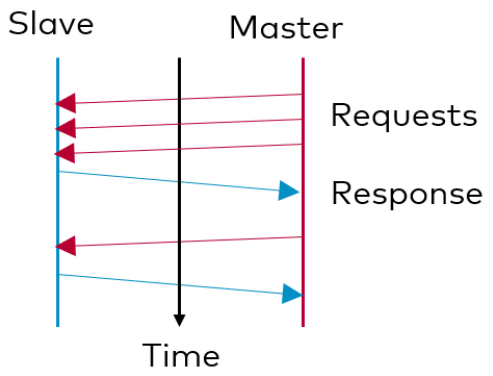


Figure 8 Master Block Transfer Mode

Slave Block Transfer Mode

Also, for time optimization the slave can send several frames one after the other to the master. For example, to transfer a new data status from the slave. The XCP command concerned is UPLOAD.
The limitations mentioned for the Master Block Transfer Mode do not have to be observed for the Block Transfer Mode in the direction of the master. The concept is based on the fact that the performance of the master is always sufficient.

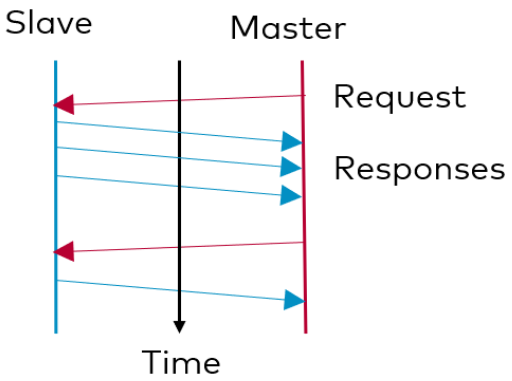


Figure 9 Slave Block Transfer Mode

Interleaved Communication Mode

In standard communication, the master and slave wait for each other. The interleaved communication mode is available to accelerate the data transmission. Here, the master may send several requests in succession. It must consider the size of the receive buffer on the slave side. The slave can send several responses in succession to the master. The slave does not have to consider any limitation of the master.

The interleaved mode must not be used simultaneously with a block mode. This mode is optional and - in contrast to the block transfer mode - has no practical relevance.

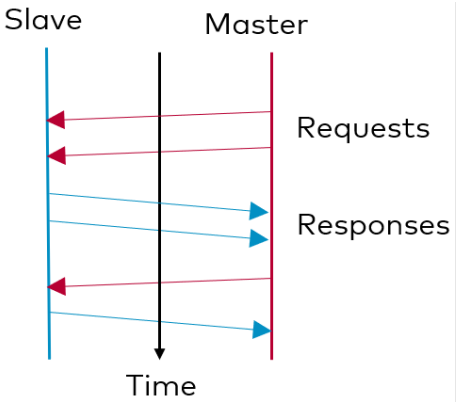


Figure 10 Interleaved Communication Mode

1.2. XCP protocol layer

The XCP frame consists of three parts: the XCP header, the XCP packet and the XCP tail. XCP Header and XCP Tail depend on the transport protocol. For example, an XCP on CAN packet is enclosed by a CAN frame.

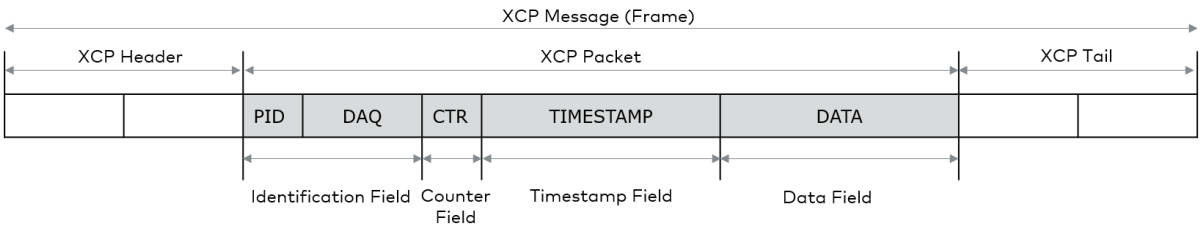


Figure 11 XCP package

The structure of the XCP package is independent of the transport protocol used.
An XCP packet consists of an "Identification Field", an optional "Counter Field", an optional "Timestamp Field" and a "Data Field".

The maximum XCP packet size depends on the transport layer. It is set separately for the CTO and for the DTO packets:
MAX_CTO specifies the maximum length of a CTO packet in bytes.
MAX_DTO specifies the maximum length of a DTO packet in bytes.

1.2.1. Identification Field

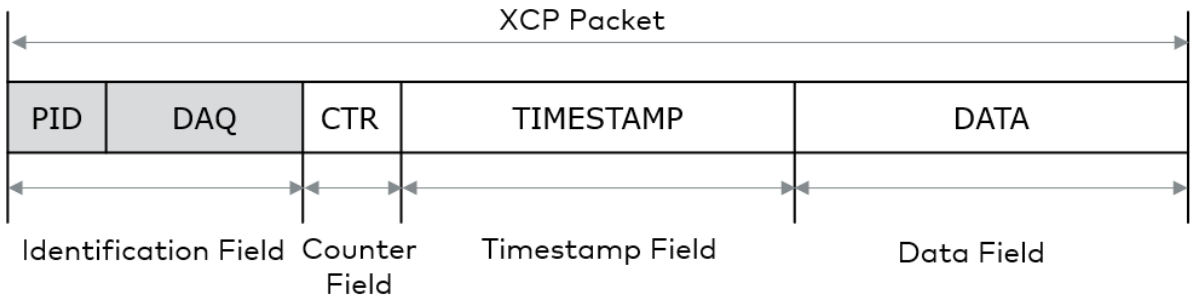


Figure 12 The package identification

When exchanging packets, both master and slave must be able to recognize which information was sent by the other. This process is done with the help of the identification field. Therefore, each packet starts with the packet identifier (PID).

When transmitting CTOs, the PID field is completely sufficient to identify a CMD, RES, or another CTO packet. In Figure 13 Overview of the XCP Packet Identifier (PID) the commands from the master to the slave use a PID from 0xC0 to 0xFF. The XCP slave responds or informs the master with PIDs from 0xFC to 0xFF. This means that there is a unique assignment of the PID to the CTOs sent in each case.

If DTOs are transmitted, further components of the identification field are used (see chapter XCP packet addressing for DAQ and STIM). The counter field CTR is also described in this context.

The following overview shows which PIDs are defined:

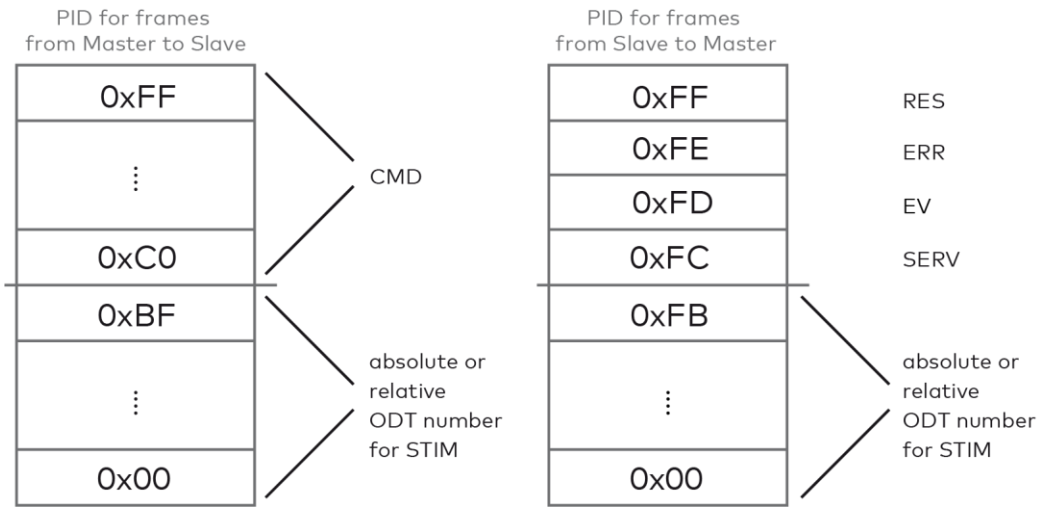


Figure 13 Overview of the XCP Packet Identifier (PID)

1.2.2. The Counter Field

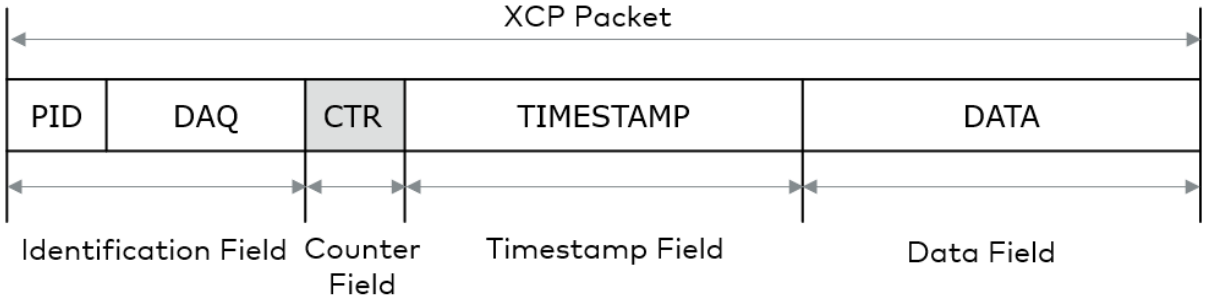


Figure 14 Counter Field

A DTO package can optionally contain a counter field. If present, the counter field is located directly after the identification field and has the size BYTE.

If the counter field is used when transmitting DAQ packets, the XCP slave inserts a counter in the first ODT packet of a DAQ list. Which counter is inserted is a property of the event for which this DAQ list is configured.

1.2.3. Timestamp

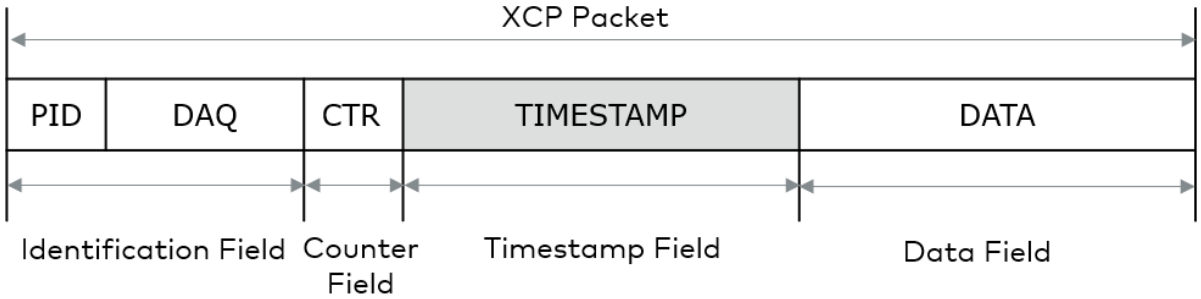


Figure 15 Timestamp

DTO packets can be provided with a timestamp. BYTE, WORD and DWORD are available as data types for timestamps. A CTO packet is transmitted without a timestamp.

In the case of measurement data transmission via DAQ, the XCP slave writes the measurement time to the timestamp field. Since all measured variables in a DAQ list are acquired at the same time, the timestamp is transmitted only once per DAQ cycle.

During STIM data transfer from the master to the slave, the XCP master writes a value to the timestamp field. The XCP standard does not define the meaning of the value.
More information about handling timestamps is explained in the chapter Exchange of DTOs - synchronous data exchange.

1.2.4. Data Field

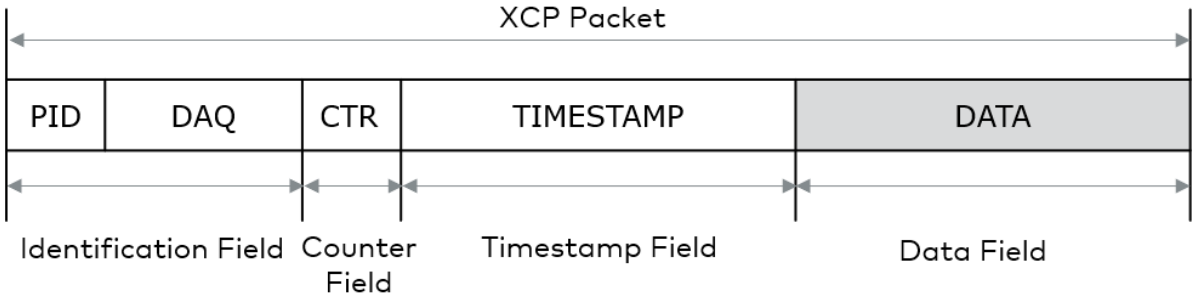


Figure 16 Data field in XCP package

The actual data is transferred in the data field. In the case of CTO packets, the data field consists of specific parameters for the different commands. DTO packets contain the measured values from the slave and in the case of sending STIM data the values from the master.

1.3. Exchange of CTOs

Commands from the master to the slave as well as the responses and messages from the slave to the master are transmitted via CTOs.

1.3.1. XCP command structure

The slave receives a command from the master and must react to it with a positive or negative response. The communication structure is always the same:

Command (CMD):

Position	Type	Description
0	BYTE	Packet Identifier CMD (0xC0...0xFF)
1.. MAX_CTO-1	BYTE	Command Data

Each command is assigned a unique number. In addition, further specific parameters can also be transmitted. The maximum number of parameters is defined as MAX_CTO-1. MAX_CTO specifies the maximum length of the CTO packets in bytes.

Positive answer:

Position	Type	Description
0	BYTE	Command Positive Response Packet Code = RES 0xFF
1..MAX_CTO-1	BYTE	Command specific Parameters

Negative answer:

Position	Type	Description
0	BYTE	Error Packet Code = 0xFE
1	BYTE	Error code
2..MAX_CTO-1	BYTE	Command specific Parameters

Not only at the positive but also at the negative response specific parameters can be transmitted as additional information. The establishment of the connection between master and slave shall serve as an example. At the beginning of a communication between master and slave, the master sends a connect request to the slave, which the slave must answer positively to establish a continuous point-to-point connection.

Master → Slave: Connect
Slave → Master: positive response

Connect command:

Position	Type	Description
0	BYTE	Command Code = 0xFF
1	BYTE	Mode 00 = Normal 01 = user defined

Mode 00 means that the master requests an XCP communication with the slave. If the master uses 0xFF 0x01 when establishing the connection, the master requests an XCP communication with the slave. At the same time, it informs the slave that it should change to a certain - user defined - mode.

Positive response from the slave:

Position	Type	Description
0	BYTE	Packet ID: 0xFF
1	BYTE	RESOURCE
2	BYTE	COMM_MODE_BASIC
3	BYTE	MAX_CTO, Maximum CTO size [BYTE]
4	WORD	MAX_DTO, Maximum DTO size [BYTE]
6	BYTE	XCP Protocol Layer Version Number (most significant byte only)
7	BYTE	XCP Transport Layer Version Number (most significant byte only)

The positive response of the slave can be somewhat more extensive. The slave already informs the master about communication-specific information when the connection is established. RESOURCE, for example, is information with which the slave reports whether it supports page switching or whether flashing via XCP is possible. With MAX_DTO the slave informs the master about the maximum packet length for the measurement data transfer supported by the slave and so on. The details of the parameters can be found in the ASAM XCP Part 2 Protocol Layer Specification.

1.3.1.1. CMD

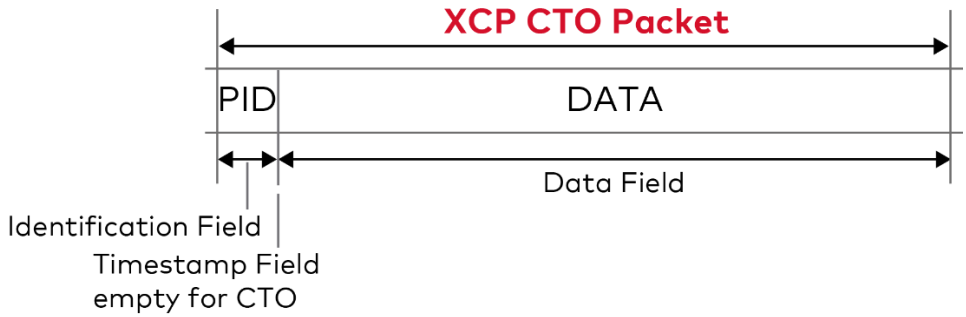


Figure 17 Overview of the structure of the CTO package

The master sends a general request to the slave via CMD. The PID (Packet Identifier) field contains the identification number of the command. The additional specific parameters are transported in the data field. Then the master waits for a reaction from the slave in the form of a response (RES) or an error (ERR).

XCP is also very scalable in its implementation, so that not every command has to be implemented. In the A2L file, in the so-called XCP IF_DATA the available CMDs are listed. If there is a discrepancy between the definition in the A2L file and the implementation in the slave, the master can recognize from the reaction of the slave that the slave does not support the command after all. If the master sends a command that has not been implemented in the slave, the

slave must acknowledge with ERR_CMD_UNKNOWN and no further activities are triggered in the slave. Thus, the master recognizes fast that an optional command was not implemented in the slave. Partly further parameters are contained in the commands.

The commands are divided into groups: Standard, Calibration, Page, Programming, Debugging and DAQ commands. If a group is not needed at all, its commands do not have to be implemented. If the group is necessary, the corresponding commands must be available in the slave in any case, others from the group are optional.

The following overview serves as an example. The SET_CAL_PAGE and GET_CAL_PAGE commands in the Page switching group are marked as not optional. This means that at least these two commands must be implemented in an XCP slave that supports page switching. If page switching support is not required in the slave, these commands do not have to be implemented. The same applies accordingly for other commands.

Standard commands:

Command	PID	Optional
CONNECT	0xFF	No
DISCONNECT	0xFE	No
GET_STATUS	0xFD	No
SYNCH	0xFC	No
GET_COMM_MODE_INFO	0xFB	Yes
GET_ID	0xFA	Yes
SET_REQUEST	0xF9	Yes
GET_SEED	0xF8	Yes
UNLOCK	0xF7	Yes
SET_MTA	0xF6	Yes
UPLOAD	0xF5	Yes
SHORT_UPLOAD	0xF4	Yes
BUILD_CHECKSUM	0xF3	Yes
TRANSPORT_LAYER_CMD	0xF2	Yes
USER_CMD	0xF1	Yes
GET_VERSION	0xC0, 0x00	Yes

Calibration commands:

Command	PID	Optional
DOWNLOAD	0xF0	No
DOWNLOAD_NEXT	0xEF	Yes
DOWNLOAD_MAX	0xEE	Yes
SHORT_DOWNLOAD	0xED	Yes
MODIFY_BITS	0xEC	Yes

Page switching:

Command	PID	Optional
SET_CAL_PAGE	0xEB	No
GET_CAL_PAGE	0xEA	No
GET_PAG_PROCESSOR_INFO	0xE9	Yes
GET_SEGMENT_INFO	0xE8	Yes
GET_PAGE_INFO	0xE7	Yes
SET_SEGMENT_MODE	0xE6	Yes
GET_SEGMENT_MODE	0xE5	Yes
COPY_CAL_PAGE	0xE4	Yes

Cyclic data exchange - basics:

Command	PID	Optional
SET_DAQ_PTR	0xE2	No
WRITE_DAQ	0xE1	No
SET_DAQ_LIST_MODE	0xE0	No
START_STOP_DAQ_LIST	0xDE	No
START_STOP_SYNCH	0xDD	No
WRITE_DAQ_MULTIPLE	0xC7	Yes
READ_DAQ	0xDB	Yes
GET_DAQ_CLOCK	0xDC	Yes
GET_DAQ_PROCESSOR_INFO	0xDA	Yes
GET_DAQ_RESOLUTION_INFO	0xD9	Yes
GET_DAQ_LIST_MODE	0xDF	Yes
GET_DAQ_EVENT_INFO	0xD7	Yes
DTO_CTR_Properties	0xC5	Yes
SET_DAQ_PACKED_MODE	0xC0, 0x01	Yes
Get_DAQ_PACKED_Mode	0xC0, 0x02	Yes

Cyclic data exchange - static configuration:

Command	PID	Optional
CLEAR_DAQ_LIST	0xE3	No
GET_DAQ_LIST_INFO	0xD8	Yes

Cyclic data exchange - dynamic configuration:

Command	PID	Optional
FREE_DAQ	0xD6	Yes
ALLOC_DAQ	0xD5	Yes
ALLOC_ODT	0xD4	Yes
ALLOC_ODT_ENTRY	0xD3	Yes

Flash programming:

Command	PID	Optional
PROGRAM_START	0xD2	No

PROGRAM_CLEAR	0xD1	No
PROGRAM	0xD0	No
PROGRAM_RESET	0xCF	No
GET_PGM_PROCESSOR_INFO	0xCE	Yes
GET_SECTOR_INFO	0xCD	Yes
PROGRAM_PREPARE	0xCC	Yes
PROGRAM_FORMAT	0xCB	Yes
PROGRAM_NEXT	0xCA	Yes
PROGRAM_MAX	0xC9	Yes
PROGRAM_VERIFY	0xC8	Yes

Time synchronization:

Command	PID	Optional
TIME_CORRELATION_PROPERTIES	0xC6	Yes

For connected ASAM standards:

Command namespace	PID	Optional
ASAM AE MCD-1-XCP AS SW-DBG-over-XCP	0xC0, 0xFC	Yes
ASAM AE MCD-1 POD BS	0xC0, 0xFD	Yes

The optional commands are described in the standards:
ASAM AE MCD-1 XCP AS SW-DBG-over-XCP and ASAM AE MCD-1 POD BS

1.3.2. RES

If the slave can successfully implement a master request, it confirms positively with RES.

Position	Type	Description
0	BYTE	Packet Identifier = RES 0xFF
1..MAX_CTO-1	BYTE	Command response data

For more detailed information about the parameters, refer to the ASAM XCP Protocol Layer specification.

1.3.3. ERR

If the request of the master is unusable, the slave sends an error message ERR with error code.

Position	Type	Description
0	BYTE	Packet Identifier = ERR 0xFE
1	BYTE	Error code
2..MAX_CTO-1	BYTE	Optional error information data

The possible error codes are described in the ASAM XCP Protocol Layer specification.

1.3.4. EV

An EV can be sent if the slave wants to inform the master about an asynchronous event. The implementation is optional.

Position	Type	Description
0	BYTE	Packet Identifier = EV 0xFD
1	BYTE	Event code
2..MAX_CTO-1	BYTE	Optional event information data

For more detailed information about the parameters, refer to the ASAM XCP Protocol Layer specification.

In connection with measurements and stimulation, there will be even more frequent talk about events. The action of the XCP slave that triggers the sending of an EVENT has nothing to do with this. Here it is a matter of the slave reporting a fault, for example, such as the failure of a functionality.

1.3.5. SERV

Via this mechanism the slave can request the execution of a service from the master. For example, the slave recognizes that a restart is necessary and requests a corresponding reset command from the master.

Position	Type	Description
0	BYTE	Packet Identifier = SERV 0xFC
1	BYTE	Service request code
2..MAX_CTO-1	BYTE	Optional service request data

The service request code table can be found in the ASAM XCP Protocol Layer specification.

1.3.6. Adjust parameters in the slave

To change a parameter in an XCP slave, the XCP master must send both the address where the parameter is located and the value itself to the slave.

XCP always defines addresses with five bytes: four for the actual address and one byte for the address extension. Based on a CAN transmission, only seven useful bytes are available for XCP contents. If, for example, you change a 4-byte value and want to send both pieces of information in one CAN frame, the number of bytes is not sufficient. Since a total of nine bytes are required for the transmission of the address and the new value, the change cannot be transmitted in one CAN frame (seven user bytes). The change request is therefore made with two packets from the master to the slave. The slave must acknowledge both packets, in total four packets are exchanged.

The following figure shows the communication between master and slave necessary for setting a parameter value. In the line with the envelope symbol the data content of the packet is shown.

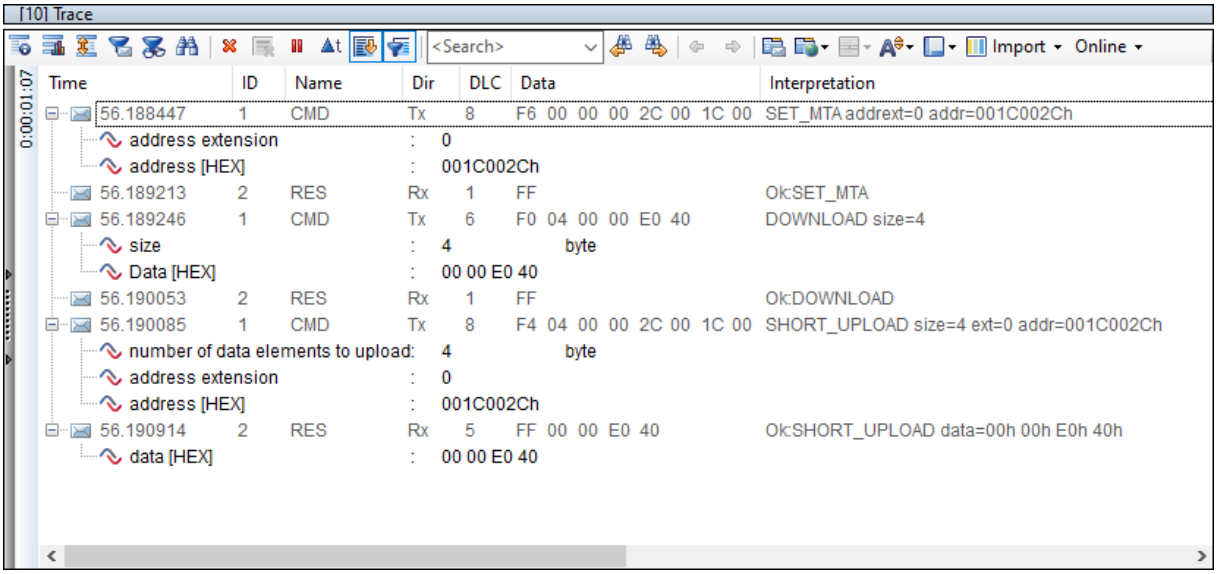


Figure 18 Trace representation of an calibration process

In the first packet the master transmits the command SET_MTA with the address (HEX: 001C002C, Extension: 0), to which a new value is to be written to the slave. With the second packet, the slave confirms the command positively with Ok:SET_MTA.

The third packet DOWNLOAD transmits the hex value (00 00 E0 40) and additionally the valid number of bytes. In this example, the valid number of bytes is four because it is a float value. The slave confirms positive again with the fourth packet.

This completes the actual calibration process. In the trace display you can see a subsequent SHORT_UPLOAD in the trace display - a special feature of CANape, Vector's measurement and calibration tool. To make sure that the calibration has worked, the value is read out again after the process and the display in CANape is updated with the read-out value. In this way, the user can see directly whether his calibration command has been successfully processed. This command is also confirmed positively with Ok:SHORT_UPLOAD.

Save parameter set file

With the change of the parameter in the RAM of the control unit, the application processes the new value. However, rebooting the ECU leads to deletion of the value and overwriting of the value in RAM with the original value from the flash (see chapter Calibration concepts).

So how can the changed parameterization be saved permanently?

Basically, there are two options:

A) Saving the parameters in the ECU
The changed data in the RAM can, for example, be saved in the EEPROM in the control unit, either automatically, e.g. when the control unit is shut down, or manually by the user. The prerequisite is that the data can be stored in a non-volatile memory of the slave. In the case of an ECU, both EEPROM and flash are non-volatile. EEPROM can be rewritten in individual cells, whereas flash memory must be rewritten in blocks. However, ECUs with thousands of parameters seldom have sufficient unused EEPROM memory space, so this procedure is rarely used.

Another possibility is to write back the RAM sizes to the Flash memory of the control unit. This procedure is relatively complex. A flash memory must first be erased before it can be rewritten. This in turn can only be done block by block. It is therefore not possible to write back individual bytes. More about this can be found in the chapter on calibration concepts.

B) Saving the parameters in the form of a file on the computer
Much more common is the storage of parameters on the computer. All parameters - or subsets of them - are stored in the form of a file. Different formats are available for this purpose, in the simplest case an ASCII text file containing only the name of the object and the value. Other formats allow the storage of further information, e.g., statements about the degree of maturity of the parameter or the history of changes.

Scenario: After work is done, the application engineer wants to go home for a well-deserved rest. He saves the changes made in the RAM of the ECU in the form of a parameter set file on his computer. The next day, he wants to

continue working where he left off the day before. He starts the control unit. When booting, the parameters in RAM are initialized. But for this, the ECU uses the values that are stored in the flash. This means that the changes from the previous day are no longer present in the ECU. To be able to continue working with the status from the previous day, the application engineer transfers the contents of the parameter set file to the RAM of the ECU via XCP using the DOWNLOAD command.

[10] Trace

<Search> Import Online

0:00:16:55

Time	ID	Name	Dir	DLC	Data	Interpretation
1004.248982	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.249000	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.249851	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.249875	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.250681	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.250709	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.251470	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.251492	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.252343	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.252363	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.253374	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.253403	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.254393	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.254425	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.255192	2	RES	Rx	1	FF	Ok:DOWNLOAD
1004.255218	1	CMD	Tx	8	F0 06 00 00 00 00 00 00	DOWNLOAD size=6
1004.260116	2	RES	Rx	1	FF	Ok:DOWNLOAD

Figure 19 Transfer of a parameter set file into the RAM of the control unit

Save and flash parameter set file in hex files

Flashing an ECU is a way to change the parameters in the flash. These are then written into the RAM as new parameters when the ECU is booted. A parameter set file can be transferred into a C or H file and the new flash file is generated by a new compiler/linker run. Depending on the size of the code, the generation of a flashable hex file can take some time. In addition, depending on the work process, the application engineer may not have the source code of the ECU. Thus, this way is not available to him.

Alternatively, the parameter set file can be copied into the existing flash file. In the flash file there is a hex file containing both the addresses and the values. Now a parameter file can be copied to a hex file. To do this, CANape takes the address and the value from the parameter set file and updates the value of the parameter at the corresponding position in the hex file. This creates a new hex file that contains the changed parameter values. However, this hex file may now have to pass through further process steps to obtain a flashable file from it. One problem here is always the checksums that the ECU checks to see whether it has received the data correctly. If the flashable file is available, it can be flashed into the ECU and after the reboot the new parameter values are available in the ECU.

1.4. Exchange of DTOs - synchronous data exchange

As shown in Figure 6 XCP communication model with CTO and DTO is shown, DTOs (Data Transfer Objects) are available for exchanging synchronous measurement and stimulation data. Via DAQ data is sent from the slave to the master synchronously to internal events. This communication is divided into two phases:
In an initialization phase, the master informs the slave which data the slave should send at different events. After this phase, the master triggers the measurement at the slave and the measurement phase begins. From this point on, the slave sends the desired data to the master, which only "listens" until it sends a "measurement stop" to the slave. The triggering of the measurement data acquisition and the sending are done by events in the control unit.

Via STIM the master sends data to the slave. This communication also consists of two phases:
In the initialization phase, the master tells the slave which data it will send to the slave. After this phase, the master sends the data to the slave and the STIM processor stores the data. As soon as a corresponding STIM event is triggered in the slave, the data is transferred to the application's memory.

1.4.1. Measurement method: Polling versus DAQ

Before explaining how event-synchronous correlating data is measured out of a slave, there is a short description of another measurement method, the so-called polling. It is not based on DTOs but on CTOs. This topic should be described in another chapter, but an understanding of polling helps one very nicely derive the necessity of the DTO-based measurement; thus, this small digression follows.

Via the command SHORT_UPLOAD the master can request the value of a measured variable from the slave. This is called polling. This is the simplest case of a measurement: send the measured value of a variable at the time when the SHORT_UPLOAD command has been received and executed.

In the following example, the measured variable "Triangle" is measured from the slave:

Type	Name	Comment	Address	Address Extension	Data Type
	PWM	Pulse width signal from PWM_level and Triangle	0x1BDDDF	0	UByte
	PWMFiltered	Low pass filtered PWM signal	0x1BDDE2	0	UByte
	syncCounter_float		0x1BF448	0	Float32
	syncCounter_int		0x1BF454	0	SLong
	Triangle	Triangle test signal used for PWM output PWM	0x1BDDDE	0	SByte

Figure 20 Address information of the parameter "Triangle" from the A2L file

The address (0x1BDDDE, Address Extension: 0) is found in the CAN frame as an address with five bytes: one byte for the address extension and four bytes for the actual address. and four bytes for the actual address.

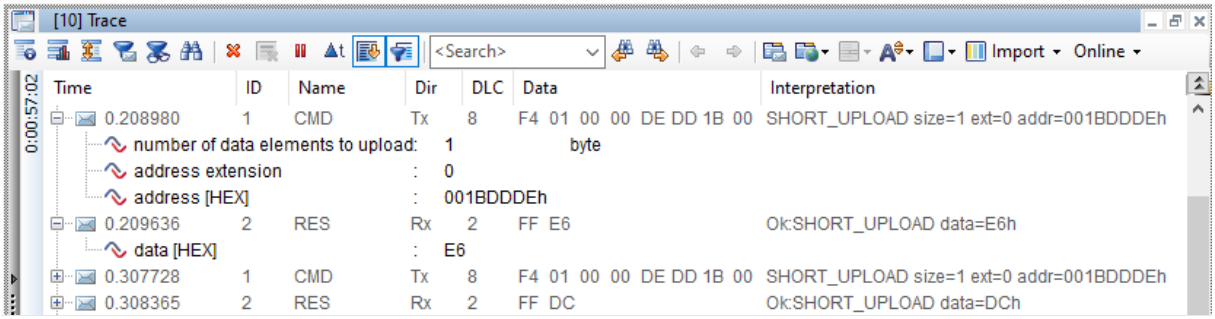


Figure 21 Polling communication in CANape's Trace Window

The XCP specification specifies for polling that the value of each measured variable is queried individually. Two frames must be transmitted for each value to be measured via polling: the request from the master to the slave and the response from the slave to the master. The data rate is also low with this transmission because the maximum number of user data is not transmitted.

Besides this additional bus load, there is another disadvantage of the polling method: When polling much data, the user normally attaches importance to the data correlating with each other. However, several values that are measured one after the other with polling do not have to correlate with each other. There is no information about which calculation cycles the values come from. Polling is therefore only suitable for measurement to a limited extent since it produces an unnecessarily large amount of data traffic and does not take the measurement data into account in relation to the processes in the ECU.

An optimized measurement must therefore solve two tasks:

> Bandwidth optimization during measurement
> Ensuring data correlation

This task is taken over by the DAQ procedure. DAQ stands for Data Acquisition and is used via the sending of DTOs (Data Transfer Objects) from the slave to the master.

1.4.2. The DAQ-measurement method

The DAQ method solves the two polling problems as follows:

> The correlation of the measured data is achieved by linking the measured value acquisition to the processes (events) in the control unit. The

measured values are only recorded and transmitted when the event occurs. This ensures that all data match each other.

> To reduce the bus load, the measuring process is divided into two phases: In a configuration phase, the master informs the slave which values it is interested in, and in the second phase, only the transmission of the slave's measurement data to the master takes place. During the transmission, the measurement data are summarized as compactly as possible in the packets.

How can the acquisition of measured values be linked to events in the control unit? In Figure 22 Events in the ECU the connection between the calculation cycles in the ECU and the value development of the variables X and Y is shown again slightly modified.

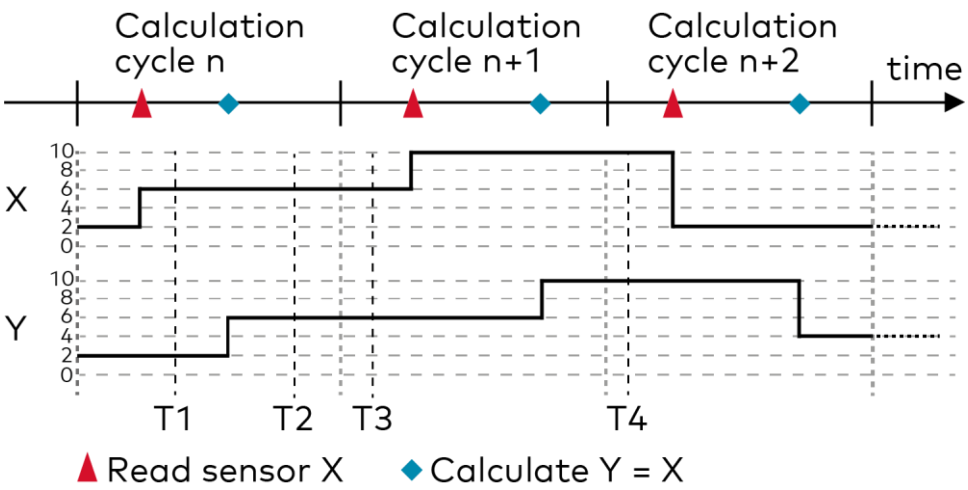


Figure 22 Events in the ECU

When event E1 (= end of the calculation cycle) is reached, all algorithm calculation has taken place. At this point in time, all values of this calculation cycle fit to each other. Exactly this circumstance is exploited with the DAQ mechanism: When the code in the slave reaches the event "Calculation cycle E1 completed", the XCP slave collects the values of the measured quantities, stores them in a buffer, and sends them to the master.

An event is not necessarily a cyclic event equidistant in time, but can also be angularly synchronous, e.g. in the case of a combustion engine ECU. Thus, the time interval between two events depends on the engine speed. A singular event, such as the actuation of a button by the driver, is also an event that is not equidistant in time in any way.

The selection of the measurement object and the associated event is made by the user. The events are described in the A2L file.

Event List

General | Expert settings

Event name	Event channel	Rate	Unit	Event type	Priority	DAQ/STIM	Multisample count
Key T	00h	0	ms	Sporadic	0	DAQ	0
10 ms	01h	10	ms	Cyclic	1	DAQ/STIM	0
100ms	02h	100	ms	Cyclic	2	DAQ/STIM	0
1ms	03h	1	ms	Cyclic	3	DAQ/STIM	0
FilterBypassDaq	04h	10	ms	Cyclic	4	DAQ/STIM	0
FilterBypassStim	05h	10	ms	Cyclic	5	STIM	0

Figure 23 Examples of event definitions

In principle, a measurement object can be measured for every event. However, this does not necessarily make sense since the change of a measurement object does not occur at every event. For example, if an algorithm is calculated in a 10-millisecond time grid, a measurement grid of 1 millisecond would not provide any additional information.

Properties - variable 'Triangle'

General
Address
Annotations
Data Type
Physical Display
XCP

DAQ events: Variable
Available events: 10 ms
100ms
1ms
Default events: 10 ms

Figure 24 Assignment of "Triangle" to the possible events in the A2L

Figure 24 Assignment of "Triangle" to the possible events in the A2L shows that the measured variable "Triangle" can basically be measured with the events “1ms”, “10 ms” and “100 ms”. The default setting is “10 ms”. If the user selects the measurement object “Triangle” without changing the measurement grid, the event “10 ms” is used.

However, the assignment of the measured variables to the events in the ECU can of course be changed directly by the user.

No.	Type	Active	Name	Measurement mode	Rate
11		☑	map1_8_8_uc_measure	10 ms	
0		☑	Comment	on input	
1		☑	Triangle	10 ms	
3		☑	PWM	1ms	
4		☑	PWMFiltered	1ms	
6		☑	channel1	polling	100

Figure 25 Selection of events (Measurement mode) per measurand

After configuring the measured variables, the user starts the measurement. The XCP master summarizes the desired measured variables in so-called DAQ lists. The measurement signals are assigned to the selected events. This configuration information is sent to the slave before the actual start of the measurement. The slave then knows which addresses it should read out and transmit when which event is reached. This division of the measurement into a configuration and a measuring phase was already mentioned at the very beginning of the chapter.

This solves both problems that occur during polling: both the optimized use of bandwidth, since the master no longer must request each value individually during the measurement, and the correlation of the measurement data.

[10] Trace

0:03:19:22

Time	ID	Name	Dir	DLC	Data	Interpretation
-0.305415	1	CMD	Tx	4	DE 02 03 00	START_STOP_DAQ_LIST mode=02h daq=3
-0.304572	2	RES	Rx	2	FF 0E	Ok:START_STOP_DAQ_LIST firstPid=0Eh
-0.304553	1	CMD	Tx	8	E0 10 04 00 03 00 01 03	SET_DAQ_LIST_MODE mode=10h daq=4 event=3 pr
-0.304378	2	RES	Rx	1	FF	Ok:SET_DAQ_LIST_MODE
-0.304367	1	CMD	Tx	4	DE 02 04 00	START_STOP_DAQ_LIST mode=02h daq=4
-0.303411	2	RES	Rx	2	FF 10	Ok:START_STOP_DAQ_LIST firstPid=10h
-0.302370	1	CMD	Tx	1	DC	GET_DAQ_CLOCK
-0.302217	2	RES	Rx	8	FF 10 6D 61 F9 4E D8 0E	Ok:GET_DAQ_CLOCK timestamp=249057017
0.003740	1	CMD	Tx	1	DC	GET_DAQ_CLOCK
0.009123	2	RES	Rx	8	FF 10 6D 61 15 5B D8 0E	Ok:GET_DAQ_CLOCK timestamp=249060117
0.009326	1	CMD	Tx	2	DD 01	START_STOP_SYNCH mode=01h
0.009836	2	RES	Rx	1	FF	Ok:START_STOP_SYNCH
0.019464	2	DAQ	Rx	7	00 00 79 5B D8 0E DE	Data
0.019485	2	DAQ	Rx	7	00 01 79 5B D8 0E CE	Data
0.019496	2	DAQ	Rx	8	00 02 79 5B D8 0E AF B3	Data
0.019513	2	DAQ	Rx	8	01 02 B7 BB BC BB B7 AE	Data
0.019523	2	DAQ	Rx	8	02 02 97 9E A9 B1 B5 B5	Data

Figure 26 Excerpt from the CANape Trace window of a DAQ measurement

In the trace, the command-response communication (black font) between master and slave becomes clear. It is altogether more extensive and is only partially shown here for space reasons. This is the transmission of the DAQ configuration to the slave. After that the measurement start is triggered and the slave sends the requested values while the master listens.

So far, the selection of the measured variables by their name and the assignment to a measurement event have been described. But how exactly is the configuration transferred to the XCP slave?

Let's look at the problem from the side of the memory structure in the controller: The user has selected signals and wants to measure them. In order not to have to use a complete frame for sending a signal value, the slave packs as many values as possible into one frame. The slave does not create the definition of the composition by itself, otherwise the master could not interpret the data when receiving the frame. Therefore, the master sends the instructions to the slave how to distribute the values to the frames.

The assignment is made via so-called Object Description Tables (ODTs). To identify a measured variable unambiguously, the address and the object length are important. Thus, one receives with an ODT the assignment of the RAM contents from the slave to the structure of a frame on the bus. According to the communication model the transfer is done as DAQ DTO (Data Transfer Object).

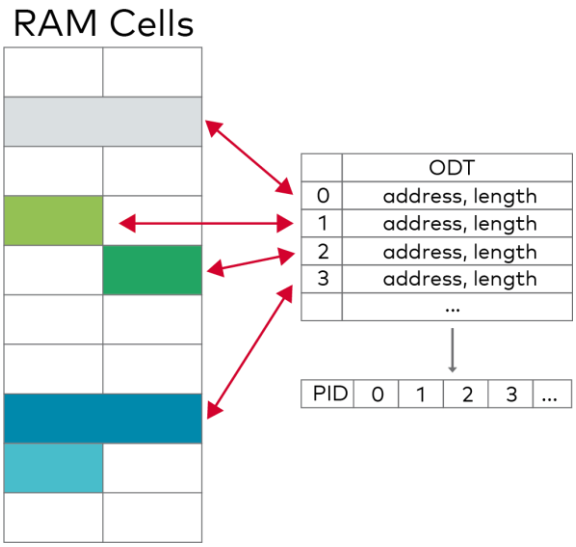


Figure 27 Assignment of RAM addresses to ODT

More precisely, an entry in an ODT list references a memory area in RAM by the address and length of the object.

After receiving the measurement start command, an event is reached at some point that is linked to a measurement. The XCP slave records the values and transmits them via the bus. The master reads the information from the bus. Since it has defined the assignment of the values to the packets itself, the interpretation of the data is simple.

The maximum number of payload bytes that can be transmitted in a packet depends on the transport medium. In the case of CAN, this is seven bytes. If more data is to be measured, one ODT is not sufficient. If two or more ODTs must be used for the transmission of the measurement data, then both the slave must copy the data into the correct ODT and the master must be able to uniquely identify the received ODTs. If several measuring grids of the control unit are used, the connection between ODT and measuring grid must also be clearly assignable.

In the XCP protocol, the ODTs are combined into DAQ lists. Each DAQ list contains a number of ODTs and is assigned to an event.

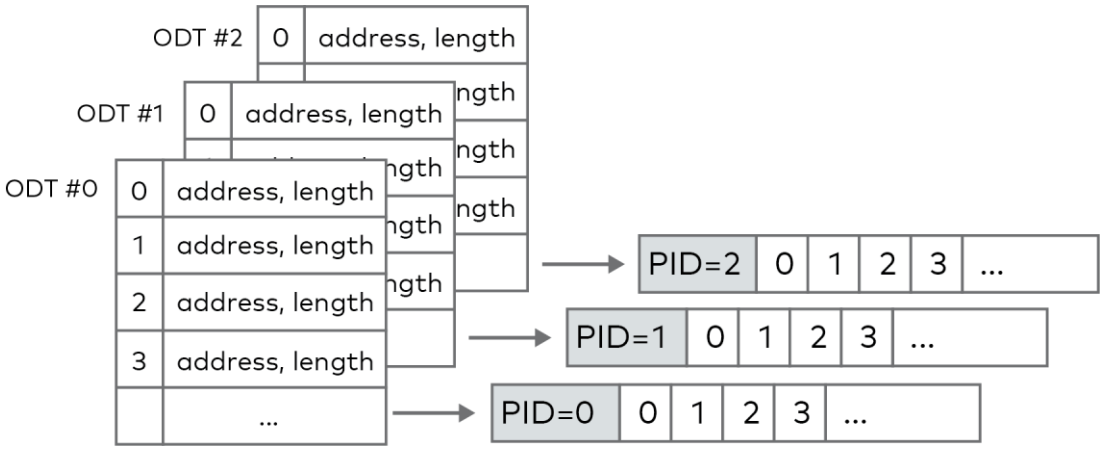


Figure 28 DAQ list with three ODTs

If, for example, the user uses two measuring grids (= two different events in the ECU), two DAQ lists are also used. One DAQ list is required for each event used. Each DAQ list contains the entries concerning the ODTs and each ODT contains the references to the values in the RAM cells.

Optionally, there is the possibility that the slave transmits timestamp information as well. A DAQ list represents the values that belong to a specific event. When the values are recorded in the slave, the valid time value of the ECU is also recorded and transmitted within the first ODT. The further ODTs of this DAQ list do not contain a timestamp. A timestamp is implemented in the form of an incrementing counter. The value is incremented in fixed time intervals. The time interval at which the incrementing is performed is stored in the A2L.

PID 0	00:00		S1	S2	S3	S4	S5
PID 1	S6	S7	S8	S9	S10		

PID 0	00:01		S1	S2	S3	S4	S5
PID 1	S6	S7	S8	S9	S10		

Figure 29 Transmission of signals 1-10 with timestamp

DAQ lists come in three different forms: Static, Predefined and Dynamic.

Static DAQ-lists:

If the DAQ lists and ODT tables are permanently defined in the control unit, as known from CCP, they are called static DAQ lists. Here it is not defined which measured variables are in the ODT lists. A static DAQ list defines the frame which can be filled (in contrast to this see predefined DAQ lists).

For static DAQ lists, the definitions are defined in the ECU code and described in the A2L. Figure 30 Static DAQ lists shows a section of an A2L in which static DAQ lists are defined:

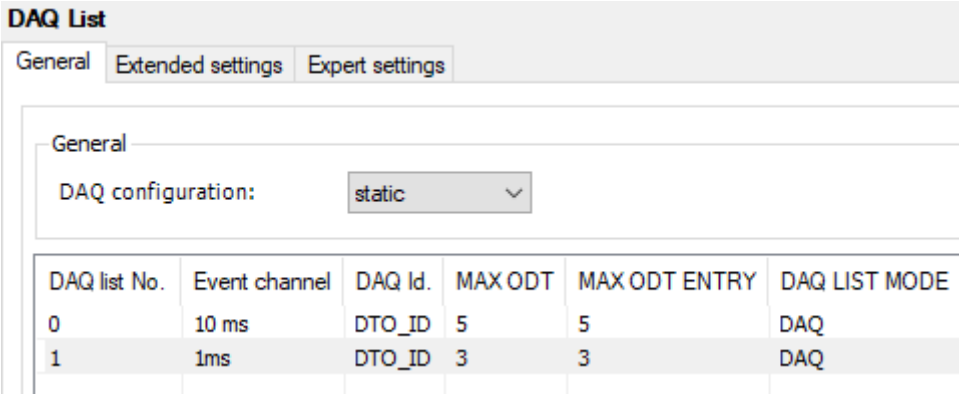


Figure 30 Static DAQ lists

In the example above, there is a DAQ list with the number 0, which is associated with a 10ms event and can carry a maximum of five ODTs. The DAQ list numbered 1 has three ODTs and is associated with the 1ms event.

With the download of the application into the ECU, in the case of static DAQ lists, the number of DAQ lists and the respective ODT lists contained therein are fixed. If the user now tries to measure more signals with an event than fit into the assigned DAQ list, the slave in the ECU will not be able to fulfill the requirements and the configuration attempt will be aborted with an error. The fact that the other DAQ list is still completely available and therefore transmission capacity is still free is irrelevant.

Predefined DAQ-lists:

Completely predefined DAQ lists can also be determined in the ECU. However, this procedure is practically not used in ECUs due to the lack of flexibility of the user. The situation is different for analog measurement systems that transfer their data via XCP: Here, the flexibility is not necessary because the physical structure of the measurement system remains the same over its lifetime.

Dynamic DAQ-lists:

Dynamic DAQ lists are mostly used in ECU development. It is not the absolute sizes of the DAQ and ODT lists that are permanently defined in the ECU code, but only the size of the memory area that can be used for the DAQ lists. The advantage is that the measurement tool has more leeway when combining the DAQ lists and can manage the structure of the DAQ lists dynamically.

Specifically for this dynamic management, XCP provides various functions such as ALLOC_ODT with which the master can define the structure of a DAQ list in the slave.

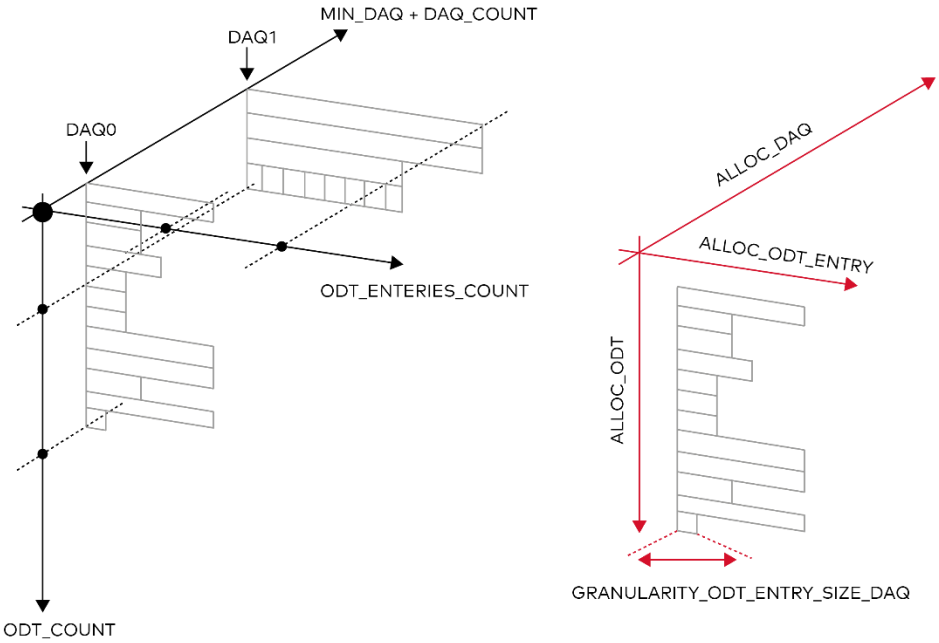


Figure 31 Dynamic DAQ lists

When compiling the DAQ lists, the master must be able to distinguish whether dynamic or static DAQ lists are used, what the sizes and structures of the DAQ lists are, etc.

Higher data rates due to Packed Mode

With version 1.4 of the XCP standard, the so-called "Packed Mode" was defined. The mode enables the transmission of several measured values of a signal in one ODT.

PID 0	00:00		S1	S1	S1	S1	S1
PID 1	S2	S2	S2	S2	S2		

PID 0	00:01		S1	S1	S1	S1	S1
PID 1	S2	S2	S2	S2	S2		

Figure 32 Transmission of five measured values of signals S1 and S2 in one ODT each

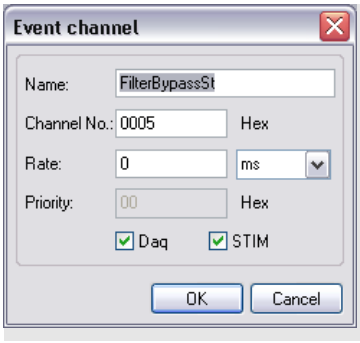
With this form of transmission, only one timestamp is transmitted per DAQ list. The XCP master calculates the timestamps of the individual measured values by an offset calculation on the transmitted timestamp. The sampling rate is defined in the A2L.
A typical application is the measurement of physical quantities. For example, the voltage values at an inverter in the electric vehicle are sampled at one megahertz. The measurement system summarizes the data and transmits it via XCP on Ethernet. A defined number of measured values is transmitted in an ODT. The XCP master calculates the timestamps of the individual measured

values by incrementing the timestamp by one microsecond for each measured value.

1.4.3. The STIM-transmission method

The XCP calibration procedure has already been presented in the chapter on the exchange of CTOs. This type of calibration is available in every XCP driver and forms the basis for adjusting objects in the ECU. However, there is no synchronization between the sending of a calibration command and an event in the ECU.

In contrast, the use of STIM is not based on the exchange of CTOs, but on the use of DTOs with communication synchronized to an event in the slave. The master must therefore know which events in the slave it can synchronize to at all. This information must also be available in the A2L.




```
/begin EVENT
  "FilterBypassSt"      // Event Channel Name
  "FilterByp"           // Event Channel Short Name
  0x0005                // Event Channel Number
  DAQ_STIM              // both directions for DAQ and STIM
  0x01                  // MAX_DAQ_LIST
  0x00                  // TIME_CYCLE
  0x06                  // TIME_UNIT
  0x00                  // PRIORITY
/end EVENT
```

Figure 33 Event for DAQ and STIM

The basic task of the STIM method is to provide data to the control unit. For example, when working with radar sensors, you always need an environment that reflects the radar beams. For a laboratory setup, this is unfavorable. The solution via XCP looks like this: In the vehicle, raw radar data is recorded by the sensor. In the laboratory, these recordings can then be transferred back to the sensor via XCP STIM.

1.4.4. XCP packet addressing for DAQ and STIM

The addressing of XCP packets was already introduced at the beginning of the chapter. Now that the terms DAQ, ODT and STIM have been introduced, XCP packet addressing is presented in more detail.

While the use of a PID is completely sufficient for the transmission of CTOs to uniquely identify a packet, this is no longer sufficient for the transmission of DAQ and STIM data. The following figure gives an overview of the possible addressing that can be used for DTOs:

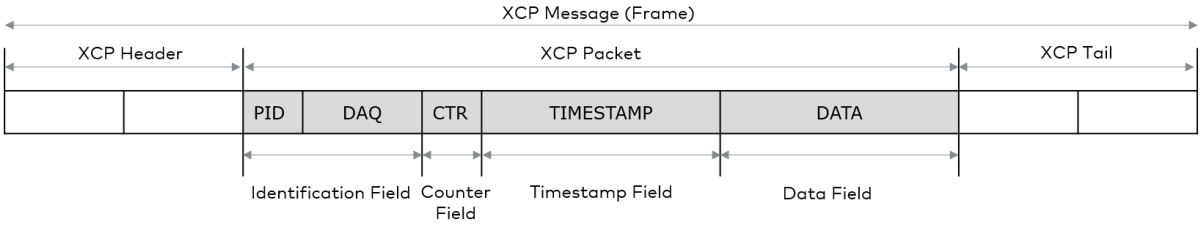


Figure 34 Structure of XCP package for DTO transmissions

Transmission type "absolute ODT numbers"

"Absolute" means that during the entire communication - i.e. across all DAQ lists - the ODT numbers are unique. In this respect, the use of absolute ODT numbers requires a transformation step in which a so-called "FIRST_PID for the DAQ list" is used.

If a DAQ list starts with PID j, the PID of the first packet has the value j, the second packet PID j + 1, the third packet PID j + 2 and so on. Of course, the slave has to ensure that the sum of FIRST_PID + relative ODT number stays below the PID of the next DAQ list.

DAQ list: 0 ≤ PID ≤ k
DAQ list: k + 1 ≤ PID ≤ m
DAQ list: m + 1 ≤ PID ≤ n
etc.

In this case, the identification field is very simple:

Identification Field

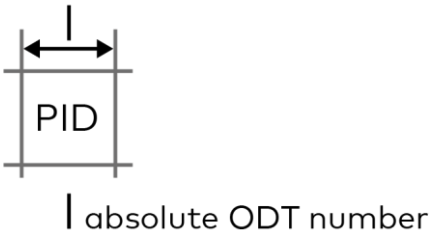


Figure 35 Identification field with absolute ODT numbers

Transmission type "relative ODT numbers and absolute DAQ list numbers".

In this case, both the DAQ list number and the ODT number are transmitted in the identification field. The ODT number is transmitted with one byte, the DAQ number with one or two bytes.

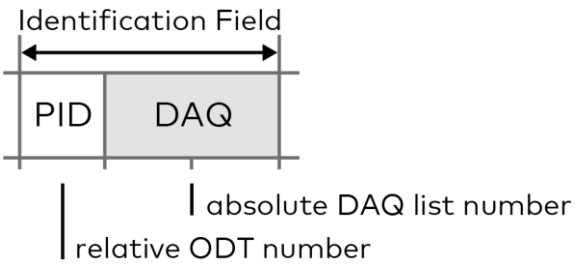


Figure 36 ID field with relative ODT and absolute DAQ numbers (two bytes)

If a transmission with three bytes is out of the question, four bytes can also be used with the aid of a fill byte:

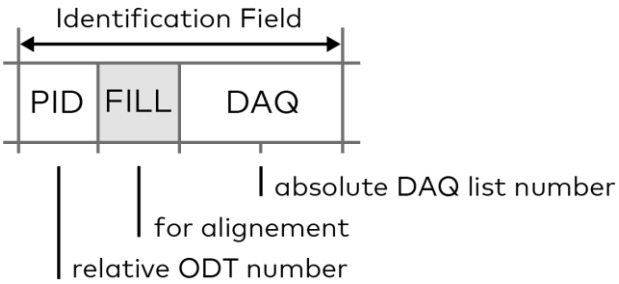


Figure 37 ID field with relative ODT and absolute DAQ numbers as well as fill byte (total four bytes)

How does the XCP master find out which method the slave uses? The XCP master can determine the communication variant either through the A2L file or via a query to the slave. If the slave has DAQ enabled, the slave will respond.

With the response to the GET_DAQ_PROCESSOR_INFO request the DAQ_KEY_BYTE is set, with the help of which the slave informs the master which transmission type is used. If not only DAQ but also STIM is to be used, the master must use the same procedure for STIM that the slave uses for DAQ.

1.4.5. Bypassing = DAQ + STIM

Bypassing can be implemented by sharing DAQ and STIM (see Figure 6 XCP communication model with CTO and DTO) and represents a special form of rapid prototyping solution. However, further details are required for a deeper understanding, so this procedure will not be explained until the Bypassing chapter.

1.5. Time correlation and synchronization

1.5.1. Introduction

Since the first version of the XCP specification, a time correlation between master and slave is available, which is based on the GET_DAQ_CLOCK command. The XCP master sends the GET_DAQ_CLOCK command to the slave. When the command is received, the XCP slave records its time and sends it back as a response. Since the XCP master knows the time at which it sent the request, the XCP master can calculate a time offset between the master clock and the slave clock.
The achievable accuracy of the time correlation is limited with this method. Currently, synchronization accuracies in the single-digit microsecond range or even below are required, which cannot be achieved with this method.

To meet these requirements, the XCP specification has been significantly extended. As a central improvement, an XCP master is now able to obtain a detailed insight into the clock system of the overall setup. Multiple clocks with different characteristics can be present in a measurement setup. In addition to three basic techniques, the extended time correlation provides all features to significantly improve time synchronization.

1.5.2. XCP's own method for improving time synchronization

The basic idea is based on the GET_DAQ_CLOCK procedure. However, the command is now sent by the master in the form of a multicast (= master sends to many). All XCP slaves connected to the same transport layer receive the GET_DAQ_CLOCK_MULTICAST command at the same time. Each slave sends back the timestamp at the time of reception as a response. The command is sent periodically by the master. This enables the XCP master to correlate the timestamps of the XCP slaves with each other and with its own master time.
To achieve the best accuracy, two requirements must be met:

- Participating slaves must immediately sample their timestamps when the event occurs.
- The latency between the XCP master and the participating XCP slaves should be consistent to ensure simultaneous occurrence of the event initiated by the XCP master at each XCP slave.

1.5.3. Support for external time synchronization techniques

In this case, it is assumed that the clock of the XCP slave supports the Precision Time Protocol (PTP, IEEE 1588 standard). The clock of the XCP slave must be either synchronized or syntonized to a Grandmaster Clock. Synchronized means that the clocks show the same time at the same point in time.

Syntonized means that one second is the same for all clocks. But the clocks do not have to show the same time.

1.5.3.1. Short introduction to PTP

PTP is a network protocol that is used to synchronize different clocks with each other. The objective is that all clocks of a time domain show the same time. The synchronization runs cyclically so that synchronicity is maintained. With a hardware based PTP implementation, synchronizations in the nanosecond range can be achieved.

Placement of PTP in the TCP/IP stack:

Application Layer	PTP
Transport Layer	UDP
Network Layer	IP (IPv4, IPv6)
Data Link Layer	Ethernet
Physical Layer	Ethernet PHY

Initial situation: Several clocks are connected to each other via a network.
To synchronize them with each other, the clock that has the highest accuracy is first searched for. The clock that is best suited for this is selected cyclically via the "Best Master Clock Algorithm (BMCA)". It is called the "Grandmaster Clock". It serves as a synchronization source for all other clocks in the network. A clock that synchronizes to the Grandmaster Clock is called Slave Clock. Just as with XCP, in the PTP environment we also speak of master and slave. However, an XCP slave can be the master clock in the network.
For time synchronization, the slave clocks and the Grandmaster Clock exchange Ethernet packets with each other. Time differences are calculated via the transmission times of the packets, which are used to synchronize the clocks. The ability to precisely timestamp the send and receive time of Ethernet packets is a basic requirement for time synchronization.
If the clocks are connected via routers and switches, these must also support the PTP protocol. A PTP capable Ethernet Switch represents itself to a connected Grandmaster Clock as Slave Clock. Other slave clocks, which are connected to the Ethernet switch, see the Ethernet switch as master clock.

The time synchronization in two-step procedure:
The master clock sends a sync packet to the slave at time TM1. The reception time at the slave is TS1. Immediately afterwards, the master clock sends a FollowUp packet to the slave. The packet contains the time at which the Sync packet was sent. Now the time difference between TM1 and TS1 is calculated in the slave:

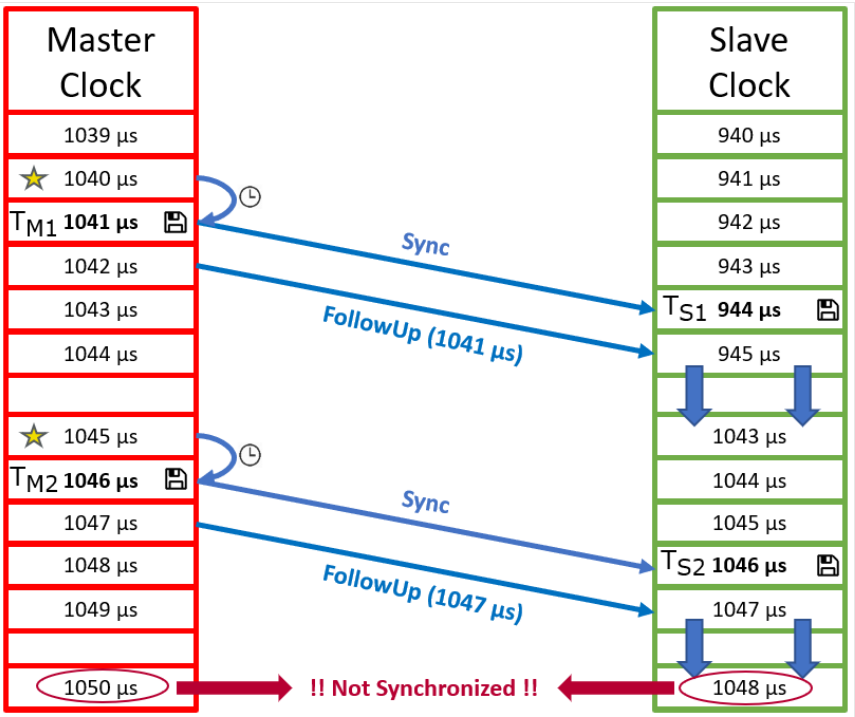


Figure 38 First stage of time synchronization

In the picture above, the calculation results in 1041µs-944µs = 97µs. The slave adds this time difference to its own time. The synchronism between both clocks is already pretty good, but due to the signal propagation time there is still a time difference of 2µs in this example. This synchronization runs cyclically; for example, once per second.
The reverse path also runs cyclically. With a DelayRequest the slave clock sends a packet to the master clock. The slave clock remembers the time of sending TS3 (1050µs).

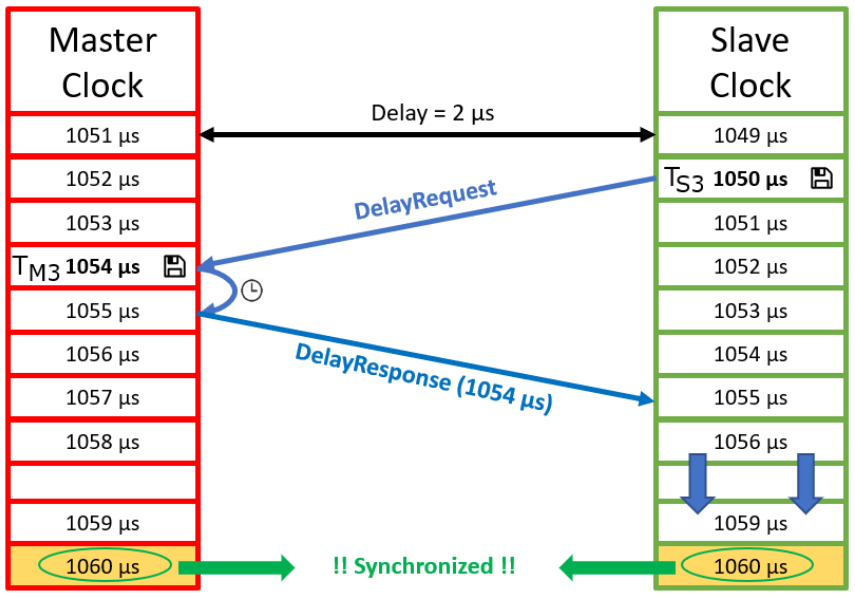


Figure 39 Second stage of time synchronization

The master clock receives the packet at time TM3. The master clock sends back the time TM3 (1054µs) to the slave clock. The slave calculates its new time from this. The time difference between TM3 and TS3 is 4µs. If we now assume that the runtime of the Ethernet packets is the same in both directions, we only have to divide by 2 to calculate the runtime difference. In this example it amounts to 2µs. The slave adds this difference again to its current time. Thus, the time of the slave clock is identical with the time of the master clock.

1.5.4. Sharing PTP and XCP

Several scenarios are presented in this chapter. They differ in whether and how the XCP slave is synchronized with a master clock. Not all scenarios are relevant in practice. Anyone who takes the trouble to provide a PTP master in the overall system will certainly also ensure that the XCP slave can synchronize.

The XCP master must know

- Whether the clock of the XCP slave is synchronized with a Grandmaster Clock at all.
- With which Grandmaster Clock the XCP slave is synchronized. This is important if there is more than one Grandmaster Clock in the overall system.

An XCP slave can know several clocks. Each clock has a unique UUID identifier. The XCP slave communicates the identifier of the Grandmaster Clock to the XCP master. This gives the XCP master an overview of all clocks in the system.

If the XCP slave is not able to synchronize its time with the Grandmaster Clock, the following procedure can be used: The XCP slave sends timestamp tuples to the XCP master. One is the local timestamp, and the other is the timestamp of the globally synchronized clock that was valid when the local timestamp was read. Thus, the XCP master knows the relation between the clocks and can perform a precise clock correlation within the XCP master.

The extended time correlation mechanisms use the commands/events:

- Protocol layer command TIME_CORRELATION_PROPERTIES
- Extended format of the positive response of the GET_DAQ_CLOCK command.
- Extended format of the EV_TIME_SYNC event packet.
- Transport layer subcommand GET_DAQ_CLOCK_MULTICAST for the transport layers CAN, FlexRay and Ethernet.

The support of multicast packets in an ECU is quite complex. Instead of GET_DAQ_CLOCK_MULTICAST, the command GET_DAQ_CLOCK can also be used. For better readability, both commands are not always mentioned in the following text. Only GET_DAQ_CLOCK_MULTICAST is mentioned. However,

both commands are meant. In a PTP network, the clocks of the participants are synchronized anyway.

Command TIME_CORRELATION_PROPERTIES
With this command, the XCP master queries the general information about the synchronization status of the clocks available in the XCP slave as well as their properties. The command is also used to configure the behavior of the XCP slave with regard to time correlation.

Extended positive response to GET_DAQ_CLOCK command:
XCP version 1.0: When the command arrives at the XCP slave, the timestamp is taken and sent to the XCP master as a response. The timestamp has a length of 4 bytes.

With XCP version 1.3, the following extension was added:
The timestamp of the XCP slave is transmitted in any case. Optionally, however, two further timestamps can be transmitted in addition. These are the timestamps of the Grandmaster Clock and the ECU.

Thus, up to 3 timestamps can be sent in one packet. A timestamp is transmitted either with 4 or with 8 bytes. This depends on the sources. Since the timestamps must be transmitted in one packet, the method cannot be used on a CAN bus as a transport medium. The number of bytes exceeds the maximum number of possible data bytes.

Extended format of event packet EV_TIME_SYNC
To improve the time correlation between the XCP slaves, the EV_TIME_SYNC event package has been extended in XCP version 1.3. The scope and amount of information that the XCP slave sends to the master via this depends on several factors. For example, the number of clocks, the accessibility of the clocks and dynamic runtime effects that occur when synchronization is lost. With this event, the slave can send up to three timestamps to the master, which is comparable with the positive extended response to GET_DAQ_CLOCK.

Transport layer subcommand GET_DAQ_CLOCK_MULTICAST
Instead of sending an individual GET_DAQ_CLOCK command to each individual XCP slave, the GET_DAQ_CLOCK_MULTICAST command can be used to send the command to all XCP slaves simultaneously. To do this, however, the XCP slaves must all be accessible via the same transport layer.

1.5.4.1. Scenario 1: Free-running clock in XCP slave

In the simplest case, the XCP slave has a free-running clock. If the XCP slave sends a timestamp, it is always from this clock. When a GET_DAQ_CLOCK_MULTICAST packet is received, the clock is read, and the received timestamp is sent back to the XCP master.

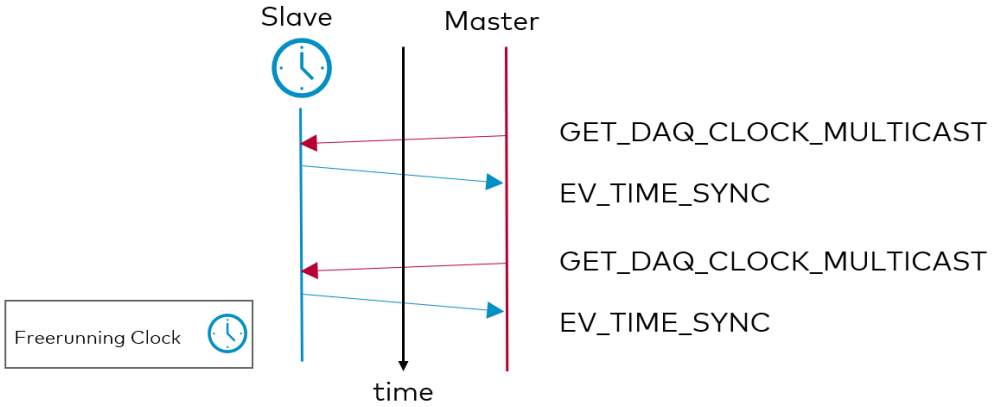


Figure 40 Free-running clock in XCP slave

1.5.4.2. Scenario 2: XCP slave clock is synchronized with Grandmaster Clock

The use of the commands GET_DAQ_CLOCK_MULTICAST and GET_DAQ_CLOCK does not differ from the previous scenario, but the XCP master must know two things: whether the clock is synchronized and with which Grandmaster Clock the synchronization is done.
With this information, the XCP master can put all XCP slaves that are synchronized to the same Grandmaster Clock into one logical domain. For the correlation of DAQ timestamps between XCP slaves of this domain, the XCP master does not need to perform time correlation.

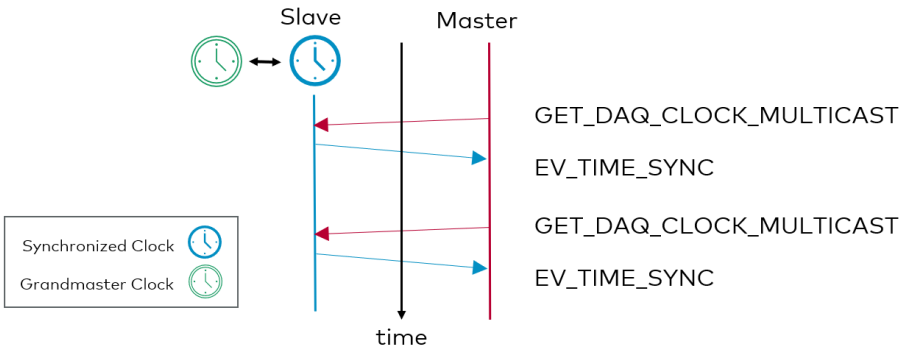


Figure 41 XCP Slave Clock synchronized with Grandmaster Clock

1.5.4.3. Scenario 3: XCP slave clock is syntonized with Grandmaster Clock

This scenario differs from the previous one in that there is an offset between the Grandmaster Clock and the XCP Slave Clock. However, the clock change of both clocks is the same. The XCP master must know the offset between both clocks and the basic clock rate. The XCP master sends the command TIME_CORRELATION_PROPERTIES which requests details about the clock information of the slave. In the positive response to the command, the slave informs the master that information about the clocks and their relationship is available. The details are then retrieved by an UPLOAD command. After that, the timestamps are requested again by GET_DAQ_CLOCK_MULTICAST.

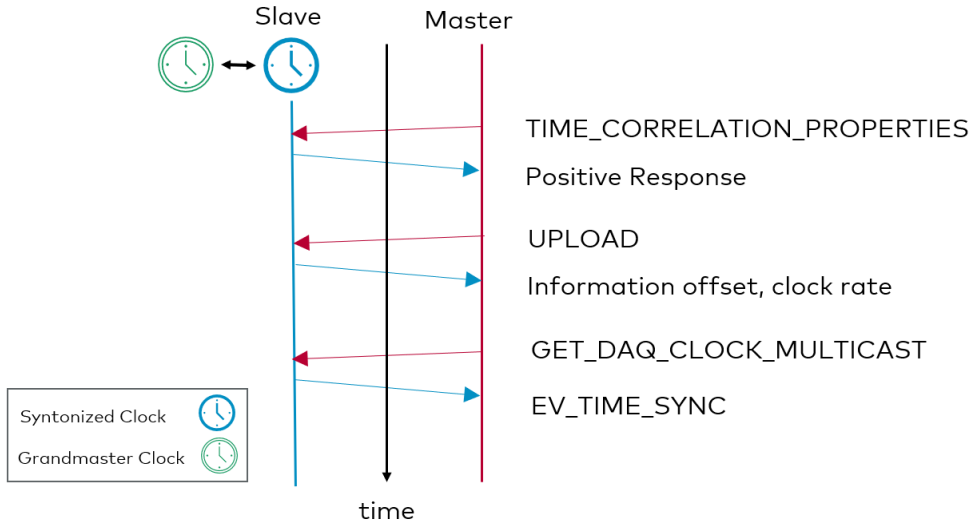


Figure 42 Slave clock syntonized with Grandmaster Clock

1.5.4.4. Scenario 4.a: XCP slave with two clocks: Free-running clock combined with a synchronized clock

Initial situation: The XCP slave cannot synchronize its own free-running clock with the Grandmaster Clock. However, there is another clock available in the XCP slave that is synchronized with a Grandmaster Clock. For example in the Ethernet PHY. If the timestamps of both clocks are available to the XCP master, it can perform the correlation of both clocks.

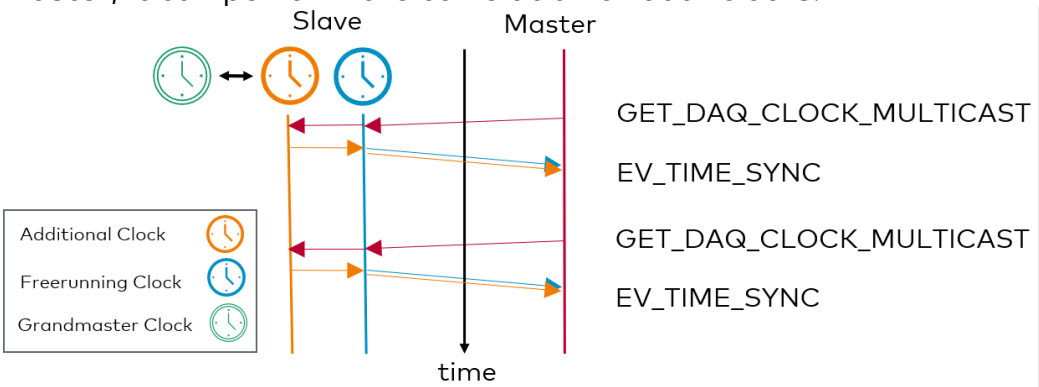


Figure 43 Free running clock combined with synchronized clock

Scenario 4b:

The prerequisite for the scenario just described is that both clocks can be read at any time. However, this may not be possible. For example, several resources access the Ethernet PHY. If the synchronized clock is located there, immediate processing of the GET_DAQ_MULTICAST request may be delayed by another access.

Mostly, however, these devices offer the possibility to generate a trigger periodically, e.g., one pulse per second. This trigger can be used to detect the clock of the XCP slave. Since the time of the synchronized clock is known when the trigger occurs, the XCP slave can transmit both timestamps via EV_TIME_SYNC.

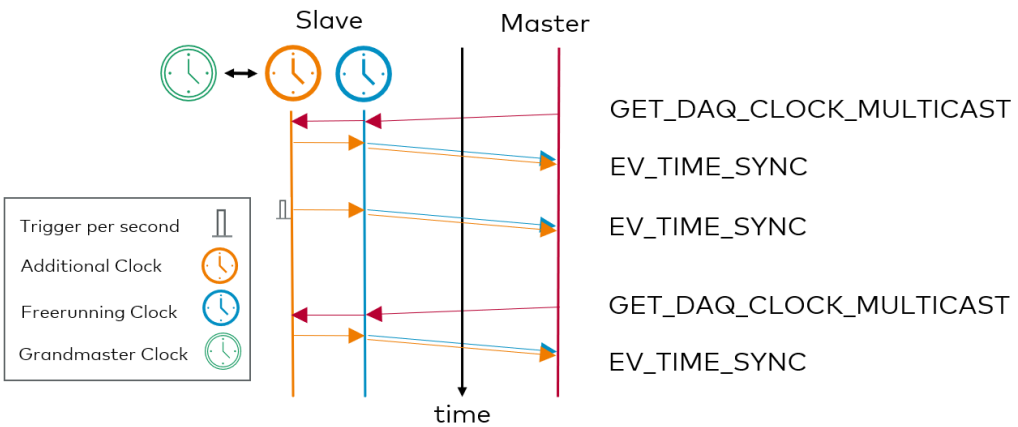


Figure 44 Two clocks with triggered EV_TIME_SYNC shipping

1.5.4.5. Scenario 5: Free-running XCP slave clock and a controller clock

Initial situation: The DAQ timestamps transmitted by the XCP slave refer to the ECU timestamp, which can be read out at any time. At first glance, the situation is very simple. The XCP slave transmits the timestamps of the ECU, and the setup is basically the same as scenario 1 so you would not have to transmit both timestamps at all.
However, since ECU resets can occur, it still makes sense to transmit both timestamps to the XCP master. The XCP slave could generate an EV_TIME_SYNC packet when an ECU reset is triggered and thus inform the XCP master about this event. The transmission of both timestamps at the time of the reset can be useful for the algorithm for calculating the time correlation. If the ECU clock cannot be read at any time, the XCP slave must periodically generate an EV_TIME_SYNC packet with both timestamps.

Two further scenarios are described in the XCP specification. A three-clock system and, in the other case, an XCP slave without its own clock. These cases are not presented in this book.

1.6. XCP transport layers

A main requirement for the design of the XCP protocol was to support different transport layers. The following layers are defined at the time of writing this document: XCP on CAN, XCP on FlexRay, XCP on Ethernet, XCP on SxI and XCP on USB. The CAN and FlexRay bus systems are explained on the Vector E-Learning platform.

1.6.1. CAN

XCP has been developed as a successor protocol to the CAN Calibration Protocol (CCP) and must necessarily meet the requirements of the CAN bus. Communication via the CAN bus is defined by the associated description file. Usually the DBC format is used, though occasionally also the AUTOSAR format ARXML.

A CAN frame is identified by a unique CAN identifier. The communication matrix is defined in the description file: Who sends which frame and how are the eight data bytes of the CAN bus occupied. The following figure illustrates the process:

Data Frame	**CAN node A**	**CAN node B**	**CAN node C**	**CAN node D**
ID=0x12	Sender	Receiver		
ID=0x34		Sender	Receiver	Receiver
ID=0x52		Receiver		Sender
ID=0x67	Receiver	Receiver	Sender	Receiver
ID=0xB4	Receiver		Sender	
ID=0x3A5	Sender	Receiver	Receiver	Receiver

Figure 45 Determination which CAN node sends which frames

The frame with the ID 0x12 is sent by CAN node A and all other nodes on the bus receive this frame. CAN nodes C and D conclude during the acceptance check that they do not need this frame and discard it. CAN node B, on the other hand, determines that the layers further up the bus need the frame's information and makes it available via the receive buffer. The CAN nodes are linked as follows:

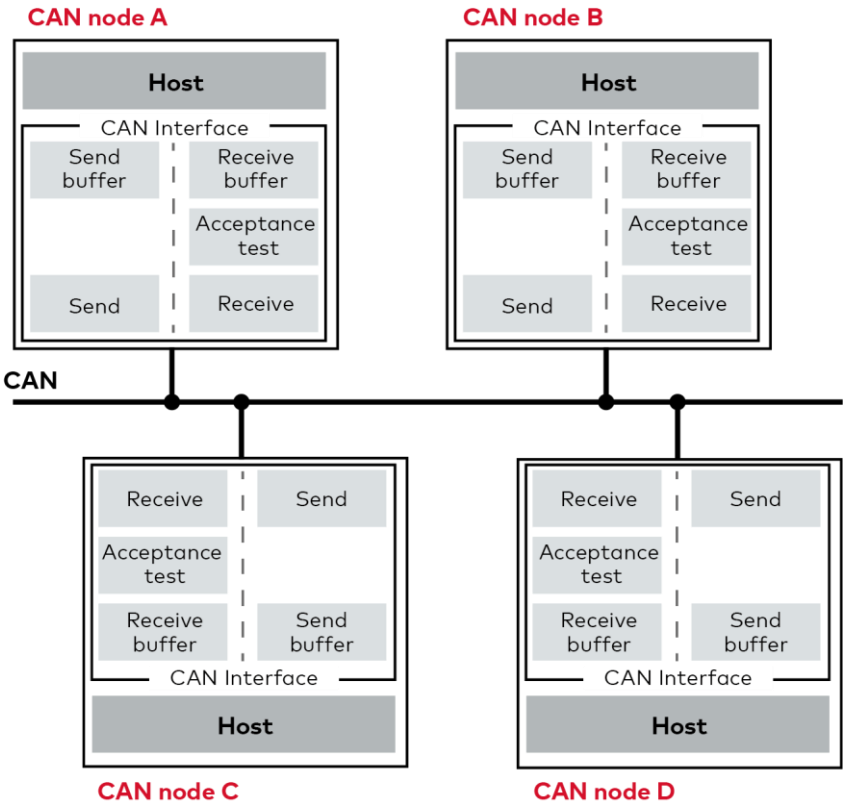


Figure 46 Representation of a CAN network

The XCP frames are not described in the communication matrix! If, for example with the help of XCP, measurement data is sent from the slave via dynamic DAQ lists, the composition of the frames is carried out by the user depending on the selected signals. If the signal composition changes, the frame content also changes.

CAN identifiers are required to transmit XCP data via CAN. To occupy as few CAN identifiers as possible, XCP communication is limited to the use of only two CAN identifiers that are not used in the DBC for "normal" communication. One identifier is needed to send information from the master to the slave, the other is used by the slave to reply to the master.

The excerpt from the CANape Trace window shows the CAN identifiers used under the "ID" column. In this example, two identifiers are used: ID 554 for the frames from the master to the slave (Dir Tx) and ID 555 for sending the frames of the slave to the master (Dir Rx).

[20] Trace - XCP

Time	ID	Name	Dir	Len	Data	Kommando
1m 7.52009s	554	CMD	Tx	2	FF 00	CONNECT mode=0
1m 7.520126s	555	RES	Rx	8	FF 1D C0 08 08 00 01 01	Ok:CONNECT resource=1Dh commMode=C0h ctoSize=8 dtoSize=8 protVer=1 trar
1m 7.520228s	554	CMD	Tx	1	FB	GET_COMM_MODE_INFO
1m 7.520251s	555	RES	Rx	8	FF 1D 01 08 2B 00 00 19	Ok:GET_COMM_MODE_INFO commModeOptional=01h maxBs=43 minSt=0 queueSize=0
1m 7.520325s	554	CMD	Tx	1	FD	GET_STATUS
1m 7.520337s	555	RES	Rx	6	FF 00 1D 08 00 00	Ok:GET_STATUS sessionStatus=00h protectionStatus=1Dh configurationID=0C
1m 7.520414s	554	CMD	Tx	1	DA	GET_DAQ_PROCESSOR_INFO
1m 7.520426s	555	RES	Rx	8	FF 57 00 00 06 00 00 40	Ok:GET_DAQ_PROCESSOR_INFO prop=57h maxDAQlists=0 evtChan=6 predefDAQlis
1m 7.520499s	554	CMD	Tx	1	D9	GET_DAQ_RESOLUTION_INFO
1m 7.520511s	555	RES	Rx	8	FF 01 06 01 06 44 0A 00	Ok:GET_DAQ_RESOLUTION_INFO granularityODTEntryDAQ=1 maxSizeODTEntryDAQ=
1m 7.520584s	554	CMD	Tx	4	D7 00 00 00	GET_DAQ_EVENT_INFO eventChannel=0
1m 7.520595s	555	RES	Rx	7	FF 04 01 05 00 06 00	Ok:GET_DAQ_EVENT_INFO eventProp=04h maxDAQLists=1 length=5 timeCycle=0
1m 7.520668s	554	CMD	Tx	2	F5 05	UPLOAD size=5
1m 7.520679s	555	RES	Rx	6	FF 4B 65 79 20 54	Ok:UPLOAD data=4Bh
1m 7.520996s	554	CMD	Tx	4	D7 00 01 00	GET_DAQ_EVENT_INFO eventChannel=1
1m 7.521009s	555	RES	Rx	7	FF 0C 01 05 0A 06 00	Ok:GET_DAQ_EVENT_INFO eventProp=0Ch maxDAQLists=1 length=5 timeCycle=1C
1m 7.521082s	554	CMD	Tx	2	F5 05	UPLOAD size=5
1m 7.521098s	555	RES	Rx	6	FF 31 30 20 6D 73	Ok:UPLOAD data=31h
1m 7.521133s	554	CMD	Tx	4	D7 00 02 00	GET_DAQ_EVENT_INFO eventChannel=2
1m 7.521166s	555	RES	Rx	7	FF 0C 01 05 64 06 00	Ok:GET_DAQ_EVENT_INFO eventProp=0Ch maxDAQLists=1 length=5 timeCycle=1C
1m 7.521197s	554	CMD	Tx	2	F5 05	UPLOAD size=5
1m 7.521237s	555	RES	Rx	6	FF 31 30 30 6D 73	Ok:UPLOAD data=31h

Figure 47 Example of XCP on CAN communication

The CAN bus transmits a maximum of eight data bytes per frame. In the case of XCP, however, information about the command used or the response sent is required. The first byte of the CAN data bytes is used for this purpose. Thus, seven bytes are available for the transport of user data per CAN frame.
In CANape you will find several XCP on CAN sample configurations with the virtual ECU XCPsim. For more details about the standard, see the ASAM XCP on CAN Transport Layer specification.

1.6.2. CAN FD

CAN FD (FD: Flexible Data rate) is an extension of the CAN protocol. The main differences to CAN are the extension of the user data from 8 to up to 64 bytes. In addition, CAN FD offers the possibility to send the user data with an increased data rate. Thus, the data bytes can be sent with a higher transmission rate after the arbitration phase than during the arbitration phase.

Starting with the XCP specification 1.2, XCP on CAN FD is supported. The information about the two different transmission rates must be reflected in the A2L. Therefore, a CAN FD transport layer block is needed in the A2L file. It is not possible to use CAN and CAN FD frames on the same physical bus at the same time.

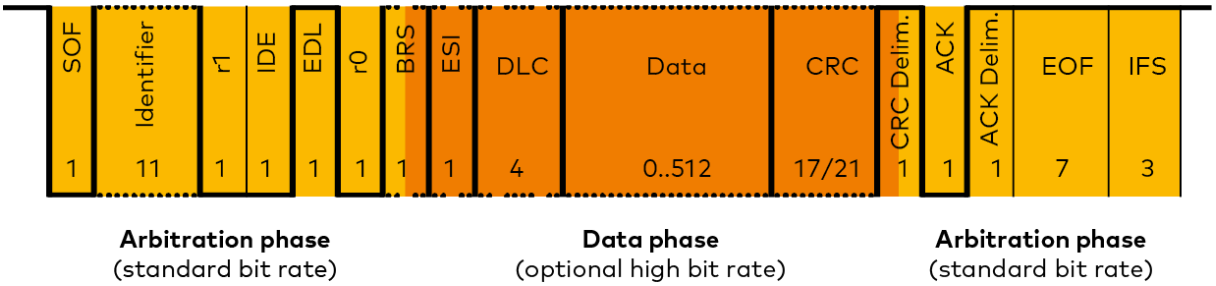


Figure 48 Representation of a CAN FD frame

CAN FD requires enhancements and adaptations in hardware and software. Among other things, CAN FD introduces three new bits in the control field:

- Extended Data Length (EDL: 0 = CAN Frame, 1 = CAN FD Frame)
- Bit Rate Switch (BRS: 0) = No change of bit rate for data phase, 1 = Change to higher bit rate for data phase)
- Error State Indicator (ESI: 0 = CAN FD node is error active, 1 = CAN FD node is error passive)

Frames in the CAN FD format can be distinguished from the standard CAN format by means of a recessive EDL bit (High Level), since they are identified by a dominant EDL bit (Low Level). Similarly, a recessive BRS bit switches to the higher bit rate when sending the data field. The ESI bit is used to identify the error state of a CAN FD node. In addition, another four bits form the so-called Data Length Code (DLC), which maps the extended user data length as possible values of 12, 16, 20, 24, 32, 48 and 64 bytes.

The use of XCP on CAN FD requires the definition of a second transmission rate for the user data in the A2L file. For the user who receives a complete A2L file, it is completely transparent. A measurement configuration in the XCP master takes the maximum packet length into account, and the user does not have to worry about any further settings.

1.6.3. FlexRay

A basic idea in the development of FlexRay was the realization of a redundant system with deterministic timing. The connection redundancy was created by using two channels: Channel A and Channel B. If several FlexRay nodes (= control units) are connected redundantly and one channel fails, the nodes can switch to the other channel and thus create connection redundancy.

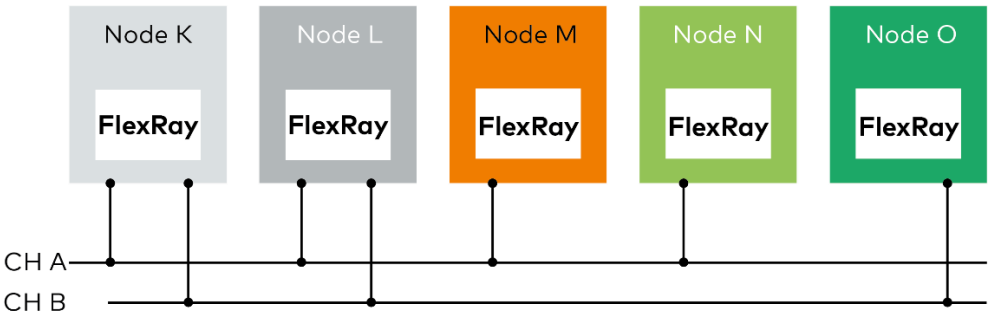


Figure 49 Nodes K and L are redundantly connected to each other

The deterministic behavior is achieved by transmitting data within defined time slots. This also defines which subscriber sends which content in which time slot. These time slots are combined to form a cycle. The cycles are repeated as long as the bus is active. The compilation of the time slots and the transmission contents (who sends what at what time) is called scheduling.

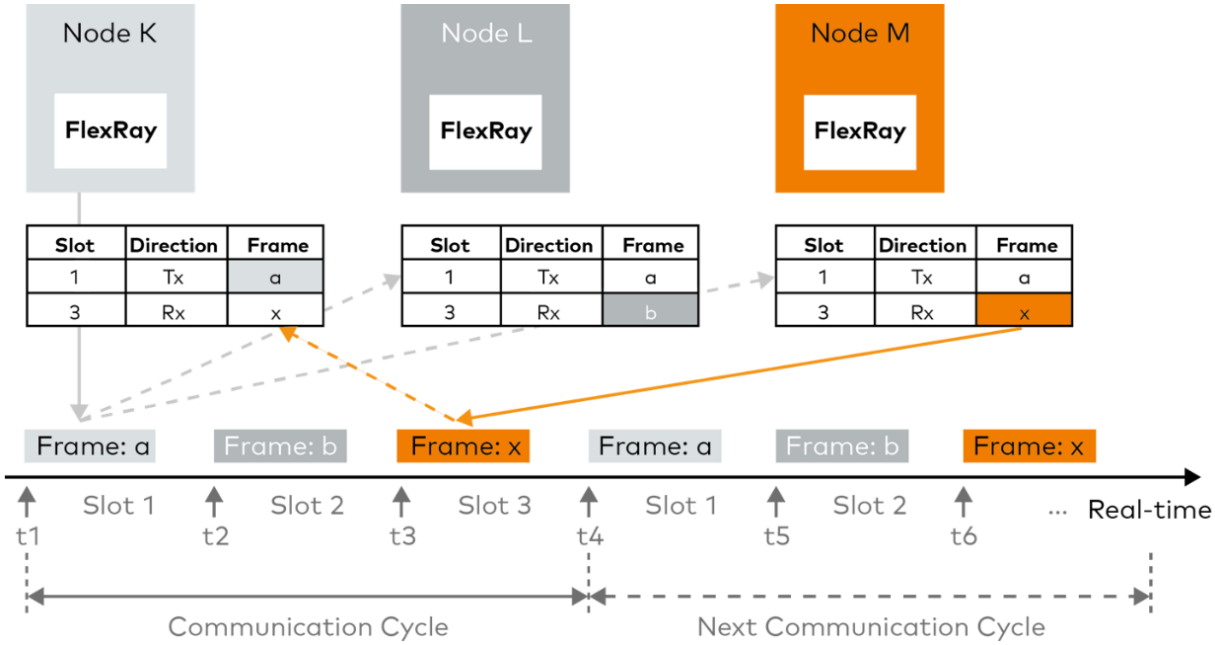


Figure 50 Communication via slot definition

In the first communication cycle, node K sends the frame a in slot 1. Scheduling is also stored in the software of nodes L and M. The frame a is then sent to the higher communication layers. Therefore, the content of frame a is passed on to the respective higher communication layers.

The scheduling is summarized in a description file. However, this is not a DBC file, as in the case of CAN, but a FIBEX-file. FIBEX stands for "Field Bus Exchange Format" and could also be used for other bus systems. However, the current use is practically only for the description of the FlexRay bus. FIBEX is an XML format and the XCP on FlexRay specification refers to the FIBEX version 1.1.5 and the FlexRay specification version 2.1.

	Slot	ECU	Channel	Cycles								
				0	1	2	3	4	5	6	...	63
Static Segment	1	Node K	A	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]		b [rep: 1]
			B	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]	b [rep: 1]		b [rep: 1]
	2	Node M	A	c [rep: 4]	x [rep: 2]	y [rep: 4]	x [rep: 2]	c [rep: 4]	x [rep: 2]	y [rep: 4]		x [rep: 2]
			B									
	3	Node L	A	a [rep: 1]	a [rep: 1]	a [rep: 1]	a [rep: 1]	a [rep: 1]	a [rep: 1]	a [rep: 1]		a [rep: 1]
			B	d [rep: 1]	d [rep: 1]	d [rep: 1]	d [rep: 1]	d [rep: 1]	d [rep: 1]	d [rep: 1]		d [rep: 1]
Dynamic Segment	4	Node L	A	n [rep: 1]	n [rep: 1]	n [rep: 1]	n [rep: 1]	n [rep: 1]	n [rep: 1]	n [rep: 1]		n [rep: 1]
		Node O	B	m [rep: 1]	m [rep: 1]	m [rep: 1]	m [rep: 1]	m [rep: 1]	m [rep: 1]	m [rep: 1]		m [rep: 1]
	5	Node N	A	r [rep: 1]	r [rep: 1]	r [rep: 1]	r [rep: 1]	r [rep: 1]	r [rep: 1]	r [rep: 1]		r [rep: 1]
			B									
	6		A									
		Node K	B	o [rep: 1]	o [rep: 1]	o [rep: 1]	o [rep: 1]	o [rep: 1]	o [rep: 1]	o [rep: 1]		o [rep: 1]
	7	Node M	A		t [rep: 2]	p [rep: 4]	t [rep: 2]		t [rep: 2]	p [rep: 4]		t [rep: 2]
			B									
		Node L	A	u [rep: 4]				u [rep: 4]				
		Node L	B			v [rep: 8]						
			A									
		Node O	B		w [rep: 4]				w [rep: 4]			

Figure 51 Representation of a FlexRay communication matrix

As a result of the development of AUTOSAR solutions, a further format for describing bus communication was defined: the AUTOSAR Description File, which is in XML format. In the AUTOSAR 4.0 specification, the aspect of the XCP on FlexRay definition is also considered.

Due to further characteristics of the FlexRay bus, the specification of the slot number alone is not sufficient as a reference to the content. For example, multiplexing is supported: If a cycle repeats, the same content does not have to be sent each time. Multiplexing can be used, for example, to specify that information is only ever sent every second time the slot is passed through.

Instead of the pure slot number, the so-called "FlexRay Data Link Layer Protocol Data Unit Identifier" (FLX_LPDU_ID) is used, which can be thought of as a kind of generalized slot ID. In order to describe such an LPDU, four specifications are required:

> FlexRay Slot Identifier (FLX_SLOT_ID)
> Cycle Counter Offset (OFFSET)
> Cycle Counter Repetition (CYCLE_REPETITION)
> FlexRay Channel (FLX_CHANNEL)

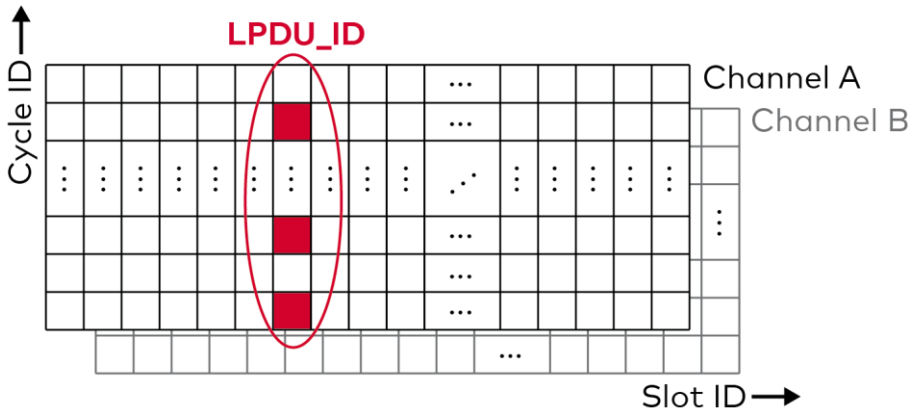


Figure 52 Representation of the FlexRay LPDUs

Scheduling also has an impact on the use of XCP on FlexRay since it defines exactly what is sent. With XCP, this cannot be defined easily. The user determines which measurement data is sent by compiling the signals only at runtime of the measurement. Thus, it can only be controlled which aspect of the XCP communication can be used in which LPDU: CTO or DTO from master to slave or from slave to master.

The following example explains the process: In slot n the XCP master may send a command (CMD) and in slot n + 2 slave A answers with the response (RES). XCP on FlexRay frames are always defined with the aid of LPDUs.

The A2L description file, in which the objects with the corresponding address in the ECU are defined, is required for access to ECU-internal variables. In addition, the FIBEX file is required so that the XCP master knows with which LPDUs it may send and on which LPDUs the XCP slaves send their responses. Only in combination, i.e. the establishment of the reference of an A2L file to a FIBEX file, the communication between XCP master and XCP slave(s) can work.

Excerpt from an A2L with XCP on FlexRay configuration:

```
        ...
/begin XCP_ON_FLX
                ...
"XCPsim.xml"
"Cluster_1"
        ...
```

Here "XCPsim.xml" is the reference from the A2L to the FIBEX file.

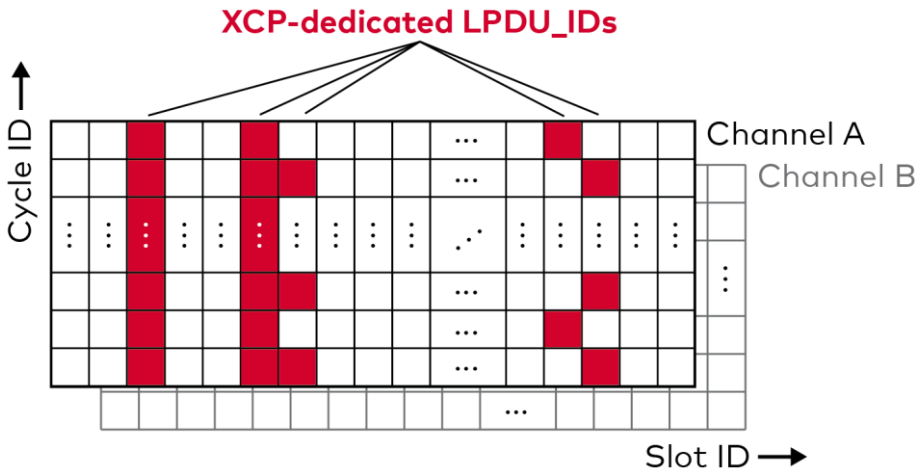


Figure 53 Assignment of XCP communication to LPDUs

Further details on XCP on FlexRay can be found, for example, in the CANape help. CANape comes with a so-called FIBEX Viewer that allows convenient viewing of the scheduling. The XCP frames are conveniently assigned to the LPDUs via the driver settings of the XCP on FlexRay device in CANape.

The protocol is explained in detail in the ASAM XCP on FlexRay Transport Layer specification. An XCP on FlexRay sample configuration with the XCPsim virtual ECU can be found in CANape. The sample configuration assumes real Vector FlexRay hardware.

1.6.4. Ethernet

XCP on Ethernet can be operated either with TCP/IP or with UDP/IP. TCP is a secured transport protocol on Ethernet in which the loss of a packet is detected by means of the handshake procedure. TCP organizes the retransmission of the packet in case of packet loss. UDP does not provide this security mechanism. If a packet is lost here, UDP does not provide any mechanisms for resending the lost packet at this protocol level.

XCP on Ethernet can be used not only with real ECUs, but also to measure and calibrate virtual ECUs. A virtual ECU is the use of the code that otherwise runs in the ECU as an executable program (e.g. DLL) on the computer. Other resources are available there than in an ECU (CPU, memory ...).

But first to the actual protocol. IP packets always contain the address of the sender and the recipient. The simplest way to think of an IP packet is as a kind of letter on which the addresses of the recipient and the sender are written. The addresses of the individual subscribers must always be unique. A unique address includes IP address and port number.

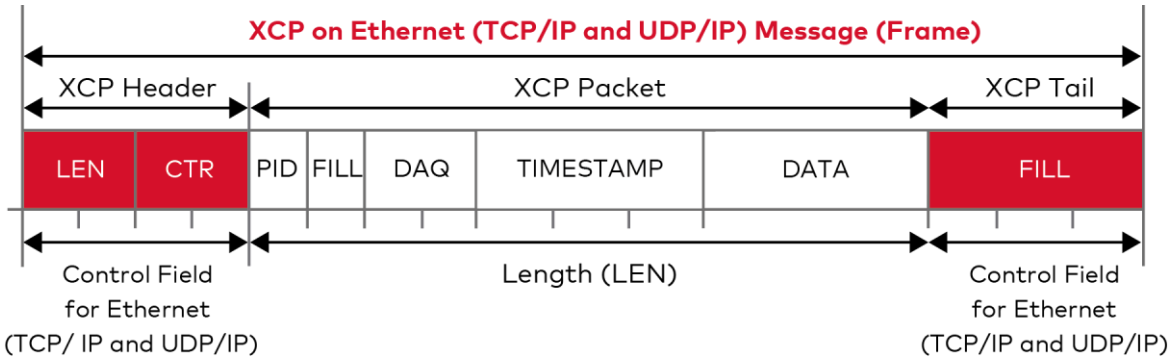


Figure 54 XCP packet with TCP/IP or UDP/IP

The header consists of a control field with two words in Intel format (= four bytes). They contain the length (LEN) and a counter (CTR). LEN specifies the number of bytes of the XCP packet. The CTR is used to identify a packet loss. UDP/IP is not a secure protocol. If a packet loss occurs, it is not intercepted by the protocol layer. The control is done by the counter information. When the master sends the packet to the slave, it generates a counter number that is incremented each time a packet is sent. The slave responds with the same pattern: it increments its own counter with each packet it sends. The counter of the slave and the counter of the master work independently. It depends on the measuring task whether UDP or TCP is to be used.

For XCP on Ethernet (TCP/IP and UDP/IP) the end consists of a control field with 0,1,2 or 3 FILL bytes.

An Ethernet packet can carry multiple XCP packets, but an XCP packet must never exceed the limits of a UDP/IP packet.

1.6.4.1. Detecting XCP on Ethernet slaves

With the enhancements through version 1.3 of the XCP standard, an extension for XCP slave detection was defined specifically for XCP on Ethernet.

The master can detect XCP slaves via the GET_SLAVE_ID command. The master sends out a multicast packet (IPv4) with the IP address 239.255.0.0 on port 5556. Regardless of whether an XCP slave already has a connection to a master or not, the slave must process the request and send back a response.

The response of the slave contains among others:

- the IP address (IPv4)
- the port number
- whether TCP, UDP or both are supported
- and an information about the status (whether there is already a connection to an XCP master or not)

For more information on the protocol, see the ASAM XCP on Ethernet Transport Layer specification. In CANape you will also find an XCP on Ethernet sample configuration with the virtual ECU XCPsim or with virtual ECUs in the form of DLLs implemented using Simulink models and the Simulink Coder.

1.6.5. Sxl

Sxl is the collective name for SPI or SCI. Since this is not a data bus but a controller interface that is only suitable for point-to-point connections, there is no addressing for this transmission. The communication between both participants is either synchronous or asynchronous.

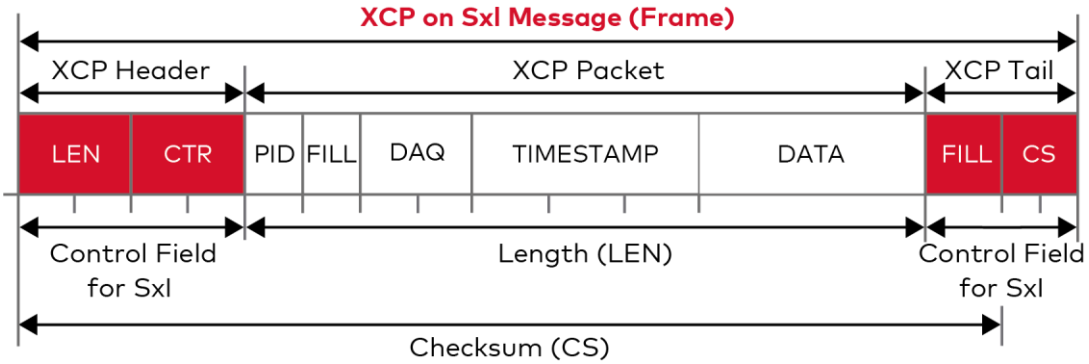


Figure 55 XCP on Sxl package

The XCP header consists of a control field with two pieces of information: the length LEN and the counter. The length of these quantities can be byte or word (Intel format). LEN indicates the number of bytes of the XCP packet. The CTR is used to identify packet loss. The control is done in the same way as for XCP on Ethernet via the counter information. Under certain circumstances it may be necessary to add padding bytes to the packet, for example if SPI is used in WORD or DWORD mode or if the minimum packet length would be undershot. These fill bytes are appended in the control field.

For more information about the protocol, see the ASAM XCP on Sxl Transport Layer specification.

1.6.6. USB

XCP on USB has no practical significance. Therefore, no further space is given to the topic here, but only reference is made to the documents of the standard.

1.6.7. LIN

Currently, no XCP on LIN standard is defined. However, there is a solution from Vector (XCP on LIN driver and CANape as XCP on LIN master) that violates neither the LIN nor the XCP specification and is already being used in some customer projects. For more detailed information, please contact Vector.

1.7. XCP Services

This chapter lists and explains further services that can be implemented via XCP. They are all based on the mechanisms of communication using CTOs and DTOs already described. Some XCP services have already been explained, e.g. synchronous data acquisition/stimulation or read/write access to the device memory.
Although the XCP specification clearly defines the different services, it also states whether the service must be implemented in any case or whether it is optional. For example, an XCP slave must support a "Connect" for the master to establish a connection. However, flashing via XCP is not mandatory, so the XCP slave does not have to support it. It simply depends on the project and software requirements. All services listed in this chapter are optional.

1.7.1. Memory page switching

As already explained in the description of the calibration concepts, parameters are normally located in the flash memory and are copied to RAM if required. Some calibration concepts offer the possibility of switching memory segment pages from RAM and Flash. XCP describes a somewhat more general, generic approach where a memory segment can contain multiple switchable pages. Normally, this is a RAM page and a Flash page. However, multiple RAM pages or the absence of the Flash page are also conceivable.

To better understand the XCP commands for page switching, the terms sector, segment and page should be explained at this point.

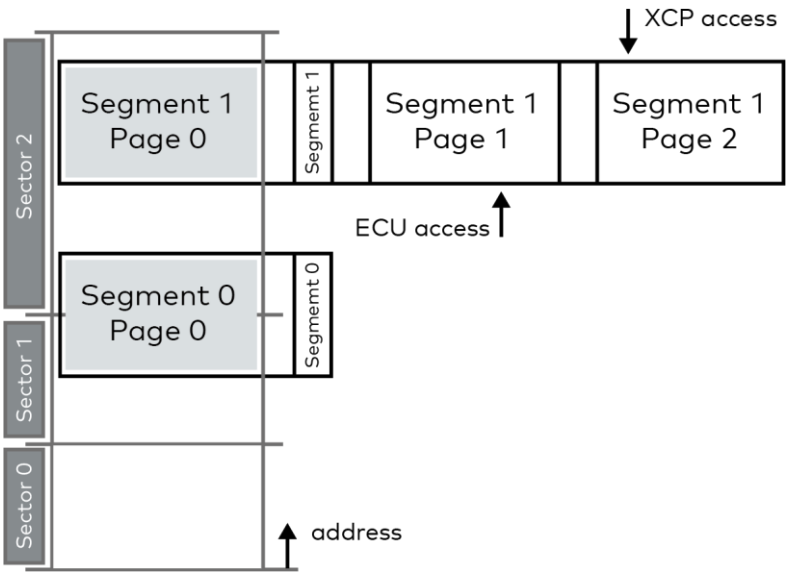


Figure 56 Memory representation

From the XCP point of view, the memory of a slave consists of a continuous memory that is addressed with a 40-bit width. The physical structure of a memory is in sectors. Knowledge of the Flash sectors is mandatory for flashing because the Flash memories can only be erased block by block.

The logical structure takes place in so-called segments, which describe where calibration data is located in the memory. The start address and the size of a segment do not have to be oriented to the start addresses and sizes of the physical sectors. Each segment can be divided into several pages. The pages of a segment describe the same parameters on the same addresses. The values of these parameters and the write/read rights can be regulated individually for each page.

The assignment of the algorithm to a page within a segment must be always unique. Only one page can be active in a segment at any one time. This page is the so-called "active page for the control unit in this segment". Which page is actively accessed by the control unit and the XCP driver can be switched individually. There is no dependency between these settings. Based on the name for the control unit, the active page for XCP access is called "active page for XCP access in this segment".

This again applies individually to each segment. Segments must be specified in the A2L file, and each segment is assigned a number via which the segment is referenced. Within an XCP slave the SEGMENT_NUMBER must always start at 0 and then be incremented continuously.
Each segment has at least one page. The pages are also referenced by numbers. The first page is PAGE 0. One byte is available for the number, so that a maximum of 255 pages can be defined per segment.

The slave must initialize all pages for all segments. The master uses the GET_CAL_PAGE command to ask the slave which page is currently active for ECU access and which for XCP access. It can happen that a mutual locking is necessary for access. For example, the XCP slave may not access a page if this page is currently active for the ECU. As previously mentioned, there can be a dependency, but there doesn't have to be. It is a question of how the slave implementation is realized.

If the slave supports the optional commands GET_CAL_PAGE and SET_CAL_PAGE then it also supports the so-called page switching. Using these two commands, the master can therefore query which pages are currently being used and can switch them over for the control unit and XCP access if required. The XCP master has full control over the switching of the pages. The XCP slave cannot trigger a switchover on its own. But of course, the master must respect the restrictions resulting from the implementation in the slave.

What is the benefit of switching?
On the one hand, the switchover allows a sudden exchange of complete conditions - a quasi before-and-after comparison. On the other hand, the controlled system is in a stable state while the applicator makes extensive parameter changes on another side in the control unit. This prevents a critical or unstable state of the controlled system, e.g., due to incomplete data statuses in the control system.

1.7.2. Backup of memory pages - Data Page Freezing

If an application engineer adjusts parameters on a page, XCP offers the conceptual option of saving the data directly in the control unit. In this case, the data of a RAM page is saved to a page with non-volatile memory. If the non-volatile memory is a Flash, it must be considered that the segment start address and the segment size do not necessarily have to match the Flash sectors, which poses a problem when erasing and rewriting the flash memory (see ASAM XCP Protocol Layer specification).

1.7.3. Flash programming

Flashing means writing data to an area of the flash memory. To do this, it must be known exactly how the memory is structured. A Flash memory is divided into several sectors (physical sections), which are written with a start address and a length. To distinguish them, they are given a consecutive identification number. One byte is available for this purpose so that the maximum number of sectors is 255.

SECTOR_NUMBER [0, 1, 2 ... 255]

The information about the flash sectors is also part of the A2L data.

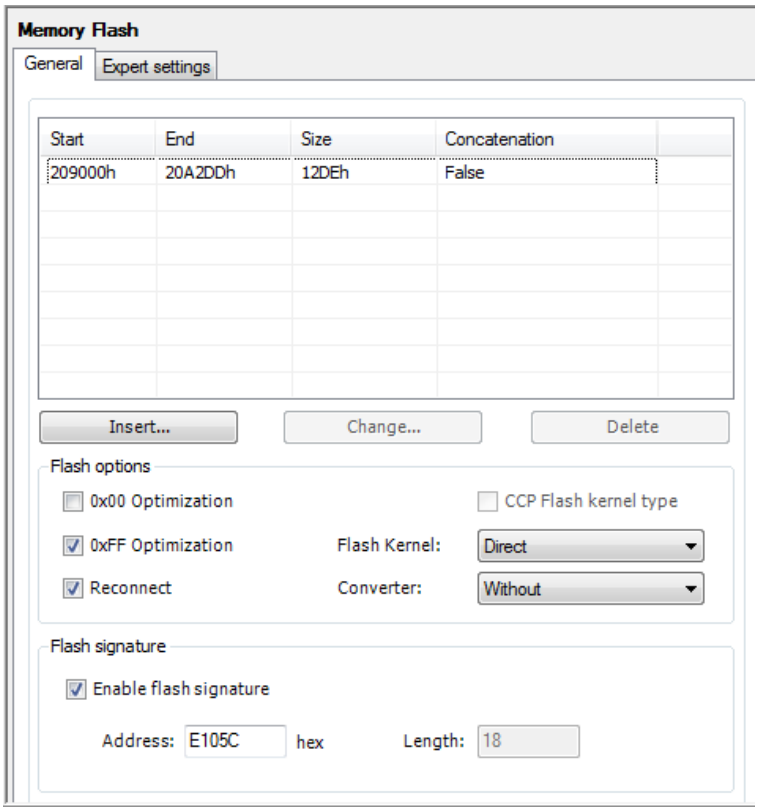


Figure 57 Illustration of the driver setting for the Flash area

Flashing can be realized via so-called "Flash Kernels". This is an executable code that is sent to the RAM area of the slave before the actual flashing and then takes over the communication with the XCP master. This may contain the algorithm responsible for erasing the Flash memory. For safety and space reasons, this is very often not permanently present in the Flash memory of the control unit. Under certain circumstances, a converter is used if, for example, checksum or similar calculations are to be performed.

Flashing via XCP roughly divides the entire flash process into three areas:

> Preparation (e.g. for version control and thus to check whether the new content can be flashed at all)
> Execution (the new content is sent to the control unit)
> Follow-up (e.g. checksum control etc.)

In the XCP standard, the main focus is on the actual execution of flashing. If you compare this process with flashing via diagnostic protocols, you will see that the process-specific elements, such as serial number handling with metadata, are supported rather sparsely in XCP. Flashing in the development phase was clearly the focus of the definition, not complex process steps that are necessary at the end of the line.

In the preparation phase, it is therefore essentially a matter of determining whether the new content fits the ECU at all. There are no special commands for version control. Rather, the practice has established itself that project-specific definitions are made as to which commands are supported.

The following XCP commands are available:

PROGRAM_START: Start of the flash sequence
This command indicates the start of the flash process. If the ECU is in a state that does not allow flashing (e.g. vehicle speed > 0), the XCP slave must acknowledge with an error (ERR). Only when the PROGRAM_START is successfully acknowledged by the slave may the actual flash process begin.

PROGRAM_CLEAR: Call of the actual flash memory erase routine.
Before a flash memory can be overwritten with a new content, it must first be erased. The call of the erase routine via this command must be implemented in the control unit or made available to the control unit with the help of the Flash Kernel.

PROGRAM_FORMAT: Selection of the data format of the flash data
The XCP master uses this command to specify the format (e.g. compressed or encrypted) in which the data is transferred to the slave. If the command is not sent, the default setting is non-compressed and non-encrypted transmission.

PROGRAM: Transmission of the data to the XCP slave
For the users who are familiar with flashing via diagnostics: This command corresponds to TRANSFERDATA in the diagnostics. With the help of this command, data is transferred to the XCP slave, which is then stored in the Flash memory.

PROGRAM_VERIFY: Request to verify the new flash content.
The master can request the slave to perform an internal check - whether the new content is OK.

PROGRAM_RESET: Reset request to the slave
Request of the master to the slave to execute a reset. After that the connection to the slave is in any case disconnected and a new CONNECT must be sent.

1.7.4. Automatic detection of the slave

By reading the A2L file, the master knows the details of the slave. However, the XCP protocol also allows the master to query the slave about its protocol-specific properties. A number of commands are available for this purpose:

GET_COMM_MODE_INFO
The response to this command provides the master with information about various communication options of the slave, e.g., whether block transfer or interleaved mode is supported and which minimum time intervals between requests in these modes must be observed by the master.

GET_STATUS
The response to this query provides all status information of the slave. This includes which resources (CAL/PAG, DAQ, STIM, PGM and DBG) are generally available and which of these resources are currently active (e.g., memory activities).

GET_DAQ_PROCESSOR_INFO
The master receives the general information it needs to know about the restrictions of the slave: E.g., number of predefined DAQ lists, available DAQ lists and events.

GET_DAQ_RESOLUTION_INFO
Further information about the DAQ capabilities of the slave is exchanged via this command: maximum size of an ODT for DAQ and for STIM, granularity of the ODT entries, number of bytes for timestamp transmission.

GET_DAQ_EVENT_INFO
If this command is used, the call is made once per event of the control unit. Information is requested about whether the event can be used for DAQ, STIM or DAQ/STIM, whether the event occurs cyclically and, if so, what the cycle time is.

There are other commands, like GET_DAQ_LIST_MODE and GET_DAQ_LIST_INFO. This queries information about predefined DAQ lists.

1.7.5. Block transfer mode for upload, download and flashing

In the standard communication mode, each command of the master is acknowledged by a response of the slave. In some cases, however, it is desirable for performance reasons to use the so-called block transfer mode.

The master learns whether the slave supports the method by means of the GET_COMM_MODE_INFO query.

1.7.6. Saving a DAQ configuration and direct data transfer on power-up

Before a DAQ measurement can be made via XCP, the DAQ configuration must be present in the slave. XCP provides a mechanism to load the DAQ configuration in the slave into a non-volatile memory so that the configuration is already available when the slave is started. This is called RESUME mode.

This has two benefits: it allows for cold start measurements, and it saves configuration time in case of repeated, unchanged use of a measurement configuration.

The approach is based on a simple idea: The configuration and the measurement are separated in time. After the configuration phase, the measurement is not started directly, but the control unit is switched off. After rebooting, the XCP slave directly accesses the already existing configuration and immediately starts sending the first packets. The difficulties associated with this are obvious: The configuration of the DAQ lists is stored in RAM and thus the information is no longer available after a boot.

To use the RESUME mode, a non-volatile memory is required in the embedded system, which does not lose its data even without power supply. EEPROM is used for this method. Whether this is a real EEPROM or one emulated by a flash memory is irrelevant for this consideration.

1.7.7. Protection mechanisms with XCP

If possible, an unauthorized user should not be able to establish a connection to a control unit. The "seed and key" procedure is available for checking whether a connection is authorized. Seed-and-key can be used to protect the various resources: CAL/PAG (calibration and page switching), DAQ (data acquisition), STIM (stimulation), PGM (flashing) and DBG (software debugging).

The "seed-and-key" procedure works as follows: When the master makes a connection request, the slave sends a random number (= seed) to the master. Now the master must generate a response (= key) via an algorithm. The key is sent to the slave. The slave also calculates the expected response and compares the master's key with its own result. If results match, both master and slave have used the same algorithm. The slave then accepts the connection to the master. If there is no match, the slave refuses to communicate with the master.

Normally, the algorithm is available in the master as a DLL. If a user has the "seed-and-key" DLL and the A2L file, there is nothing to prevent access to the resources in the ECU. As the ECU approaches production startup, the XCP driver is often disabled. To restore XCP access to the ECU, an individual sequence of diagnostic commands is usually used. Thus, the XCP driver is still available in production vehicles for the most part but is normally deactivated to protect against manipulation of the ECU.

Whether or not seed-and-key or disabling the XCP driver is used in a project is implementation-specific and independent of the XCP specification.

2. ECU description file A2L

One reason why an A2L file is needed has already been mentioned: the assignment of symbolic names to addresses. For example, if the software developer has implemented a PID controller in his application and has given the names P1, I1 and D1 to the variables proportional, integral, and differential components, the application developer should be able to access these variables by their names. Let's take the following picture as an example:

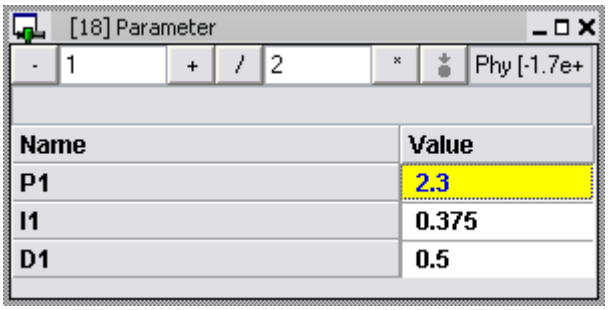


Figure 58 Parameters in a calibration window

By using symbolic names, the user can conveniently access the desired values. Another example provides the consideration of signal quantities measured from the control unit:

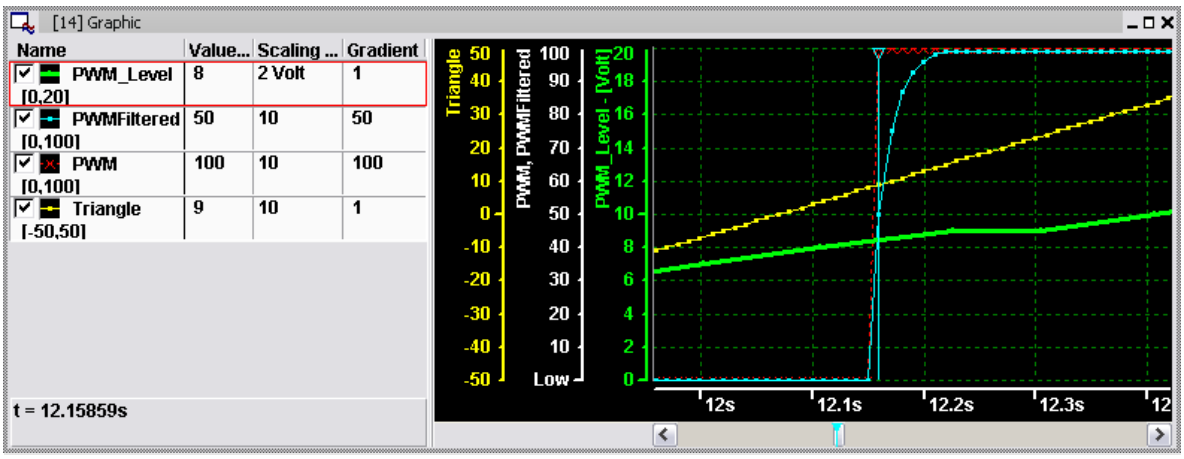


Figure 59 Signal display over time

In the legend the user can read the logical names of the signals. On which address the variables once stood in the control unit is irrelevant for offline viewing of the values. The correct address is necessary for the request for the values in the control unit, but the address itself is irrelevant for the user. He uses the logical name for selection and visualization. The user selects the object based on the name, and the XCP master looks up the associated address and data type in the A2L.

As a further attribute of a parameter, for example, a minimum and a maximum value can be defined. The value of the object must be within these limits. The limits are intended to prevent a user from using a value beyond the limits and causing damage.

Rules for the conversion between physical and raw values are also defined in the A2L. A sensor that supplies an 8-bit value serves as a simple example. The numeric values supplied by the sensor are therefore between 0 and 255, but the value of the sensor is to be displayed as a percentage. The mapping of the sensor value [0 ... 255] into [0 ... 100 %] is done by a conversion rule, which again is stored in the A2L. If the raw value of a variable is measured, the measurement and calibration tool uses the stored formula and displays the physical value.

In addition to scalar quantities, there are also characteristic curves and maps. Example for the use of a characteristic curve: A Hall sensor can be used for distance measurement. The distance is determined via the strength of a magnetic field. The relationship between magnetic field and distance is non-linear. This non-linearity makes the calculation of the distance difficult. With the help of a characteristic curve, the output value of the Hall sensor can first be linearized before the values are calculated as input variables in the algorithm.

Maps are often used instead of complex calculations. If, for example, there is a relationship y = f(x) and the function f can only be determined with a great deal of computational effort, it is often easier to calculate the values over the possible range of x and to store the results in the form of a table. If the value x is now fixed in the controller, the value y does not have to be calculated at runtime of the controller, but the map supplies the result y to the input variable x. The result y is then stored in a table. It may still be necessary to interpolate between two values, but that would be all.

How is this characteristic curve stored in the memory? Do all x-values come first and then all y-values? Or is the storage done according to the scheme: x1, y1; x2, y2; x3, y3 ...? Since here different possibilities can be used in the code, the way of the memory storage is described over a storage scheme in the A2L.

The convenience for the user results from the symbolic handling of the quantities, the direct view of the physical values and the access to complex elements such as maps without having to worry about complex storage schemes.

The communication parameters offer a further advantage. These are also stored in the A2L. The data from the A2L is used for communication between the XCP master and the XCP slave. This means that the A2L contains everything that the XCP master needs to communicate with the XCP slave.

2.1. Structure of an A2L file

The A2L file is an ASCII readable file that uses keywords to describe the various aspects:

- the interface specific parameters between measuring and calibration tool and A2L file (the description is at the beginning of the A2L file and is located in a so-called AML-tree),
- the communication parameters to the ECU,
- Storage schemes for characteristic curves and fields (keyword RECORD_LAYOUT),
- Conversion rules for converting raw values to physical values (keyword COMPU_METHOD),
- Measured variables (keyword MEASUREMENT),
- Calibration parameters (keyword CHARACTERISTIC) and
- Events suitable for triggering a measurement (EVENT keyword).

Example of a description of an object ("Shifter_B3") that could be measured:

```
/begin MEASUREMENT Shifter_B3 "Single bit signal (bit from a byte shifting)"
  UBYTE HighLow 0 0 1
  READ_WRITE
  BIT_MASK 0x8
  BYTE_ORDER MSB_LAST
  ECU_ADDRESS 0x124C02
  ECU_ADDRESS_EXTENSION 0x0
  FORMAT "%.3
  /begin IF_DATA CANAPE_EXT
    100
    LINK_MAP "byteShift" 0x124C02 0x0 0 0x0 1 0x87 0x0
    DISPLAY 0 0 20
  /end IF_DATA
/end MEASUREMENT
```

Example of a calibration parameter map with the name KF1:

```
/begin CHARACTERISTIC KF1 "8*8 BYTE no axis"
  MAP 0xE0338 __UBYTE_Z 0 Factor100 0 2.55
  ECU_ADDRESS_EXTENSION 0x0
  EXTENDED_LIMITS 0 2.55
  BYTE_ORDER MSB_LAST
  BIT_MASK 0xFF
```

```
    /begin AXIS_DESCR
      FIX_AXIS NO_INPUT_QUANTITY BitSlice.CONVERSION 8 0 7
      EXTENDED_LIMITS 0 7
      READ_ONLY
      BYTE_ORDER MSB_LAST
      FORMAT "%.0
      FIX_AXIS_PAR_DIST 0 1 8
    /end AXIS_DESCR
    /begin AXIS_DESCR
      FIX_AXIS NO_INPUT_QUANTITY BitSlice.CONVERSION 8 0 7
      EXTENDED_LIMITS 0 7
      READ_ONLY
      BYTE_ORDER MSB_LAST
      FORMAT "%.0
      FIX_AXIS_PAR_DIST 0 1 8
    /end AXIS_DESCR
    /begin IF_DATA CANAPE_EXT
      100
      LINK_MAP "map3_8_8_uc" 0xE0338 0x0 0 0x0 1 0x87 0x0
      DISPLAY 0 0 255
    /end IF_DATA
    FORMAT "%.3
  /end CHARACTERISTIC
```

The ASCII text is readable, but you need a deep understanding of all the keywords to understand it. A description of the structure can be found in the ASAM XCP Protocol Layer Specification. As a user of a tool like CANape, however, you don't have to be familiar with it. You work on the CANape GUI with the measurement and calibration variables. The way in which the objects are described in the A2L is completely transparent to you.

The following demonstrates how to create an A2L. Let's concentrate on the actual contents of an A2L and its meaning and leave the details of the A2L description language to an editor. In the following, we will use the A2L editor supplied with CANape.

2.2. Manual creation of A2L file

The A2L essentially describes the contents of the memory of the XCP slave. This in turn depends on the application located in it, which was developed as C/C++ code. After the compiler/linker run of the application code, important elements of an A2L file are already present in the linker MAP file: the name of the objects, the data types, and the addresses in memory.
However, the linker map file lacks information such as the communication parameters between the XCP master and slave. The minimum and maximum

values of the parameters, the conversion rules between raw and physical values and the storage schemes for characteristic curves and maps are also missing.

Let's start by creating an empty A2L and the communication parameters: For example, if you want to create an A2L that describes an ECU with XCP on CAN interface, create a new device in CANape and select XCP on CAN as the interface. Then you can add further communication-specific information (e.g. CAN identifier). After saving, you have an A2L that contains the entire communication part of the A2L. Now the definitions of the measurement and calibration variables are still missing.

In ASAP2 Studio (part of CANape or available as a separate tool), the linker MAP file is assigned to the A2L. Using a selection dialog, you can now select all the objects of the linker MAP file that are required in the A2L: scalar measurement and calibration variables, characteristic curves and fields. Thus, the desired objects can now be inserted and grouped into the A2L file. Further object-specific information, such as maximum and minimum values, conversion rules, etc., are also added via the ASAP2 Studio editor.

After a code change with compile and link, a new linker MAP file is present. The addresses of the objects will have changed with a probability bordering on certainty. Newly added objects can be integrated into the A2L again. In any case, however, an address update in the A2L is necessary. This is done via CANape or the ASAP2 Studio by mouse click. Using the name of the A2L object, the corresponding entry is searched for in the linker MAP file, the address is read out and updated in the A2L.

If your application changes very dynamically, meaning objects are renamed, data types are adjusted, parameters are deleted and others are added, then the manual workflow is not practical. To generate an A2L from your code, other tools are available for an automatic flow.

On the Vector homepage you can find information about the "ASAP2 Tool-Set", with the help of which you can automatically generate A2Ls from the source code in a batch process.

2.3. A2L content versus ECU implementation

If an XCP master reads in an A2L file that does not completely match the ECU, misunderstandings may occur in the communication. For example, the A2L file may contain a different value for the timestamp resolution than the one implemented in the ECU. If this is the case, the problem must be recognized and solved. The user receives support from the master, which can ask the slave via the protocol what was implemented in the slave.

XCP provides a set of functions designed for the automatic detection of the slave. This assumes that automatic detection is implemented in the slave. If the master asks the slave and the answers from the slave do not match the data in the A2L description file, the master must decide which settings it will use. With CANape, the information read from the slave has a higher priority than the information from the A2L file.

Some essential commands are presented in the chapter Automatic detection of the slave.

3. Calibration concepts

ECU parameters are constant variables that are adjusted and optimized during the development of the ECU or an ECU variant. This is an iterative process in which the optimum value of a parameter is found through repeated calibrations and measurements.

The calibration concept answers the question of how parameters in the ECU can be changed during the development and calibration phase of the ECU. There is not only one calibration concept, but several. Which concept comes into question usually depends strongly on the possibilities and resources of the microcontroller used.
Normally, parameters are stored in the flash memory of the series ECU. The underlying program variables are defined as constants in the software. To make parameters changeable at runtime during development of the ECU, additional RAM memory is required.

A calibration concept deals with the following questions, among others: How do the parameters initially get from the Flash to the RAM? How is the microcontroller's access to the RAM redirected? What is the solution if there are more parameters than can be accommodated in the RAM at the same time? How do the parameters get back to the Flash? Are changes to the parameters persistent, i.e., are they retained when the ECU is turned off?
A distinction is made between transparent and non-transparent calibration concepts. Transparent means that the calibration tool does not have to take care of the above issues since all necessary mechanisms are realized by the ECU implementation. In the following, some procedures are briefly presented.

3.1. Parameters in Flash

The software developer specifies in the source code whether a quantity is a variable or a constant, i.e., whether a parameter is in Flash or RAM memory.

C code example:

```
const float factor = 0.5;
```

The parameter "factor" represents a constant with the value 0.5. When compiling and linking the code, a place in the Flash is provided for "factor". The object is therefore assigned to an address that is in the data area of the Flash memory. The value 0.5 is located at the corresponding address in the hex file. In the linker MAP file, there is the information name (factor), address, and data type.

An obvious calibration concept is to adjust the value in the C code, create a new hex file and flash it. However, this method is very cumbersome since each

value change can only be made in the code and results in a compiler/linker run with subsequent flashing. Alternatively, only the value in the hex file can be modified and this can then be flashed again. CANape can do this. This is called "offline calibration" of the hex file.

Under certain circumstances it may be necessary with certain compilers to ensure explicitly that parameters are also stored in Flash memory in any case and are not integrated into the code, for example, and thus do not appear in the linker MAP file at all. A constant's location in flash should not be left up to chance, either, so compiler-specific pragma instructions should be utilized. To prevent the compiler from embedding in the code, the "volatile" attribute for constant values is usually sufficient. A typical definition of a Flash constant looks like this:

C code example:

```
#pragma section "FLASH_Parameter"
volatile const float factor = 0.5;
```

Calibration of parameters in the Flash is normally not feasible online (=direct writing in the Flash memory). It is true that most microcontrollers can program their Flash themselves, which is required for the purpose of reprogramming in the field. However, a Flash memory is organized in larger blocks (sectors). These can only be erased and rewritten as a unit. Targeted flashing of individual bytes is out of the question since the ECU usually does not have the resources to temporarily store the rest of the sector and reprogram it again. In addition, this process would take too much time.

Some ECUs have the option of storing data in a so-called EEPROM memory. In contrast to Flash memories, EEPROM memories can erase and program each memory cell individually. The available EEPROM memory is usually limited to a few kilobytes. An EEPROM memory is often used, for example, to store programmable parameters in the workshop or to implement a persistence mechanism in the ECU, such as for the mileage. Online calibration would be conceivable here, but it is rarely used because access to EEPROM cells is relatively slow and EEPROM parameters are usually copied to RAM memory during booting where they can be accessed directly.
ECUs that do not have a physical EEPROM memory often implement a so-called EEPROM emulation. Here, several small flash sectors are used alternately to record parameter changes so that the last valid value can always be determined. This would also make online calibration conceivable.
In both cases, the memory accesses concerned would then be intercepted in the software component of the XCP driver and implemented with the software routines of the EEPROM or the EEPROM emulation.

3.2. Parameters in RAM

The most common procedure for changing parameters at runtime ("online calibration") is to create the parameters in RAM memory.

C code example:

```
#pragma section "RAM_Parameter"
volatile float factor = 0.5;
```

Here it is specified that the parameter "factor" is a RAM variable with the initial value 0.5. When compiling and linking the code, a place in RAM will be allocated for the "factor" object and the corresponding RAM address will appear in the linker MAP file. The initial value 0.5 is stored in flash memory.

During booting of the ECU, all RAM variables are initialized once with their initial values from the flash memory. This is mostly already performed in the start-up code of the compiler manufacturer; the application programmer does not have to take care of this. The application uses the values of the parameters that are in RAM, and these can be modified via normal XCP memory accesses.

Calibration parameters in RAM are still immutable from the point of view of the ECU software, i.e. the application itself does not change them. Many compilers discover this fact by code analysis and simply optimize away the required RAM memory space. Normally it is therefore also necessary here to prevent the compiler from optimizing with the "volatile" attribute.

The RAM area where the parameters are located is called Calibration RAM from the point of view of the calibration tool.

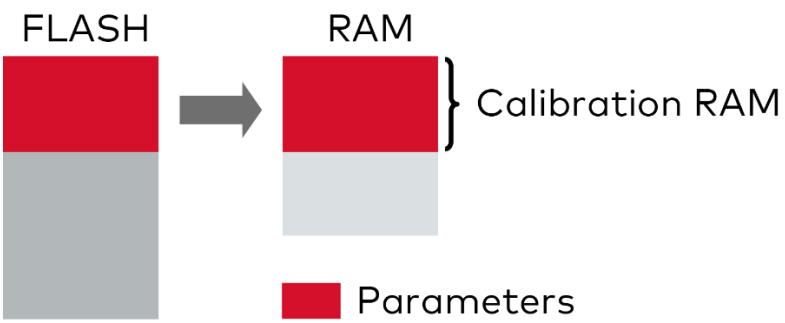


Figure 60 Initial RAM data

The Calibration RAM does not have to consist of a completely contiguous RAM area. It can also be distributed over several areas or even arbitrarily. However, it offers significant advantages to arrange the parameters in a few contiguous RAM areas and to separate them from other RAM quantities, such as state variables and intermediate results that can be changed by the application software. This is especially important if an offline calibration of the Calibration

RAM with a hex file should be possible. At the user's request, the calibration tool must be able to load the variables changed offline into the ECU during the transition from offline to online calibration.
This case occurs very often. For example, when an applicator reconnects to the control unit on the next working day, he wants to continue working at the point where he left off the previous evening. However, by booting the control unit, the flashed content has been copied to the RAM as initial data. In order for the user to continue working from the previous day, he must load his parameter set file saved the previous evening into the RAM of the control unit. This loading process can be optimized in terms of time by keeping the number of necessary transfers to a minimum. For this purpose, it is advantageous if the tool can quickly and reliably determine whether differences exist by checksum formation over larger contiguous ranges. If there are no differences between the calibration RAM contents in the ECU and the file modified by the tool, this area does not have to be transferred. If the memory area containing the calibration variables is not clearly defined, or if it contains variables that are modified by the ECU software, checksumming always results in a difference and the parameter values are transferred, either from the ECU to the XCP master or in the opposite direction. Depending on the transfer speed and the amount of data, the transfer takes several minutes.

Another advantage of clearly defined memory segments is the use of the memory area of the initial values in the flash memory for offline calibration. The content of the flash memory is defined by means of flashable hex files. If the calibration tool knows the position of the parameters in the hex file, it can change their value and realize new initial values in the ECU by flashing the changed hex file.
The calibration tool must not only know the position of the parameters in the RAM, but also that of the initial values in the Flash. The prerequisite is that the RAM memory segment is initialized from an identically structured memory segment in the Flash by copying, as is implemented in most compilers/linkers. If the addresses of the parameters in the RAM are in the A2L file, the tool must only know which offset must be added to the start address of the calibration RAM in order to arrive at the start address of the corresponding Flash area. This offset is then valid for every single parameter in the A2L.

The calibration tool can then either create flashable hex files of this range itself or it directly relies on the original hex files of the linker to change the initial values of the parameters in the hex file.

3.3. Flash overlay

Many microcontrollers offer possibilities to overlay memory areas in the flash with internal or external RAM. This process is called flash emulation or flash overlay. From the use of a memory management unit to dedicated

mechanisms that serve exactly this purpose, many things are possible. In this case, the parameters are created as parameters in the flash, as in calibration concept 1. This method has enormous advantages over the described calibration concept 2 "Parameters in RAM":

- Flash and RAM addresses are not differentiated. The A2L file, the hex file and the linker MAP file always contain the flash addresses. This provides clear relations; the Hex file is directly flashable and the A2L file fits directly to it.
- The overlay can be switched on and off as a whole, which makes it possible to switch between the values in Flash and those in RAM at lightning speed. This is referred to as the RAM page and the Flash page of a memory area (segment). XCP supports the control of memory page switching with its own commands.
- If necessary, the memory pages can be switched separately for XCP access and ECU access, i.e. XCP can access one memory page while the ECU software works with the other. This allows, for example, the offline calibration data to be downloaded to RAM while the ECU is still working with the flash data and avoids possible inconsistencies that can be problematic with a running ECU.
- The overlay with RAM does not have to be complete and can be adapted to the application. It is possible to operate with less RAM than with flash. But more about this later.

A typical procedure for establishing the connection from the calibration tool to the control unit with the subsequent download of offline calibrated values is as follows:

Connection establishment with ECU	CONNECT
Connection XCP master to RAM page	SET_CAL_PAGE XCP to RAM
Checksum calculation	CALC_CHECKSUM

If a difference has been detected during the checksum calculation via the RAM, there is normally first a query to the user as to how to proceed. Should the content of the ECU RAM be sent to the master or the content of a file on the master side to the RAM of the ECU? If the user decides to write the offline changes to the ECU, the further procedure is as follows:

ECU should use the Flash Page printing	SET_CAL_PAGE ECU to FLASH
Copy file from master to RAM page	DOWNLOAD ...
ECU should use RAM page data	SET_CAL_PAGE ECU to RAM

Finally, the memory page is always switched to RAM so that parameters can be changed. However, the user can also explicitly determine which memory page should be active in the control unit. For example, the behavior of the RAM

parameter set can be compared with the flash parameter set or, in an emergency, it is possible to switch back to a tried and tested parameter set in flash in a flash.

3.4. Dynamic Flash Overlay Allocation

The concepts described so far with calibration RAM are unproblematic if enough RAM is available for all parameters. But what if the total amount of parameters does not fit into the available RAM area?

In this case, it is a good idea to carry out the overlaying of flash with RAM dynamically and only overlay the affected flash memory area with RAM when a parameter is actually written to. This process can take place with a certain granularity and - depending on the implementation - is transparent for the calibration tool from the XCP point of view. When the XCP driver in the ECU detects a write access to the flash that would result in a change, part of the calibration RAM is used to recopy the corresponding part of the flash and activate the overlay mechanism for this part. For this purpose, the RAM is allocated, i.e. permanently assigned and marked as occupied. However, the resources of the calibration RAM are limited. During the calibration process, a RAM area that has been allocated once is not released again so that the available calibration RAM runs out with further requests. If the RAM resource is used up and a new allocation is requested, the user is notified of the exhausted RAM resource. He is then offered the option of flashing or saving the changes made so far. This frees up the allocated RAM area again and the user can calibrate again. The variant that the control unit automatically flashes the previously changed parameters is usually ruled out for the reasons already mentioned in the calibration concept "Parameters in flash".

In some cases, it may happen that the download of a parameter set created offline is infeasible due to insufficient RAM resources. In this case, the only option is to flash it. The user can undo the changes from the tool at any time, freeing up the allocated RAM blocks.

Page switching between RAM and Flash Page is also possible with this concept without any restrictions.
The parameters should be arranged in the flash in a functionally related manner so that the available RAM blocks are used as efficiently as possible. The software developer then determines that the variables that belong together thematically are also located in a contiguous memory area. After copying into RAM, the parameters necessary for tuning this function are completely available.

3.5. RAM pointer-based calibration concept according to AUTOSAR

The concept does not necessarily require the use of an AUTOSAR operating system; it can also be used in a different environment - e.g. without an operating system. The concept has a basic similarity to the previous one. The main difference is that the replacement of flash with RAM is not realized by hardware mechanisms, but by software mechanisms. The calibration parameters are always referenced by the ECU software via pointers. Flash or RAM contents are accessed by changing these pointers. The flash parameters to be changed are copied into a specified block of free RAM. This procedure, just like the previous ones, can be implemented completely transparently from the XCP point of view. Alternatively, the user of the calibration tool explicitly selects the quantities to be changed, for example by preselecting the desired parameters. This has the advantage that the resource allocation and utilization is visible to the user, and he is not surprised in the middle of the work by the situation that memory space is missing.

3.5.1. Single pointer concept

The pointer table is in the RAM. When booting the control unit, all pointers point to the parameter values in the flash. The position and size of the calibration RAM is known, but there are no parameter values in it after booting. The application initially works completely from the flash.

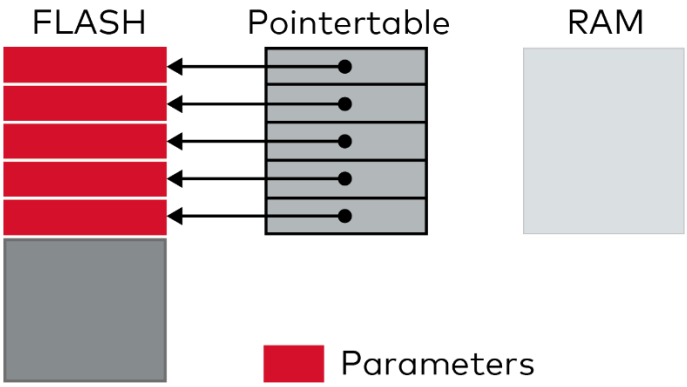


Figure 61 Initial situation after booting

If a user selects a parameter from the A2L file for the first time after booting and wants write access to it, this first triggers a copy operation within the controller. The XCP slave determines that the address to which access is to be made is in the flash area and copies the value of the parameter to the calibration RAM. To ensure that the application does not continue to fetch the parameter value from the flash area but from the RAM area, an additional change is made in the pointer table:

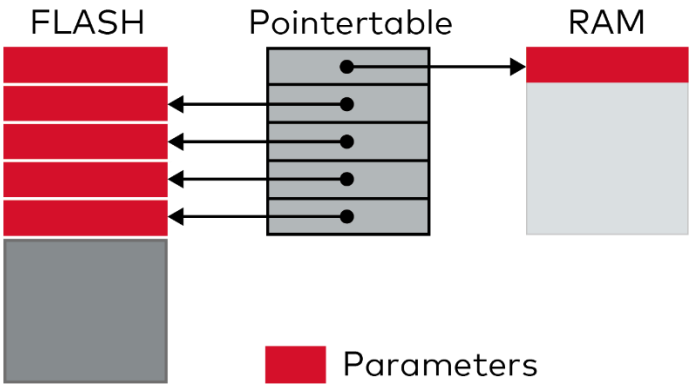


Figure 62 Pointer change and copy to RAM

The application still gets the value of the parameter via the pointer table. However, since the pointer now points to the RAM address, the value is fetched from there. Thus, the user can change the value of the parameter via XCP and observe the effect of the change via the measurement. The disadvantage of this procedure is that an entry in a pointer table must be available for each parameter, and this in turn is associated with a considerable additional RAM memory requirement for the pointer table.

The next figure illustrates the problem. Three parameters of a PID controller (P, I and D) are contained in the flash area of an ECU. The pointer table already contains the RAM addresses and the parameter values in the RAM are also changed.

Parameter	Flash		Pointertable		RAM	
	Addr.	Content	Addr.		Addr.	Content
P	0x0000100A	0x11	0x000A100A	→	0x000A100A	0x44
I	0x000012BC	0x22	0x000A100B	→	0x000A100B	0x55
D	0x00007234	0x33	0x000A100C	→	0x000A100C	0x66

Figure 63 Pointer table for individual parameters

Calibration concepts are so important because RAM resources are scarce. Large RAM pointer tables make this absurd.
So that a pointer does not have to be created for each individual parameter and the procedure is used as such, the parameters can be combined into structures. This means that only one pointer is required per structure. When the user selects a parameter, not only this parameter but the entire structure associated with it is copied into RAM. The granularity of the structures is of particular importance in this way. If large structures are selected, only a few pointers are required. This in turn means that with the decision for a certain parameter, a correspondingly large structure is copied into the RAM area, and thus the limitation of the calibration RAM can be reached quickly.

Example:
The calibration RAM should have a size of 400 bytes. Four structures with the following sizes are defined in the software:

Structure A: 250 bytes
Structure B: 180 bytes
Structure C: 120 bytes
Structure D: 100 bytes

If the user selects a parameter from structure A, the 250 bytes are copied from the flash to the calibration RAM and the user has access to all parameters located in structure A via XCP. If the calibration task is limited to the parameters of this structure, the calibration RAM is completely sufficient. If the user now selects another parameter, but this is located in structure C, these 120 bytes must also be copied to the calibration RAM. Since the 400 bytes in the calibration RAM are sufficient, the user can access all parameters of structures A and C simultaneously.

If another selected parameter is located in B instead of in structure C, the 180 bytes of structure B would have to be copied to RAM in addition to the 250 bytes of structure A. Since there is not enough space in RAM for this, the user has access to the parameters of structure A, but not to the data of structure B, because the control unit cannot execute the copy command.

3.5.2. Double pointer concept

One disadvantage of the single pointer concept is that memory page switching is not easy to implement. Although the calibration tool could simply write the pointer table completely for page switching, this is not feasible in an arbitrarily short time, so temporary inconsistencies and side effects can result. A tool-transparent implementation would double the memory requirements for the pointer table since a copy of the previous pointer table with the RAM pointers would have to be created when the memory page is switched to flash memory.

For applications with large pointer tables with a transparent implementation or a completely consistent switching, there is the possibility of an extension to a double-pointer concept. To explain this, we return once again to the initial configuration of the RAM.

Figure 60 Initial RAM data represents the pointer table. It is located in the RAM. As already mentioned, this table must be copied from the flash to the RAM. So, this table is located in the flash memory. If you now use another pointer (a table pointer) that points either to the pointer table in RAM or in flash, you arrive at the double pointer solution.

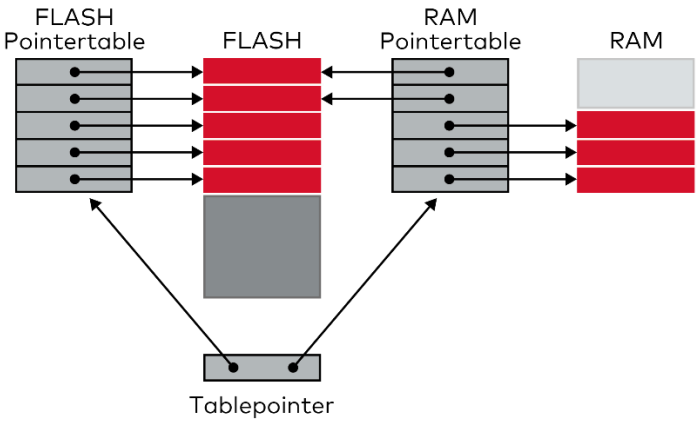


Figure 64 Double pointer concept

The parameter values are initially accessed via the table pointer. If the table pointer points to the pointer table in RAM, the application finally accesses the actual parameters via the contents of the RAM pointer table. The low access speed and the creation of more program code are the disadvantages of this solution.

3.6. Flash pointer-based calibration concept

This method was patented a few years ago by ZF Friedrichshafen under the name "InCircuit2" and has strong similarities with the pointer-based concept according to AUTOSAR. Here, too, the ECU application accesses the parameter data with the help of a pointer table. However, this pointer table is not located in RAM, but in Flash. Changes to the pointer table can therefore only be made by flash programming. A tool transparent implementation is not possible. The advantage is the saved RAM memory for the pointer table. The patent has been revoked in the meantime.

4. Areas of application for XCP

XCP is not only applicable in embedded systems / ECUs. It can basically be used in any system in which the code can be implemented.
The possibility of "external XCP slaves" is explained as a further area of application. They enable powerful access to embedded systems.
The use of debuggers via XCP on embedded systems is also explained.

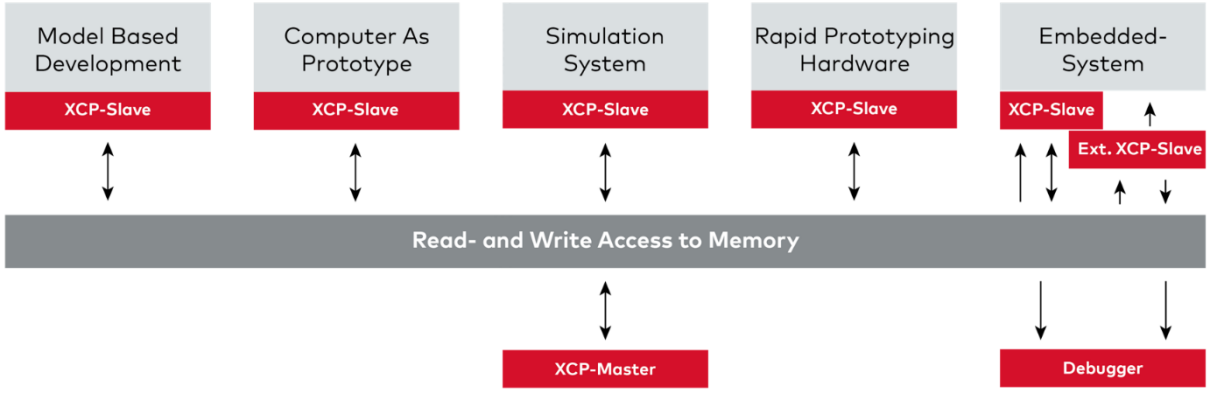


Figure 65 Areas of application and use cases

If you move along the development process, you come across the most diverse approaches to solutions for the development of electronics and software. HIL, SIL and rapid prototyping are keywords that describe different scenarios. These always have a controlled system and a controller in common.

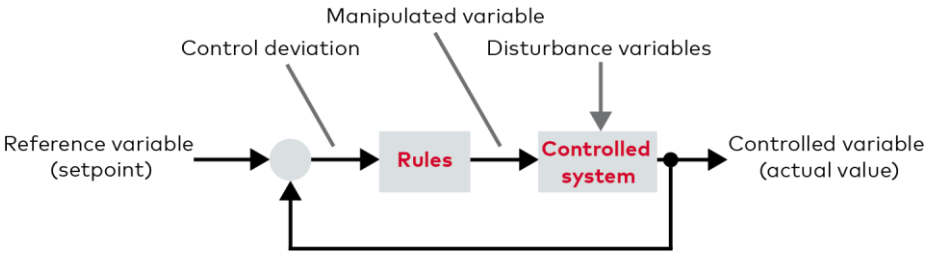


Figure 66 Controlled system (plant) and regulator (controller)

In terms of automotive development, the controller corresponds to the ECU and the controlled system is the physical system that needs to be controlled, such as e-drives, inverters, transmissions, chassis or side mirrors.

The development approaches are roughly subdivided according to whether the controller or the controlled system is real or simulated. This results in a combinatorics, which is described in more detail with the terms SIL, HIL etc.. Different platforms, such as computers, rapid prototyping systems as a runtime environment for the simulated part, come into question.

4.1. MIL: Model in the Loop

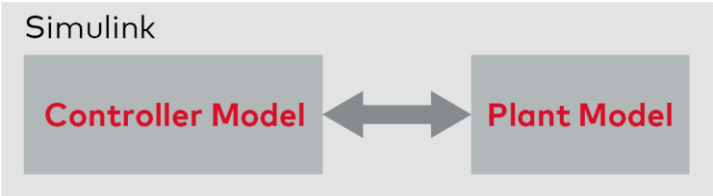


Figure 67 Model-in-the-loop in Simulink

In this development environment, both the controller and the controlled system are simulated as a model. In the example shown, both models run in Simulink as a runtime environment. The options of the Simulink runtime environment are available for analyzing the behavior.

To take advantage of the convenience of a measurement and calibration tool such as CANape at an early stage of development, an XCP slave can be integrated into the controller model. In one authoring step, the slave generates the A2L that matches the model, and the user already has complete operating convenience with visualization of the processes in graphic windows, access to characteristic curves and fields, and much more.

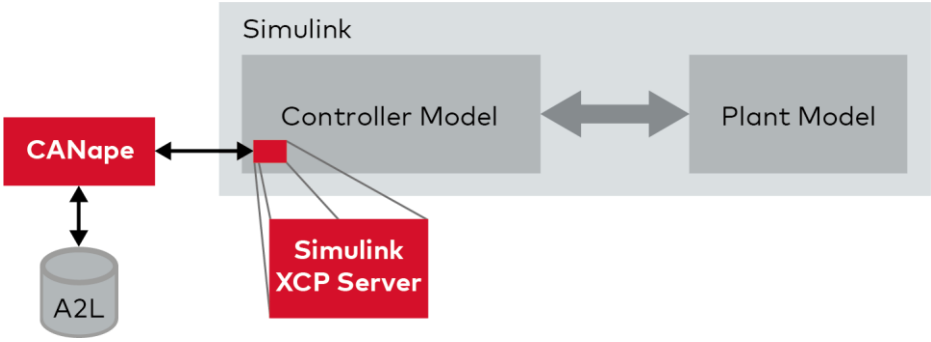


Figure 68 CANape as a measurement and calibration tool with Simulink models

This requires neither a code generation step nor instrumentation of the model. Timestamps are also sent via the XCP. CANape completely adapts to the time behavior of the Simulink runtime environment. Whether the model runs faster or slower than in real time is irrelevant. If, for example, the function developer in the model uses the Simulink debugger to step through the model, CANape still takes the time transmitted by XCP as reference time.

4.2. SIL: Software in the Loop

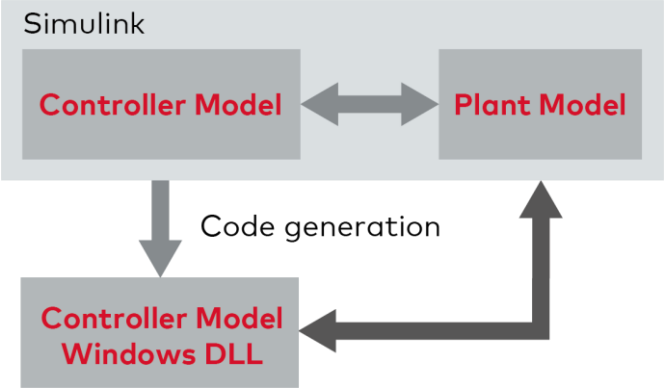


Figure 69 Software-in-the-loop with Simulink environment

In this development step, code is generated from the model of the controller, which is then used in a computer-based runtime environment. Of course, the controller may have been developed without any model-based approach. The controlled system is still simulated. XCP can be used to measure and adjust the controller. If the controller originates from a Simulink model, a code generation step generates C code for a DLL and the associated A2L. If the controller development is based on handwritten code, it is embedded in a C++ project that is delivered with CANape.

After compiling and linking, the DLL is used in the context of CANape. With support of the XCP link, the algorithms in the DLL can be measured and adjusted just as if the application were already integrated in an ECU.

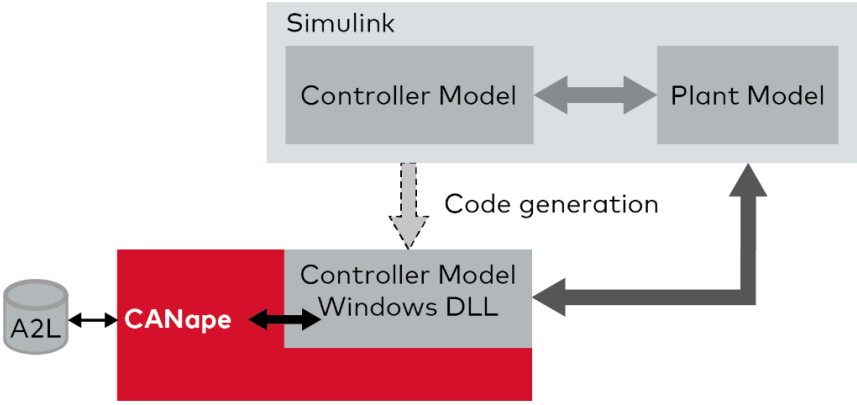


Figure 70 CANape as SIL development platform

4.3. HIL: Hardware-in-the-loop

HIL-systems are available in various forms. The following figure shows the rough concept:

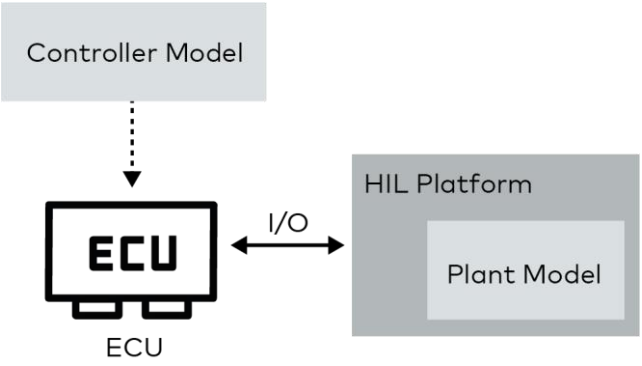


Figure 71 HIL solution

The controller algorithm runs on an embedded platform (e.g. the control unit). The controlled system is still simulated. The controller is connected to the HIL system via I/Os. Since it operates in real time, the HIL platform must also run in real time. Data from the controller algorithm and from the controlled system can be acquired or parameters can be adjusted via XCP.

4.4. RCP: Rapid Control Prototyping

In this development phase, the algorithm runs on real-time hardware. Several platforms can be considered as suitable hardware, from simple evaluation boards to special solutions, depending on which further requirements have to be met. Here, too, integration with XCP helps to establish a vendor-independent tool chain.

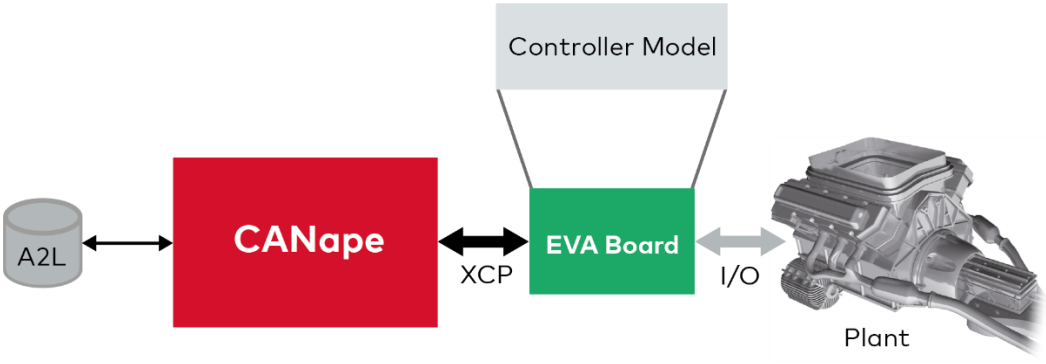


Figure 72 RCP solution

The terms "rapid" and "prototyping" describe the task very well. The aim is to develop a functional prototype as quickly as possible, to use it on the runtime environment, and to test it. This requires simple work steps throughout the entire process.

In the literature, the RCP approach is often divided into two areas: Fullpassing and bypassing.

As shown in Figure 72 RCP solution shown, the entire controller runs on separate real-time hardware. This method is called fullpassing because the entire controller runs on the controller hardware. It must have the necessary I/Os to connect to the controlled system. The technical requirements for the I/Os can very often only be realized with appropriate power electronics.

Not only the I/Os pose a challenge, but often functional elements of the ECU software (e.g. network management) are needed to enable functioning in a more complex network. However, if one uses a complete ECU for rapid control prototyping instead of a general controller platform, the complexity of the flash process, the scope of the overall software, etc. stand in the way of the "rapid" requirement.

In summary: The use of a complete control unit as the controller's runtime environment offers the advantage that the necessary hardware and software infrastructure for the controlled system is available. However, the disadvantage lies in the high complexity.
The concept of bypassing was developed to take advantage of the ECU infrastructure without the disadvantages of high complexity.

4.5. Bypassing

Bypassing is based on acquiring data from the ECU, processing it outside the ECU, and writing the result back to the ECU. Since both measuring and writing to the ECU must be done synchronously with the ECU processes, DAQ and STIM mechanisms are used. So at least two DAQ lists are needed: One with the direction DAQ (from slave to master) and another with the direction STIM (from master to slave).

In the Figure 73 Basic principle of bypassing the control unit is connected to the controlled system. The necessary I/Os and software components are available in the ECU. In the bypassing hardware, an algorithm A1 is running, which is present in the ECU in the version A. A' is a new variant of the algorithm and is now to be tested on the real controlled system.

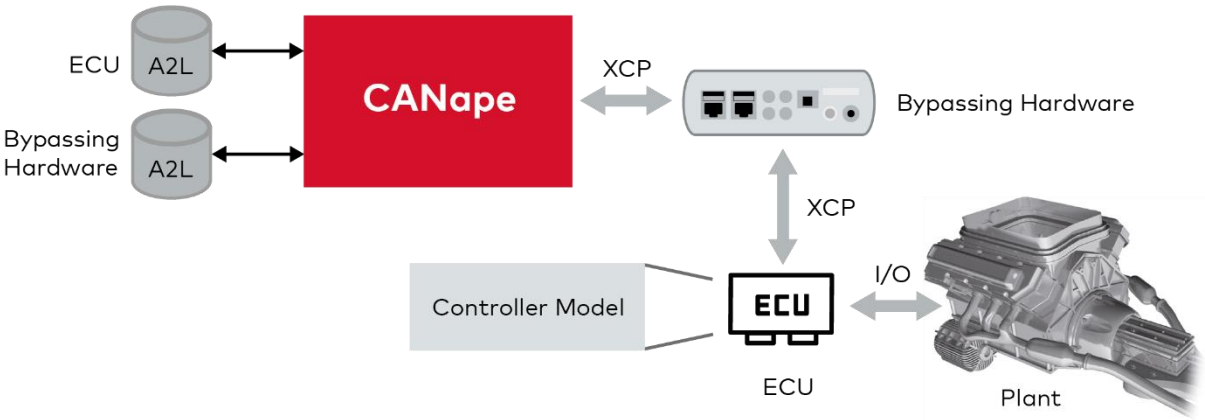


Figure 73 Basic principle of bypassing

The bypassing hardware and the ECU are connected to each other via XCP. On the one hand, this involves fetching the data required for algorithm A' from the ECU via DAQ, and on the other hand, stimulating the results of A' back into the ECU. The following figure demonstrates the schematic flow:

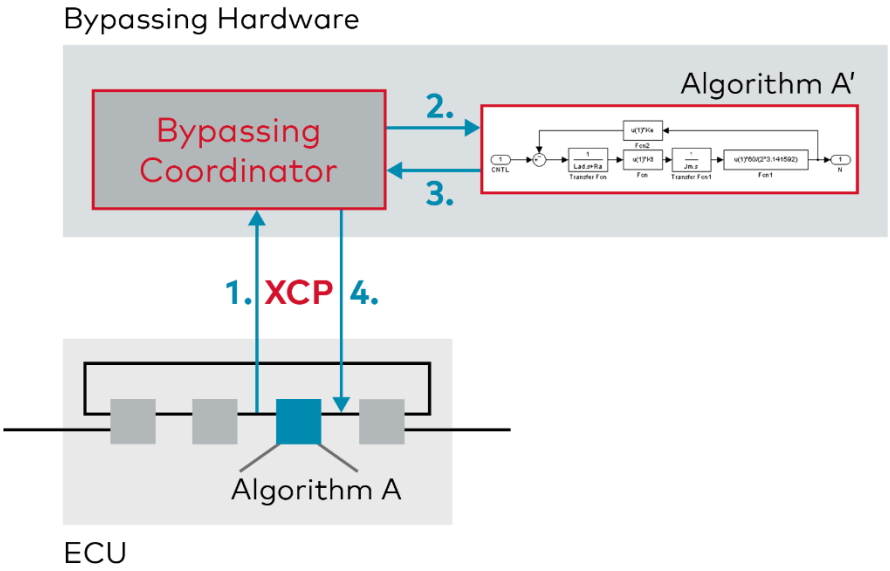


Figure 74 Bypassing sequence

A blue function block is shown in the ECU, in which algorithm A runs. In order that A' can now be used, the data that enters algorithm A as an input variable is measured out of the ECU by DAQ.

Step 1: In the ECU, the data is acquired and sent to the bypassing tool before algorithm A is calculated in the ECU. Normally, the input data to function A and function A' are identical.

Step 2: The data transferred via DAQ is passed to algorithm A'.

Step 3: The results of the calculation of algorithm A' are transferred to the bypassing tool.

Step 4: The data is transferred to the ECU via STIM. During this time, the ECU calculates algorithm A. When the stimulated results are available and the calculation of algorithm A is finished, the values calculated in the ECU are typically overwritten with the stimulated values of algorithm A'.

Thus, the calculation of algorithm A' and not A was used in the overall flow of the ECU. This method allows the combination of the fast ("rapid") exchange of the algorithms on the bypassing hardware with the use of the I/Os and the basic software of the ECU.

The performance limits of the transport protocol naturally also have a limiting effect on the bypass. If short bypassing cycles are required, access via DAQ and STIM into the ECU can also be made via debugging or trace interfaces of the controller. The Vector VX1000 measurement and calibration hardware converts the data from the controller interface into an XCP on Ethernet data stream.

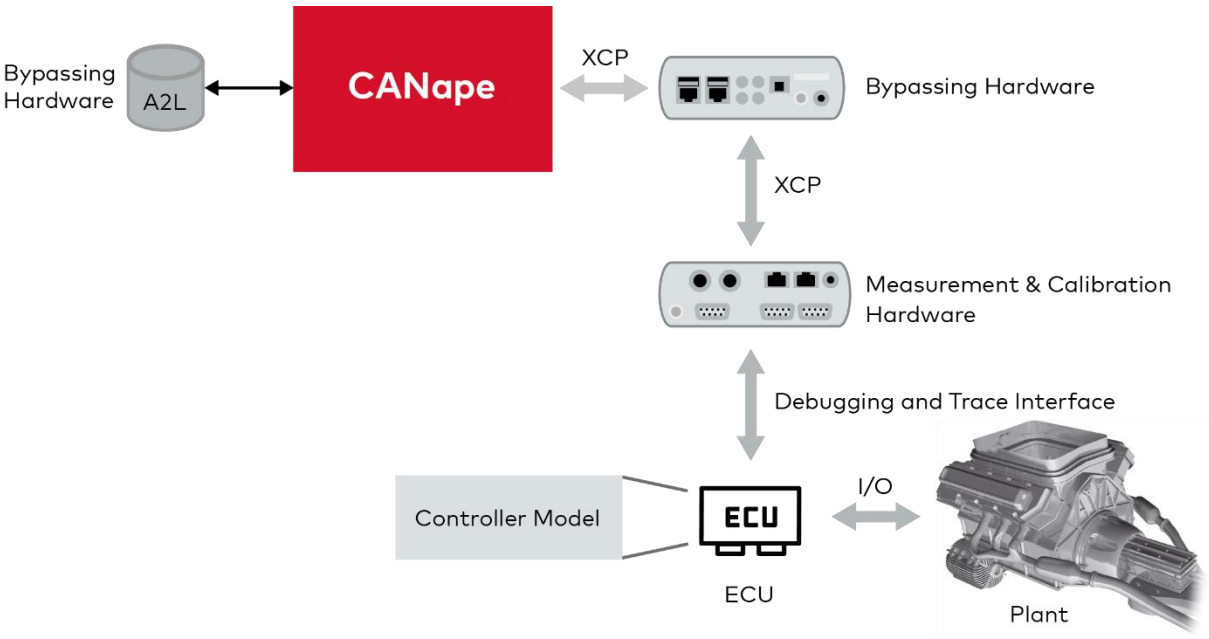


Figure 75 Bypassing with real-time bypassing hardware and fast ECU access

In the illustration, ECU access is via XCP on Ethernet and the bypass algorithm is calculated on a separate bypassing hardware (e.g. VN8900 family) with real-time operating system. This means that the variance of the calculation time is much smaller than with a calculation on a laptop, since there is no influence on the calculation time by other applications.

4.6. Shortening iteration cycles with virtual ECUs

In order to optimize the algorithm in the ECU with the help of XCP, stimulation with data is necessary. The process can take place during test drives in the ECU. However, XCP also provides another solution in which the algorithm does not run on an ECU, but in the form of executable code on the computer or as a model in Simulink in the form of a "virtual ECU". This virtual ECU does not have to run in real time, since in this case there is no coupling to a real system. It can run much faster - depending on the computing power of the PC.

The algorithm is stimulated by a previously recorded measurement file that contains all signals needed as input signals to the algorithm. The connection to CANape is established via XCP. The user can perform parameterization and measurement configuration. He then starts the execution. In the process, the data from the test drive is fed into the algorithm as a stimulus and, at the same time, the desired measured variables are measured out of the application and, in turn, stored in a measurement file.

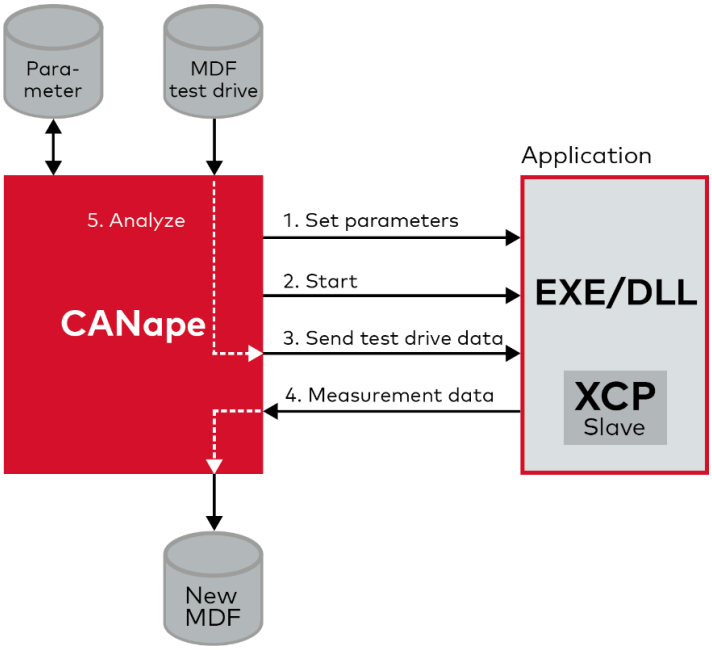


Figure 76 Short calibration cycles with virtual controllers

After the end of the calculation, a new measurement file is available to the user for analyzing the behavior of the control unit. The time length of the new measurement file corresponds exactly to the length of the input measurement file. If the duration of a test drive is one hour, for example, the algorithm on the computer calculates the entire test drive in a few seconds. Then there is a measurement result corresponding to a test of one hour duration. From the analysis of the data, the user makes decisions for parameterization and the iteration cycle starts again.

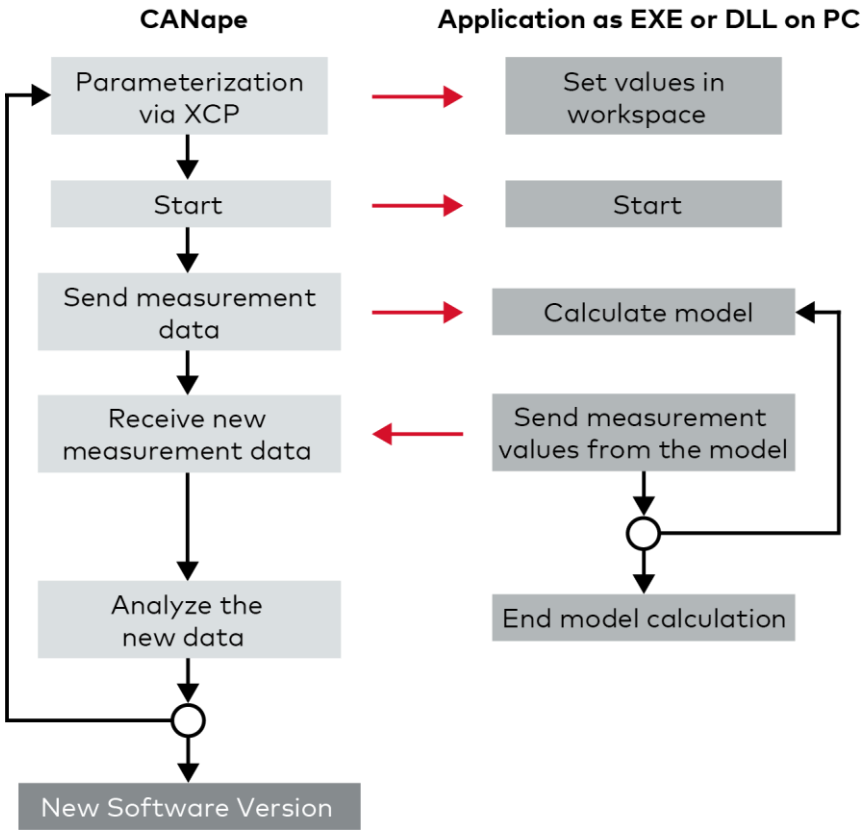


Figure 77 Process flow with virtual control units

In addition to the shortening of the iteration cycles, the algorithm is always stimulated with the same data. This makes the results with different parameters much more comparable, since the results are only influenced by the different parameters.

This process can of course be automated. CANape's integrated script language performs an analysis of the measurement results, from which parameter changes are derived and executed automatically. Control by an external optimization tool such as MATLAB is also possible via the CANape automation interface.

4.7. External XCP slaves

In addition to integrating an XCP slave into an embedded system, it is also possible to work with external XCP slaves.

Vector's VX1000 hardware product family is an example of this. The solution is based on two components: POD and the VX1000 base module.

The POD (Plug-on-Device) is installed in the embedded system and connected to the debugging or trace interface of the controller. The POD serializes the data and sends it to the remote base module.

A VX1000 base module receives the data from the POD. Via Ethernet, the base module offers two XCP on Ethernet slaves. If an XCP master connects to one of the XCP slaves, it has full access to the data in the controller.
In addition to measurement, all other use cases such as calibration and flashing are also supported.

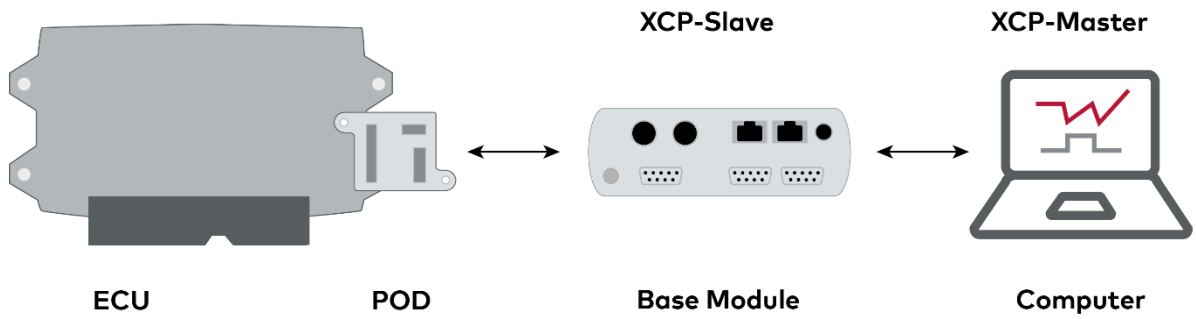


Figure 78 Integrated POD in the control unit with external XCP slave.

The VX1000 product family offers many different POD and base modules. You can find more information here.

4.8. Software debugging via XCP

The ability to debug a software application in the embedded system is elementary for development. Debugging of development and near-series ECUs is done via the debugging interfaces of the microcontrollers. However, this always requires a hardware connection to the ECU. For various reasons, however, the use of the hardware connection can be very difficult. For example, if a POD is already installed and the debugging interface of the controller is occupied or if the ECU has been installed in the vehicle and is no longer accessible.

For these cases, the XCP standard was extended in version 1.5 to include the topic of software debugging via XCP. This created significant added value for the development of embedded systems:

- Uniform hardware for all application areas reduces costs and complexity.
- Robust solution with automotive temperature range.
- The debugging configuration does not require any extension of the ECU software.

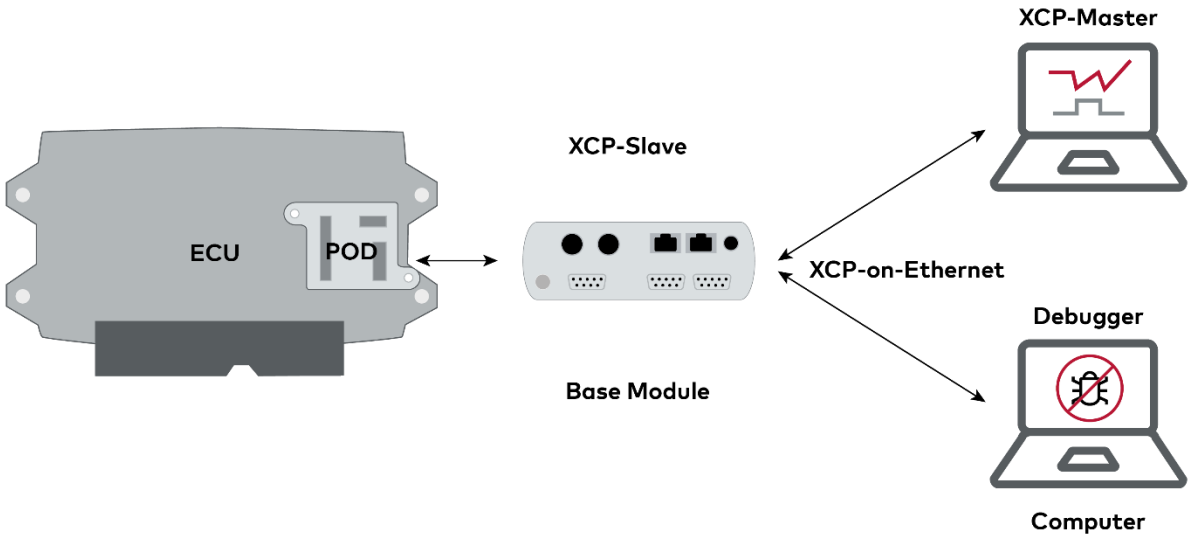


Figure 79 Debugger via XCP

5. Exemplary XCP implementation

The XCP protocol layer is based on the XCP transport layer, which in turn is based on the actual bus communication.

The software interface between the transport and protocol layers is very simple. It includes only a few functions:

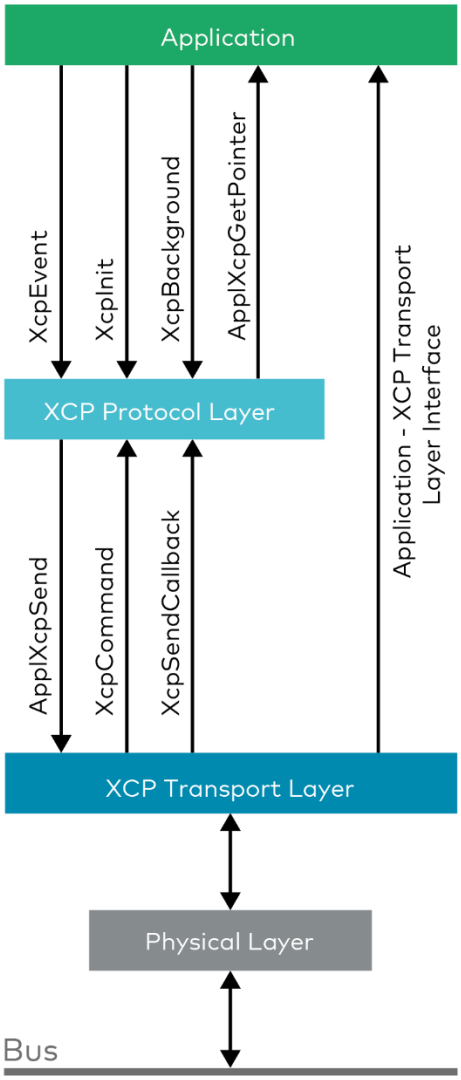


Figure 80 Integration of the XCP slave in the ECU code

- If the slave receives an XCP frame via the bus, it first arrives in the communication driver, which forwards the frame to the XCP transport layer. The transport layer informs the protocol layer about the frame via the function call XcpCommand().
- If the XCP protocol layer wants to send information (e.g. a response to an XCP command from the master or a DAQ packet), the content is forwarded to the transport layer via the ApplXcpSend() function.
- The transport layer notifies the protocol layer that the information was successfully sent by calling the XcpSendCallBack() function.

The interface between the application and the protocol layer can also be implemented using only four functions:

- The application activates the XCP driver with the help of XcpInit(). This call is made once during the startup process.
- XcpEvent(y) informs the XCP driver that event y has occurred.
- The XcpBackground() call allows the XCP driver to perform certain activities in the background (e.g. calculating a checksum).
- Since addresses are always defined as 40-bit values in A2L files (32-bit address, 8-bit address extension), the XCP driver uses the ApplXcpGetPointer() function to get a pointer from an A2L-compliant address.

These interfaces are sufficient to integrate the basic functionalities for measuring and adjusting. Only for extended functions, such as page switching, identification or seed & key, are further interfaces required.

5.1. Description of the functions

void XcpInit (void)

Task:
Initialization of the XCP driver

Description:
With XcpInit() the application activates the XCP driver. This command must be executed exactly once before any XCP driver function may be called.

void XcpEvent (BYTE event)

Task:
The application informs the XCP driver which event has occurred. Each event is assigned a unique event number.

Description:
When compiling the measurement configuration in the measurement and calibration tool, the user selects which measurement values are to be acquired synchronously with which event. The information about measured values and events comes from the A2L. The desired measurement configuration is communicated to the XCP driver in the form of DAQ lists.

Example of an event definition of an engine control unit:

```
XcpEvent (1);      // Event 1 represents the 10 ms task
XcpEvent (2);      // Event 2 represents the 100 ms task
XcpEvent (5);      // Event 5 represents the 1-ms task
```

XcpEvent (8); // Event 8 is used for ignition angle synchronous measurements.

BYTE XcpBackground (void)

Task:
Performs background activities of the XCP driver.

Description:
The function should be called periodically in a background or idle task. It is used by the XCP driver for checksum calculation, for example, since the calculation of a longer checksum in XcpCommand() could take an unacceptably long time. With each call to XcpBackground() a partial checksum of 256 bytes is calculated. The duration of a checksum calculation therefore depends on the call frequency of XcpBackground(). Otherwise, there are no requirements for the call frequency and periodicity. The return value 1 indicates that a checksum calculation is currently running.

void XcpCommand (DWORD* pCommand)

Task:
Interpretation of an XCP command.

Description:
Each time the transport layer receives an XCP frame, this function must be called. The parameter is a pointer to the frame.

void ApplXcpSend (BYTE len, BYTE *msg)

Task:
Passing a frame to be sent to the transport layer.

Description:
The protocol layer sends the information via this call, which is then sent to the master via the transport layer. A handshake procedure between protocol and transport layer is realized via the call XcpSendCallBack.

BYTE XcpSendCallBack (void)

Task:
With the callback, the transport layer informs the protocol layer about the successful sending of the last frame passed to ApplXcpSend().

Description:
The protocol layer will not call any ApplXcpSend() command until it is indicated via XcpSendCallBack() that the previous information was sent successfully.

XcpSendCallBack() returns the value 0 (FALSE) if the XCP driver is idle. If there are more frames to be sent, ApplXcpSend() is called directly from XcpSendCallBack().

BYTE *ApplXcpGetPointer (BYTE addr_ext, DWORD addr)

Task:
Conversion of an A2L compliant address into a pointer.

Description:
The function maps the 40-bit A2L-compliant addressing (32-bit address + 8-bit address extension) sent by the XCP master to a valid pointer. The address extension can be used, for example, to distinguish between different address ranges or memory types.

5.2. Parameterization of the driver

The XCP driver is scalable and parameterizable in many ways to accommodate a wide variety of functional scopes, transport protocols, and target platforms. In the simplest case, this looks like this:

```
/* Define protocol parameters */
#define kXcpMaxCTO 8 /* Maximum CTO Message Length */
#define kXcpMaxDTO 8 /* Maximum DTO Message Length */
#define C_CPUTYPE_BIGENDIAN /* Byte order Motorola */

/* Enable memory checksum */
#define XCP_ENABLE_CHECKSUM
#define kXcpChecksumMethod XCP_CHECKSUM_TYPE_ADD14

/* Enable calibration */
#define XCP_ENABLE_CALIBRATION
#define XCP_ENABLE_SHORT_UPLOAD

/* Enable data acquisition */
#define XCP_ENABLE_DAQ
#define kXcpDaqMemSize (512) /* Memory space reserved for DAQ */
#define XCP_ENABLE_SEND_QUEUE
```

For a CAN transport layer, the appropriate CTO and DTO size of eight bytes is set. The driver must know whether it runs on a platform with Motorola or Intel byte order, which in this case is a Motorola CPU (big-endian). The remaining parameters activate the functionalities: Measure, Adjust and Checksum Calculation. For the checksum calculation the algorithm is set (here summation of all bytes into a dword) and for measuring the size of the available memory is specified (here 512 bytes). The memory is mainly needed to store the DAQ

lists and to buffer the data during the measurement. The size therefore determines the maximum possible number of measurement signals.

6. Overview of the protocol development

The following overview shows some of the significant further developments of the standard. The list is not complete.

6.1. XCP Version 1.1 (2008)

- Description of the same XCP interface via two different physical interfaces within the same A2L (e.g. "XCP on Vehicle CAN" and "XCP on Calibration CAN")
- New command WRITE_DAQ_MULTIPLE allows to speed up the configuration of the slave. Successive ODTs in a DAQ list can be communicated in one step.
- High time synchronization via "TIMESTAMP_EVENT". Timestamp information is communicated by the slave. Triggering can be initiated via an external synchronization line.
- Compression of embedded A2L files

All extensions are optional. Thus, XCP 1.1 is compatible with XCP 1.0.

6.2. XCP Version 1.2 (2013)

- Parameters are in the A2L for the definition of the required ECU resources by XCP DAQ measurement configurations (e.g. RAM consumption, CPU execution time, required transmission bandwidth for transport via CAN or Ethernet). XCP Master can access the parameters and calculate the resource consumption of the measurement and warn the user if it is exceeded, for example.
- Prioritization control by the master to transport the measurement data via CAN. The objective here is to avoid disturbing the necessary communication flow of the vehicle CAN as much as possible.
- Calculation of the required bandwidth and limits when transporting data via TCP or UDP.
- Description of XCP on CAN FD.

All extensions are optional. Thus, XCP 1.2 is compatible with XCP 1.1.

6.3. XCP Version 1.3 (2015)

- Improve time correlation of XCP slaves through multicast solutions that are on the same network.
- Time synchronization between XCP slave timestamp and external clock (e.g. via IEEE1588)
- Checking the bypassing data flow and error handling

All extensions are optional. Thus, XCP 1.3 is compatible with XCP 1.2.

6.4. XCP Version 1.4 (2017)

- Packed DAQ Lists
  - This feature significantly reduces the protocol overhead for cyclic measurement data transfer by omitting timestamps when they can be recovered by the MC tool through a well-defined formula.
- DAQ data consistency improvements
  - The extension provides means for describing logical DAQ data consistency, e.g. for multi-core processors.
- Calibration page change and resource state changes forced by slave
  - This function is added to the ECU state model to reflect calibration page switches forced by the ECU or changes in the DAQ resource state, e.g. due to a reset.
- POD standard specific requirements
  - Extension of the commands
  - The number of possible protocol commands is extended. This was also made necessary by the new POD standards.
- GET_VERSION command
  - This command provides the decimal place of the protocol version as additional information, which was not accessible before.
- Alignment of XCP on Ethernet packets
  - By inserting fill bytes in the XCP on Ethernet tail it is possible to align XCP on Ethernet packets within a UDP packet to e.g. WORD boundaries
- Enhancements to the ODT optimization types
  - A "strict mode" for ODT optimization was introduced.
- Extension of the START_STOP_SYNC command
  - The "prepare for start selected" mode has been added to avoid a race condition when starting DAQ lists for data acquisition
- XCP on Ethernet slave IP address configuration
  - The MC-Tool can actively set the IP address of an XCP slave, such as an MC hardware interface
- XCP on CAN supports efficient data transmission of measurement data
  - The adaptation supports efficient data transmission of measurement data larger than the possible CAN payload by allowing splitting in certain cases
- XCP on CAN supports efficient data transmission of measurement data
  - The standard supports efficient data transmission of measurement data larger than the possible CAN payload by allowing splitting in certain cases

All extensions are optional. Thus XCP 1.4 is compatible to XCP 1.3.

6.5. XCP Version 1.5 (2017)

> Software Debugging over XCP (DBG)

All extensions are optional. Thus XCP 1.5 is compatible to XCP 1.4.

List of abbreviations

A2L	File Extension for an ASAM 2MC Language File
AML	ASAM 2 Meta Language
ASAM	Association for Standardization of Automation and Measuring Systems
BRS	Bit Rate Switch
BYP	Bypassing
CAL	Calibration
CAN	Controller Area Network
CCP	CAN Calibration Protocol
CMD	Command
CS	Checksum
CTO	Command Transfer Object
CTR	Counter
DAQ	Data Acquisition, Data Acquisition Packet
DBG	Debugging
DLC	Data Length Code
DTO	Data Transfer Object
ECU	Electronic Control Unit
EDL	Extended Data Length
ERR	Error Packet
ESI	Error State Indicator
EV	Event Packet
FIBEX	Field Bus Exchange Format
LEN	Length
MCD	Measurement Calibration and Diagnostics
MTA	Memory Transfer Address
ODT	Object Descriptor Table
PAG	Paging
PGM	Programming
PHY	Short form of Physical Layer or designation of the chip for implementation, e.g. Ethernet PHY
PID	Packet Identifier
RAM	Random Access Memory
RES	Command Response Packet
SERV	Service Request Packet
SPI	Serial Peripheral Interface
STD	Standard
STIM	Data Stimulation Packet
TCP/IP	Transfer Control Protocol/Internet Protocol
TS	TimeStamp
UDP/IP	Unified Data Protocol/Internet Protocol
USB	Universal Serial Bus
XCP	Universal Measurement and Calibration Protocol

Download	Sending data from master to slave
Upload	Send data from slave to master

Bibliography

XCP is specified by ASAM (Association for Standardization of Automation and Measuring Systems).

You can find details about the protocol and ASAM at www.asam.net.

Web addresses

Standardization bodies:

> ASAM, XCP protocol-specific documents, A2L specification, **www.asam.net**

Development software supplier:

> Vector Informatik GmbH, demo version CANape, free and freely available XCP driver (basic version), comprehensive information on ECU calibration, test and simulation, **www.vector.com**

List of Figures

Figure 1 Classic process flow when developing applications for embedded systems 6
Figure 2 Process flow when developing applications for embedded systems with the XCP standard 6
Figure 3 The interface model of ASAM 7
Figure 4 Universal access via the development process 8
Figure 5 Division of the XCP protocol into protocol and transport layers 9
Figure 6 XCP communication model with CTO and DTO 13
Figure 7 Standard CTO exchange 14
Figure 8 Master Block Transfer Mode 15
Figure 9 Slave Block Transfer Mode 15
Figure 10 Interleaved Communication Mode 16
Figure 11 XCP package 16
Figure 12 The package identification 17
Figure 13 Overview of the XCP Packet Identifier (PID) 18
Figure 14 Counter Field 18
Figure 15 Timestamp 18
Figure 16 Data field in XCP package 19
Figure 17 Overview of the structure of the CTO package 21
Figure 18 Trace representation of an calibration process 26
Figure 19 Transfer of a parameter set file into the RAM of the control unit 28
Figure 20 Address information of the parameter "Triangle" from the A2L file 29
Figure 21 Polling communication in CANape's Trace Window 30
Figure 22 Events in the ECU 31
Figure 23 Examples of event definitions 32
Figure 24 Assignment of "Triangle" to the possible events in the A2L 32
Figure 25 Selection of events (Measurement mode) per measurand 32
Figure 26 Excerpt from the CANape Trace window of a DAQ measurement 33
Figure 27 Assignment of RAM addresses to ODT 34
Figure 28 DAQ list with three ODTs 35
Figure 29 Transmission of signals 1-10 with timestamp 35
Figure 30 Static DAQ lists 36
Figure 31 Dynamic DAQ lists 37
Figure 32 Transmission of five measured values of signals S1 and S2 in one ODT each 37
Figure 33 Event for DAQ and STIM 38
Figure 34 Structure of XCP package for DTO transmissions 39
Figure 35 Identification field with absolute ODT numbers 39
Figure 36 ID field with relative ODT and absolute DAQ numbers (two bytes) 40
Figure 37 ID field with relative ODT and absolute DAQ numbers as well as fill byte (total four bytes) 40
Figure 38 First stage of time synchronization 43
Figure 39 Second stage of time synchronization 43

Figure 40 Free-running clock in XCP slave 46
Figure 41 XCP Slave Clock synchronized with Grandmaster Clock 46
Figure 42 Slave clock syntonized with Grandmaster Clock 47
Figure 43 Free running clock combined with synchronized clock 47
Figure 44 Two clocks with triggered EV_TIME_SYNC shipping 48
Figure 45 Determination which CAN node sends which frames 49
Figure 46 Representation of a CAN network 50
Figure 47 Example of XCP on CAN communication 51
Figure 48 Representation of a CAN FD frame 52
Figure 49 Nodes K and L are redundantly connected to each other 53
Figure 50 Communication via slot definition 53
Figure 51 Representation of a FlexRay communication matrix 54
Figure 52 Representation of the FlexRay LPDUs 55
Figure 53 Assignment of XCP communication to LPDUs 56
Figure 54 XCP packet with TCP/IP or UDP/IP 57
Figure 55 XCP on Sxl package 58
Figure 56 Memory representation 60
Figure 57 Illustration of the driver setting for the Flash area 62
Figure 58 Parameters in a calibration window 67
Figure 59 Signal display over time 67
Figure 60 Initial RAM data 75
Figure 61 Initial situation after booting 79
Figure 62 Pointer change and copy to RAM 80
Figure 63 Pointer table for individual parameters 80
Figure 64 Double pointer concept 82
Figure 65 Areas of application and use cases 83
Figure 66 Controlled system (plant) and regulator (controller) 83
Figure 67 Model-in-the-loop in Simulink 84
Figure 68 CANape as a measurement and calibration tool with Simulink models 84
Figure 69 Software-in-the-loop with Simulink environment 85
Figure 70 CANape as SIL development platform 85
Figure 71 HIL solution 86
Figure 72 RCP solution 86
Figure 73 Basic principle of bypassing 88
Figure 74 Bypassing sequence 88
Figure 75 Bypassing with real-time bypassing hardware and fast ECU access 89
Figure 76 Short calibration cycles with virtual controllers 90
Figure 77 Process flow with virtual control units 91
Figure 78 Integrated POD in the control unit with external XCP slave. 92
Figure 79 Debugger via XCP 93
Figure 80 Integration of the XCP slave in the ECU code 94

Appendix - XCP Solutions at Vector

The XCP standard was significantly shaped by Vector. The extensive know-how and many experiences flowed into the comprehensive XCP support:

Tools

- CANape: The XCP Master. With integrated ASAP2 Studio as A2L Editor.
- ASAP2 Tool-Set: Complete tool chain for batch-controlled generation of A2L description files. Including ASAP2 Studio as A2L editor.
- CANoe.XCP: extends CANoe with the ability to communicate with embedded systems via XCP for test and analysis tasks.
- VX1000: Comprehensive hardware family for implementing external XCP slave solutions. The hardware is connected to the embedded system via debugging and trace interfaces of the controllers and processors. In the direction of the measurement and calibration tool, communication takes place via XCP.

Embedded software

- XCP Basic driver for Ethernet (Windows and Linux) free for download.
- XCP Professional: ideal solution for those who want to purchase an integrated XCP driver along with the basic Vector software.
- MICROSAR XCP - contains the functional scope of XCP Professional and is based on the AUTOSAR specifications. Available for the Vector basic software MICROSAR.

Services

- Consulting on the use of XCP in your projects
- Support for the integration of XCP

Trainings

- On the Vector E-Learning platform you will find a free training course on XCP.
- The Vector Academy also offers an XCP seminar.

Subject index

A

A2L 67
Adress Extension 26, 30
ALLOC_ODT 36
AML 21, 69
ASAP2 Tool-Set 71

B

Bandwidth optimization 30
Bypassing 40, 87

C

CAL 13
CAN 49
CAN FD 51
CMD 13, 21
CTO 12
CYCLE_REPETITION 54

D

DAQ 13, 29, 30
DAQ List Dynamic 36
DAQ List Predefined 36
DAQ List Static 35
DATA Field 19
DBG 13
DOWNLOAD 26
DTO 12

E

ERR 13, 24
EV13, 25
EV_TIME_SYNC 45
Event 31

F

Fash memory 10
Fibex 53
Flash memory 59, 60
FLX_CHANNEL 54
FLX_LPDU_ID 54
FLX_SLOT_ID 54
Fullpassing 87

G

GET_CAL_PAGE 61
GET_COMM_MODE_INFO 64
GET_DAQ_CLOCK_MULTICAST 45
GET_DAQ_EVENT_INFO 64
GET_DAQ_PROCESSOR_INFO 64
GET_DAQ_RESOLUTION_INFO 64
GET_STATUS 64

H

HIL 86

L

LIN 59
LPDU 54

M

MAX_CTO 17
MAX_DTO 17
MIL 84

O

OFFSET 54

P

Packed Mode 37
Page switching 61
Payload 34
PGM 13
PID 17
POD 91
Polling 29

R

RAM 11
Reboot 28
RES 13, 24
RESUME 65

S

Seed-and-Key 65
SEGMENT_NUMBER 60

SERV 13, 25
SET_CAL_PAGE 61
SET_MTA..... 26
SHORT_UPLOAD..... 27, 29
SIL..... 85
Status Information 14
STIM 13, 29, 38
SxI 58

T

TCP/IP..... 56
TIME_CORRELATION_PROPERTIES 45, 46

U

UDP/IP..... 56
USB 58

V

VX1000..... 89
VX1000 Base Module..... 91